
DESIGN AND DEVELOPMENT OF ACOUSTIC COMMUNICATION TECHNIQUES FOR UNDERWATER NETWORKS

D4 – Final Report

Version 1.1
July 29, 2025

Mandar Chitre, Gabriel Chua, Li Kexin, Wu Shuangshuang
ARL, National University of Singapore

Shiraz Shahabudeen
Subnero



ARL Project Report

CONTENTS

1	Introduction	1
2	MIMO Communications	3
2.1	Theory	3
2.2	Experiment	6
2.3	Discussion	11
3	Blind Source Separation	13
3.1	Introduction	13
3.2	Experiment	14
3.3	Discussion	17
4	Acomms Experiment 2023	19
4.1	Introduction	19
4.2	Experiment	19
4.3	Signals	20
4.4	Analysis	21
5	VLF Windowed OFDM	31
5.1	Introduction	31
5.2	Experiment	31

5.3	Discussion	34
6	Differentiable Forward Error Correction	35
6.1	Introduction	35
6.2	System model	36
6.3	Transmission frames	39
6.4	Experiment	40
6.5	Discussion	41
7	Digital Twin for Link Adaptation	45
7.1	Introduction	45
7.2	Generative AI based digital twin	46
7.3	Simulation	53
7.4	Experimental validation	57
7.5	Communication performance	64
7.6	Discussion	65
8	Link Adaptation	67
8.1	Introduction	67
8.2	Digital twin based link tuning	68
8.3	Physics-informed AMC	70
8.4	Simulation	75
8.5	Experiment	82
8.6	Discussion	84
9	Summary & Recommendations	87
9.1	Research contributions & findings	87
9.2	Recommendations & future directions	89
	References	92

CHAPTER 1

INTRODUCTION

This is the final report for the MUSIQUE project than we undertook over the past 3 years, starting 25 March 2022. The key project objectives were:

- To conduct and participate in joint acoustic communications (ACOMMS) experiments with the US Office of Naval Research (ONR), thus fulfilling the technical requirements for collaboration and scientific exchange on ACOMMS techniques with the US. The experiments included new waveforms to be tested.
- To enhance underwater ACOMMS capacity for multiuser communication networks via blind source separation (BSS) and multi-input multi-output (MIMO) communications.
- To enhance ACOMMS performance in challenging environments with machine learning (ML) -based techniques, including the development of differentiable forward error correction (D-FEC) decoders and rapid link adaptation based on a small number of channel soundings.

Deliverable D1 outlined the experimental plan for the joint ACOMMS experiment (AcommsEx 2022) in Singapore. The experiment was successfully completed in July 2022, and the results were reported in deliverable D2. The results were also presented during joint US-Singapore MHCE workshop 3. We also participated in an additional (not in the original scope of work for the project) experiment (AcommsEx 2023) conducted in Panama City, US in November 2023. The data from that experiment was analyzed as a supplementary report was submitted to MINDEF in 2024. As this was not part of a formal deliverable, we include this report as Chapter 4 in this deliverable. During AcommsEx 2022 and 2023, we developed and tested a new robust communication technique called windowed OFDM (W-OFDM). The details of

the technique are outlined in deliverable D2. This technique was tested in the Popoto MF (25–33 kHz) band during those experiments. We subsequently tested this technique in the Subnero MF (20–30 kHz) and VLF (4.5–6.5 kHz) bands. The results are included in Chapter 5.

The main focus of this deliverable, however, is on MIMO communications (Chapter 2), BSS (Chapter 3), D-FEC decoding (Chapter 6), and rapid link adaptation (Chapters 7 to 8). The algorithms for some of these techniques were developed and simulated in deliverable D3. However, some of them were improved significantly in the last year, and so those are described again in detail in this deliverable. All techniques were tested in the Subnero MF band, and where possible, also in the Subnero VLF band, in experiments in November 2024, February 2025 and March 2025. The results from these experiments are presented in the respective chapters.

CHAPTER 2

MIMO COMMUNICATIONS

2.1 Theory

2.1.1 Background

Communication systems with multiple transmit and receive antennas can theoretically benefit from multi-input multi-output (MIMO) communication. It has been shown that a communication system with M transmitting antennas and N receiving antennas has $\min(M, N) \times$ the channel capacity of a communication system with a single transmitting and receiving antenna. With appropriate processing, it is possible to achieve MIMO gains from this increased channel capacity. Terrestrial wireless communication systems such as 4G and WiFi use MIMO gains to improve network throughput.

Traditional MIMO processing assumes transmissions from M transmit antennas arrive synchronously at the N receive antennas. For radio communications, this assumption is a good approximation of reality, as the speed of radio waves ensures that the time difference due to different path lengths between antennas is negligible. In underwater communication systems, however, the low propagation speed of acoustics can lead to significant violations of this assumption. In order to use conventional MIMO processing in underwater acoustic communication systems, we require the symbol length to be much more than the arrival time spread across all transmit-receive pairs. While this would lead to very low data rates in single carrier communication, OFDM is well suited to meet this constraint. In OFDM, the symbol rate is low and each symbol (OFDM block) has a long duration. To achieve high data rate, OFDM blocks carry information on many sub-carriers, making the bits/symbol be large. Subcarrier modems use OFDM for high-speed communication, and so are well placed to implement MIMO processing.

2.1.2 Terminology

There is much confusion on the terminology used related to single/multiple input/output systems. When categorizing a system, the input and output refers to the input/output of the channel, and not of the communication system. So a single-input multiple-output system has one transmitter and multiple receivers. With this in mind, we use the following terminology consistently in this project:

- MIMO – Multiple Inputs Multiple Outputs (the focus of this chapter)
- SIMO – Single Input Multiple Outputs (aka spatial diversity)
- SISO – Single Input Single Output (traditional single transducer modems)

Our focus for MIMO is on increasing the throughput of the modems, and therefore we harness the spatial multiplexing gain that MIMO offers. This is in contrast to MIMO processing that may be used for improving robustness through precoding or diversity coding. We use SISO and SIMO as baseline methods for benchmarking performance gains. Subnero modems implement SISO and SIMO (with selection diversity or diversity combining) out-of-the-box today.

2.1.3 Spatial multiplexing

Spatial Multiplexing enables us to transmit multiple low-rate streams from multiple transmit antennae simultaneously, and decode them at the receiver in parallel. The receiver array with full Channel State Information (CSI) is able to invert the channel matrix and decode the individual streams, although the transmissions collide (arrive at the same time) at the receiver array. Knowledge of CSI is only required at the receiver, and not at the transmitter. For a SISO OFDM system with sufficient cyclic prefix and proper Doppler compensation, a received PSK symbol on each sub-carrier is a phase-rotated version of the transmitted symbol on that sub-carrier. This makes channel equalization simple, as the channel can be represented as a single complex channel coefficient per sub-carrier. For a MIMO OFDM system, we have one such complex channel coefficient for every transmit-receive antenna pair and sub-carrier. When we assemble all the complex channel coefficients into a matrix, we call that matrix the *channel matrix*. In OFDM, each sub-carrier has its own channel matrix. For spatial multiplexing to work, we require the channel matrix to be invertible, i.e., have full rank. Physically, this means that every transmit-receive pair must see a slightly different channel. In practice, in presence of multipath arrivals from surface and bottom reflections, we expect this condition to be generally met provided the transmit array and receive array has sufficient inter-antenna spacing. If the condition is not met for some sub-carriers, errors may be introduced at the receiver, but they may be corrected through error correction codes.

One of the key challenges in implementing MIMO is that of obtaining CSI at the receiver. If the channel is quasi-static, one may obtain CSI by estimating the channel matrices $\mathbf{H}_i$ at each carrier i by transmitting M pilot OFDM blocks with known symbols on all carriers and observing the received symbols. The channel matrices may then be used by subsequent OFDM blocks as CSI. The method outlined in deliverable D3 is based on this approach. Unfortunately, this approach failed in practice (during preliminary tests conducted in November 2024 at St. Johns Island) as the channel varies between OFDM blocks sufficiently for the quasi-static assumption to be invalid. We therefore had to develop a novel method of estimating all channel matrices from a single OFDM block with a small number of pilot carriers. The matrices could then be used to decipher the data from all non-pilot carriers in the same block, thus circumventing the need for multiple OFDM blocks to share the same CSI.

2.1.4 Method 1

The first method to estimate channel matrices is inspired by the technique outlined in [1]. In this method, each transmitter transmits pilots on different carriers. If a carrier is used as a pilot by any transmitter, that

pilot is left empty (no transmission) on all other transmitters. This simplifies the channel estimation, as we shall see, but is inefficient in terms of data rate. Once the channel matrices at all pilot carriers are obtained, other channel matrices can be obtained via interpolation.

Let $\mathcal{P}_m$ be the set of pilots transmitted by transmitter m . The channel coefficients H_{imn} can be estimated at carrier $i \in \mathcal{P}_m$:

$$H_{imn} = Y_{in} X_{im}^*, \quad (2.1)$$

where X_{im} is the pilot symbol transmitted on carrier i by transmitter m , Y_{in} is the corresponding received symbol on receiver n , and $*$ represents a complex conjugation operation. If the pilots are uniformly spaced, the interpolation of channel coefficients to non-pilot carriers can be simplified to an inverse Fourier transform, zero padding, and a Fourier transform:

$$\mathbf{H}_{mn} = \mathcal{F} \left[\{ \mathcal{F}^{-1} [\mathbf{H}_{\mathcal{P}_m mn}], \mathbf{0}_{K-|\mathcal{P}_m|} \} \right] \quad (2.2)$$

where $\{\cdot, \cdot\}$ denotes a vector concatenation, and $|\cdot|$ is the cardinality operation (i.e., $|\mathcal{P}_m|$ is the number of pilots).

For a MIMO system with 3 transmitters ($M = 3$), we use $\mathcal{P}_1 = \{0, 9, 18, \dots\}$, $\mathcal{P}_2 = \{1, 10, 19, \dots\}$, and $\mathcal{P}_3 = \{2, 11, 20, \dots\}$. The data carriers are then $\mathcal{D} = \{3, 4, 5, 6, 7, 8, 12, 13, 14, 15, 16, 17, 21, \dots\}$. We therefore have 3 pilots for every 9 carriers, i.e., 33% pilots and 67% data carriers.

2.1.5 Method 2

The second method to estimate channel matrices tries to improve the data rate by sharing the pilot carriers on all transmitters.

Let $\mathcal{P}$ be the pilot carriers shared by all transmitters. The received symbols $\mathbf{Y}_i$ on each carrier $i \in \mathcal{P}_m$ can be written in terms of the channel matrix $\mathbf{H}_i$ and the transmitted symbols $\mathbf{X}_i$:

$$\mathbf{Y}_i = \mathbf{H}_i \mathbf{X}_i, \quad (2.3)$$

where $\mathbf{H}_i$ is a $M \times N$ matrix, $\mathbf{X}_i$ is a M -vector and $\mathbf{Y}_i$ is a N -vector. Since we have MN unknown channel coefficients and only $\min(M, N)$ equations, this is a severely underdetermined linear system and cannot be inverted to obtain $\mathbf{H}_i$.

We note, however, that the channel matrices $\mathbf{H}_i$ are not independent. Each channel coefficient H_{imn} represents the frequency response at carrier i for the same time domain impulse response h_{lmn} between transmitter m and receiver n :

$$\mathbf{H}_{\mathcal{P}mn} = \mathbf{V} \mathbf{h}_{mn}, \quad (2.4)$$

where $\mathbf{V}$ is a Fourier coefficient matrix of size $|\mathcal{P}| \times L$, and L is the maximum delay spread of the channel:

$$V_{il} = e^{-2\pi i i l} \quad \forall \quad i \in \mathcal{P}, l \in \{0 \dots L-1\}. \quad (2.5)$$

Multiplying (2.4) by X_{imn} , we get:

$$Y_{imn} = H_{imn} X_{imn} = \left(\sum_l V_{il} h_{lmn} \right) X_{imn} = \sum_l (V_{il} X_{imn}) h_{lmn}, \quad (2.6)$$

The tensor $\mathbf{h}$ has LMN unknowns. Equation (2.6) represents a system of $|\mathcal{P}|MN$ simultaneous equations. Since coefficients $V_{il} X_{imn}$ are deterministic and known, and symbols Y_{imn} are observed, as long as $|\mathcal{P}| \geq L$, this system is invertible to obtain tensor components h_{lmn} . Once the tensor $\mathbf{h}$ is known, we get all channel coefficients (including at non-pilot carriers):

$$H_{imn} = \sum_l V_{il} h_{lmn}. \quad (2.7)$$



Figure 2.1: MF and VLF multi-transducer (MT) modems developed in collaboration with Subnero for MIMO testing. Each modem supports up to 3 transmitters that can be synchronously used to transmit at a per-transmitter source level of approximately 185 dB re 1 μ Pa @ 1 m.

With this method, there is no requirement for uniform placement of pilots. However, we do usually prefer uniform pilot placement to ensure that the entire bandwidth of interest is sampled well. If we place a pilot every P carriers, we have 1 pilot carrier for every $P - 1$ data carriers.

For method 1, we used a pilot spacing of 9 carriers. For this method, if we choose $P = 9$, this represents 11% pilots and 89% data carriers – a significant improvement over method 1. However, the estimation procedure in method 2 is more susceptible to noise and so we recommend a higher number of pilots. Even if we choose a conservative $P = 4$, this represents 25% pilots and 75% data carriers, which potentially gives us a higher data rate than method 1.

2.2 Experiment

In order to test MIMO theory at sea, we require modems that can transmit synchronously from multiple transmitters. We also require modems that can sample synchronously from multiple hydrophones. The latter is easily achieved using Subnero’s multichannel modems, but the former had to be developed specifically for this project. We worked closely with Subnero to develop multi-transducer modems (MT modems) for both MF and VLF frequency bands (Figure 2.1), and used them for experimental testing of the theory at sea.

2.2.1 Experimental setup

An experiment was conducted along the east coast of Singapore from 24th to 26th February 2025 to test MIMO OFDM in the MF and VLF bands. 3 MF MIMO OFDM datasets were collected on the 24th and 25th, and 2 VLF MIMO OFDM datasets were collected on the 26th. Each dataset recorded transmissions (50–100 transmissions) of a test OFDM frame consisting of:

- 1 $\times$ SISO OFDM block transmitted from transducer 1.
- 1 $\times$ SISO OFDM block transmitted from transducer 2.
- 1 $\times$ SISO OFDM block transmitted from transducer 3.



Figure 2.2: MF and VLF multi-transducer (MT) modems being deployed at sea during the MIMO OFDM experiment. Two of the three transducers in each band are visible. The spacing between transducers was 1 m, and both MF and VLF band transducers were attached to the same vertical line with a weight at the bottom.

- $3 \times$ MIMO OFDM blocks with pilots for method 1.
- Numerous MIMO OFDM blocks with pilots for method 2.

The OFDM frames used a varying number of carriers (1024, 2048 and 4096 for MF; 128, 256, 512 and 1024 for VLF). The number of method 2 OFDM blocks was changed (between 9 and 64) depending on the number of carriers and frequency band of operation to manage signal duration and duty cycle.

The MF and VLF MT modems with 3 transducers were at a transmit vessel, and multichannel MF and VLF receive modems with 4 channels each were deployed from a receive vessel. The 3 transmitters formed a vertical array with a spacing of about 1 m between them (Figure 2.2), with the top transducer roughly 3–5 m below the water surface. The 4 receivers formed a vertical array with a spacing of about 1 m between them, with the top hydrophone roughly 10 m below the water surface. The receiver vessel was anchored (just off Kusu island) while the transmitter vessel moved to various distances R and transmitted. The setup is illustrated in Figure 2.3. The weather on all 3 days of the experiment was windy with whitecaps visible on the sea surface and strong currents. The water depth was between 20–50 m.

2.2.2 MF results

The SISO OFDM blocks from each transducer were processed using differential OFDM processing similar to that used in the Subnero modems. Since we had 3 transmitters and 4 receivers, this gave us 12 different SISO systems at each range. We use results from these channels as a benchmark to compare MIMO processing against.

As a further benchmark, we also process each of these transmissions using spatial diversity techniques (SIMO), as done in Subnero multichannel modems. Since we had 3 transmitters and 4 receivers, this gave us 3 different SIMO systems with 4-channel diversity.

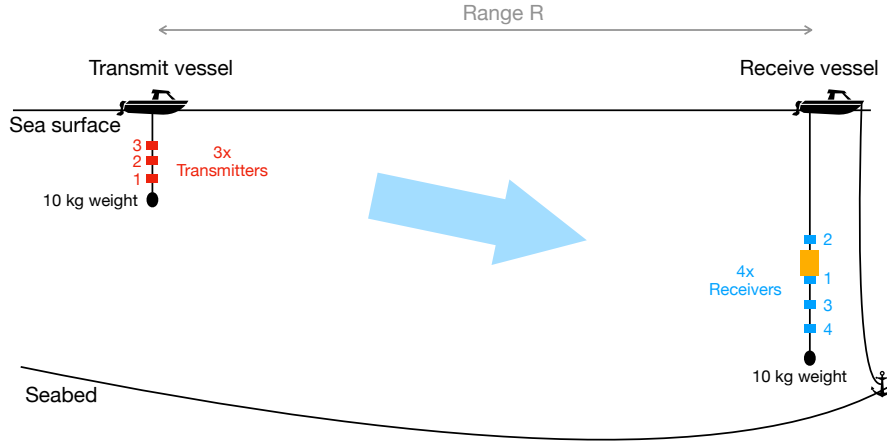


Figure 2.3: An illustration depicting the MIMO OFDM setup at sea (not to scale). The transmitter and receiver channels are numbered as deployed. Receiver channel 1 is the main transducer of the Subnero modem, whereas channels 2–4 are additional hydrophones. The intra-transmitter and intra-receiver nominal spacing is 1 m. The top transmitter is about 3–5 m deep, whereas the top receiver is about 10 m deep. The water depth is about 20–50 m. Range R varies for each dataset.

We process the MIMO OFDM blocks for methods 1 and 2 using the technique outlined above. For method 1, the number of pilots were fixed and pre-selected before transmission. However, for method 2, we are able to process the same data with different number of pilots, enabling us to explore the effect of pilots on channel estimation error and bit error rate (BER).

The results are summarized in Figure 2.4. The top panel shows the communication distance. We see 3 distinct distances for the 3 datasets we collected over 2 days. The second panel shows the number of OFDM carriers used. We rotated through 1024, 2048 and 4096 carriers during each dataset. The third panel shows the BER for SISO communication. When the BER is less than about 18%, error-free communication is possible with an appropriate selection of error correction code. We see that the BER depends strongly on suitable choice of number of carriers. The fourth panel shows the BER for SIMO communication. We see that the BER is generally lower than SISO, highlighting the advantage of spatial diversity combining. A lower BER translates to use of higher rate error correction codes, and hence higher data rates. The fifth panel shows the BER for MIMO method 1. We see an increase in BER as compared to the SIMO case. This is expected, since the MIMO channel uses the additional receivers for simultaneous data streams and hence reduces their effectiveness for spatial diversity. While this necessitates the use of a lower rate error correction code, the increased data rate due to the simultaneous MIMO data streams can still provide a net advantage. The last panel shows the BER for MIMO method 2 for 4 different pilot spacings ($P = 2$, $P = 4$, $P = 6$ and $P = 8$) in different colors. An increased pilot spacing increases the carriers available to carry data, but worsens the BER and hence forces us to use a lower rate error correction code. The net result is that there is an optimal pilot spacing for each channel, and we select the pilot spacing that maximizes data rate to estimate MIMO performance.

While the BERs are insightful, we are often interested in the data rate we can achieve for error-free communication. Based on the measured uncoded BER, we select an appropriate error correction code rate according an empirical table (Table 2.1) optimized for LDPC codes in the Subnero modems. Combining the code rate with the data carried by each OFDM frame with the given number of carriers and SISO/SIMO/MIMO settings, we obtain an estimate of error-free data rate for each dataset. This is tabulated in Table 2.2. The best results for each dataset are marked in bold. These results can be achieved by

BER Range	Code Rate
$0 \leq \text{BER} < 0.03$	2/3
$0.03 \leq \text{BER} < 0.07$	1/2
$0.07 \leq \text{BER} < 0.12$	1/3
$0.12 \leq \text{BER} < 0.15$	1/4
$0.15 \leq \text{BER} < 0.18$	1/6
$\text{BER} \geq 0.18$	no communication

Table 2.1: Code rate recommendations for various BER ranges

Date	Nominal distance (m)	Number of OFDM carriers (nc)	SISO data rate (kbps)	SIMO data rate (kbps)	MIMO-1 data rate (kbps)	MIMO-2 data rate (kbps)
24 Feb	750	1024	4.741	4.741	4.764	7.111
		2048	3.765	5.02	2.522	3.765
		4096	1.0344	1.8098	0.0	0.0
24 Feb	250	1024	2.37	4.741	0.0	0.0
		2048	0.0	2.51	0.0	0.0
		4096	0.0	0.0	0.0	0.0
25 Feb	1500	1024	4.741	4.741	4.764	7.111
		2048	5.02	5.02	5.045	7.529
		4096	3.879	5.172	1.0396	3.297

Table 2.2: Summary of achieved data rates during MF MIMO experiments. 3 different datasets were collected on 2 different days at 3 different ranges. The best result for each dataset is marked in bold.

selecting the correct number of carriers and the appropriate error correction code. We see that at 750 m on 24 February, we achieved a data rate of over 7.1 kbps using MIMO type 2. On the same day, at a shorter range of 250 m, MIMO failed to achieve a sufficiently low BER for communication. SIMO, on the other hand, provided us a 4.7 kbps link. While the signal to noise ratio (SNR) is indeed better at 250 m as compared to 750 m, the poorer performance of all OFDM schemes can be attributed to a more difficult channel geometry and variability. On 25 February, we were able to obtain an even better communication link with 7.5 kbps data rate at 1.5 km.

2.2.3 VLF results

The SISO, SIMO and MIMO OFDM blocks transmitted during the MIMO VLF tests were processed in a similar way as the MIMO MF tests. The BER results are summarized in Figure 2.5. Recall that error free communication is possible with an appropriate selection of error correction code, provided the uncoded BER does not exceed about 18%. For VLF SISO OFDM, we see that this is rarely the case, and so reliable OFDM communication was not achieved in either of the VLF datasets on 26th February. Spatial diversity processing (SIMO) improved the BER, and we see that communication is possible at 2.8–2.9 km range with $nc = 256$ and 512 in the first dataset, and at a range of 2.7–2.8 km with $nc = 256$ in the second

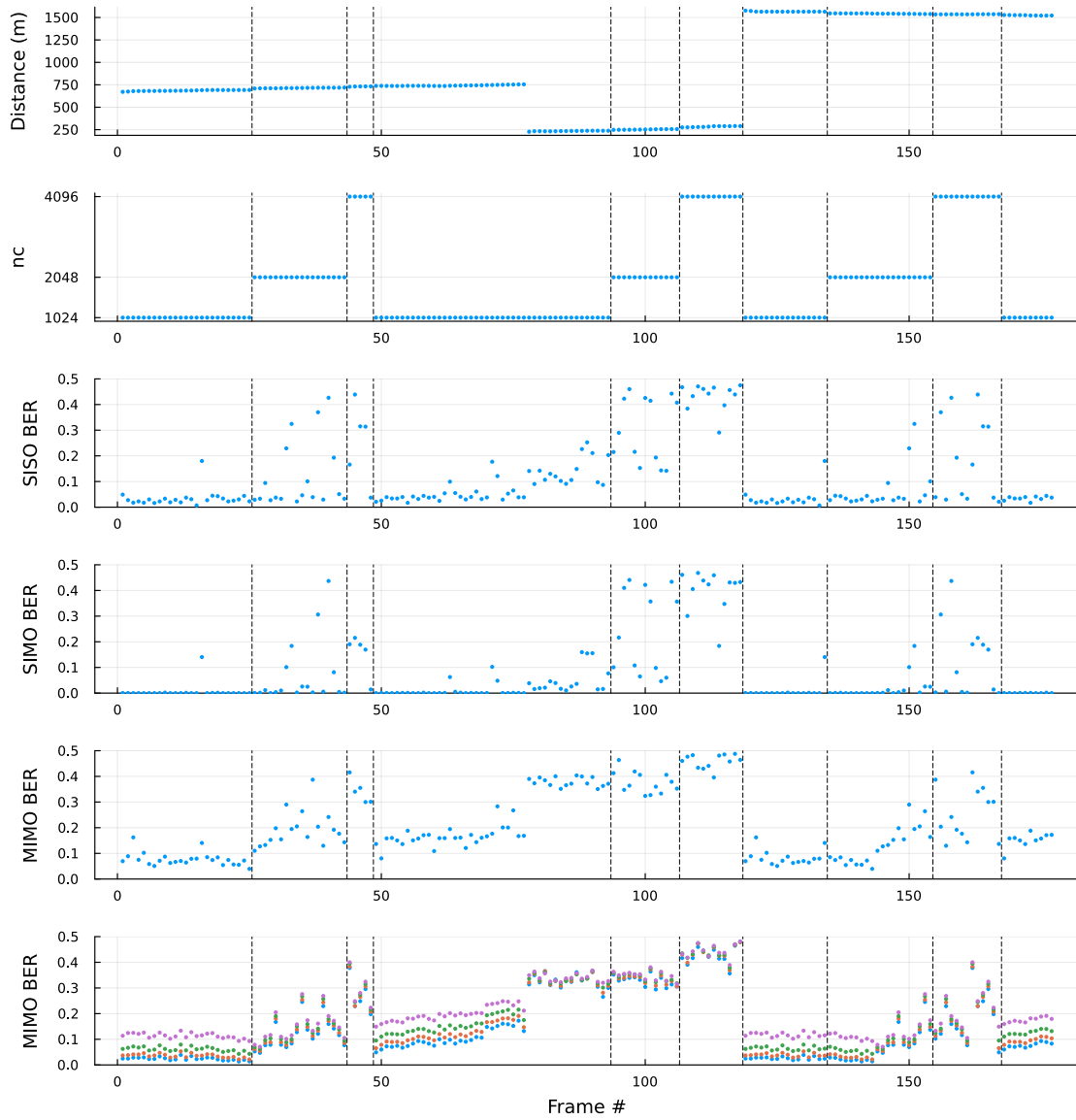


Figure 2.4: Summary of BER measured during MF MIMO experiments. The top panel shows the communication distance. The second panel shows the number of OFDM carriers used. The third panel shows the BER for SISO communication. The fourth panel shows the BER for SIMO communication. The fifth panel shows the BER for MIMO method 1. The last panel shows the BER for MIMO method 2 for 4 different pilot ratios ($P = 2$, $P = 4$, $P = 6$ and $P = 8$).

dataset. Neither MIMO type 1 or MIMO type 2 could provide error-free communication. The effective error-free data rate for VLF SIMO communication was 450 bps.

2.3 Discussion

MIMO with 3 transmitters and 3 or more receivers has a theoretical maximum data rate of $3 \times$ a SISO/SIMO system. In our experiments in the MF band, we were able to achieve about $1.5 \times$ the performance of a SISO/SIMO system when the channel conditions were favorable. In less favorable conditions¹, MIMO failed to communicate robustly and SISO performance was marginal, but spatial diversity combining (SIMO) retained the ability to communicate robustly. We conclude that MIMO OFDM is indeed a feasible option when high rate MF communication is required in favorable channel conditions.

While MIMO OFDM worked quite well in the MF band, it did not perform well in the VLF band². This is not surprising, as SISO OFDM often³ does not perform well in this band (MIMO is expected to be less robust than SISO, but SIMO is expected to be more robust than SISO/MIMO). The limited bandwidth available in the VLF band is not suitable for further sub-division, and hence multi-carrier modulation (e.g. OFDM) is not well suited for VLF communication. Even with a modest number of carriers, OFDM symbols become long in duration and hence see a lot of time-variability in the channel. OFDM is not robust to channel variability and performs poorly. Phase jitter resulting from scintillation due to inhomogeneities in the water column makes OFDM also unsuitable for long range communication. At long ranges, incoherent modulation techniques such as FH-BFSK are more suitable. These yield low data rate (tens of bps), but provide very robust performance, as demonstrated up to 20 km during the “long-range robust communication” project.

¹In this experiment, less favorable channel conditions were observed at a shorter range of 250 m, whereas the channel conditions were better at 750 m and 1.5 km. Communication performance is often not just a function of range. The channel geometry and variability plays an important role, and often at short-to-medium ranges in shallow waters, the MF channel can be more complicated than at longer ranges.

²VLF MIMO OFDM performed reasonably well during initial engineering tests at St. Johns island at short ranges (about 500 m). However, at short ranges, the MF band is more suitable and performs better. We therefore did not pursue VLF communication at ranges where MF communication is more suitable. The VLF band is best suited for low-rate long-range communication systems, whereas the objective of MIMO is high-rate communication.

³In Chapter 5, we present some experimental results in a channel where VLF SISO OFDM works reasonably well at 2.5 km. For consistent performance with coherent modulation in the VLF band, SIMO communication is recommended.

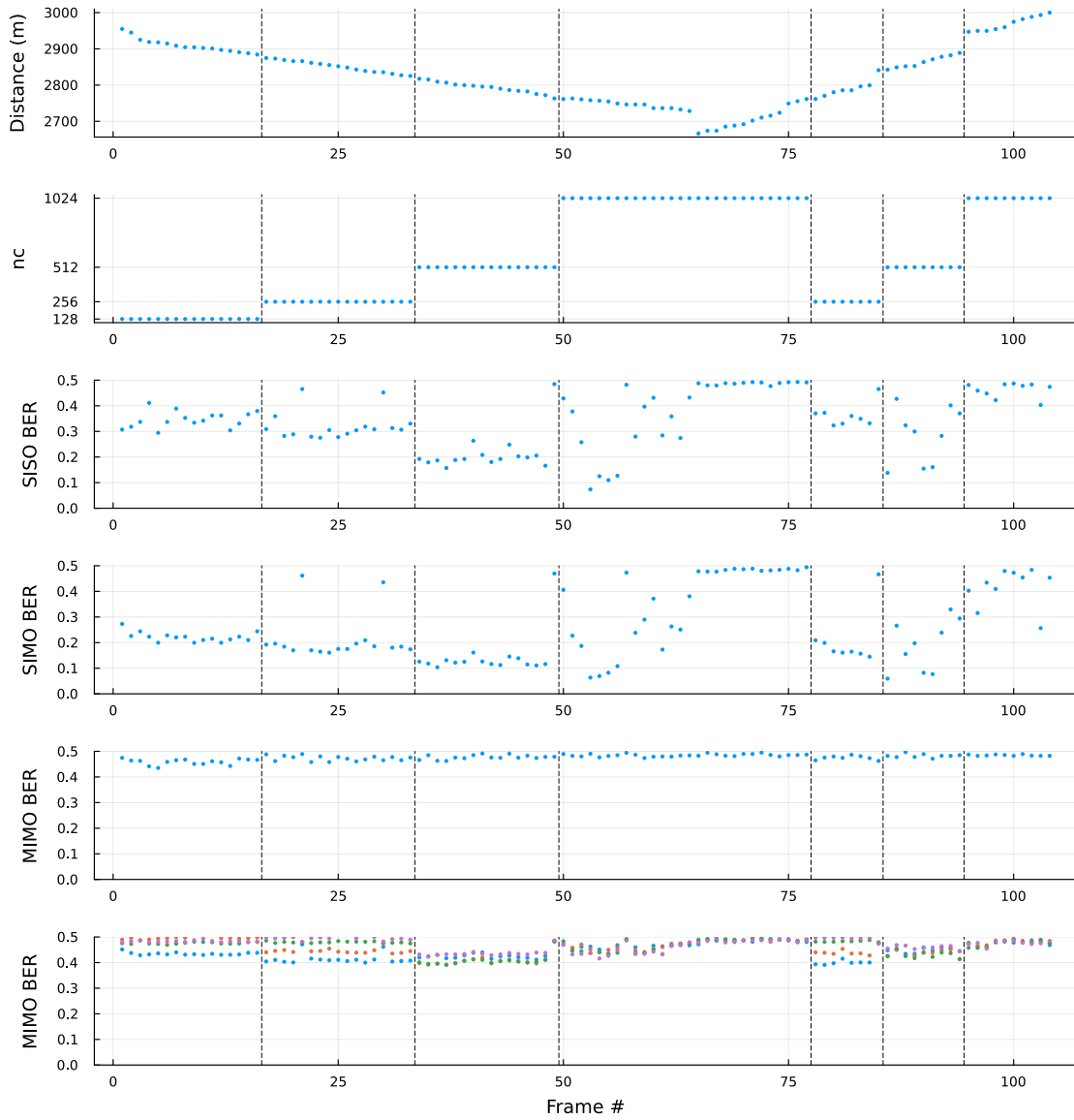


Figure 2.5: Summary of BER measured during VLF MIMO experiments. The top panel shows the communication distance. The second panel shows the number of OFDM carriers used. The third panel shows the BER for SISO communication. The fourth panel shows the BER for SIMO communication. The fifth panel shows the BER for MIMO method 1. The last panel shows the BER for MIMO method 2 for 4 different pilot ratios ($P = 2$, $P = 4$, $P = 6$ and $P = 8$).

CHAPTER 3

BLIND SOURCE SEPARATION

3.1 Introduction

In multi-node networks sharing an acoustic medium, transmissions from each node use the same frequency band. If such transmissions arrive at a receiver simultaneously, a *collision* is said to have occurred, and it usually leads unsuccessful reception for both transmissions. Nodes use some medium access control (MAC) protocol to coordinate their transmissions to reduce collisions, but avoiding collisions altogether is difficult. In networks where all the nodes are known a priori, one might allocate transmit slots for each node and guard times between those slots to avoid collisions. This is called time division multiple access (TDMA). Unless the traffic pattern is perfectly known, TDMA can be very inefficient. In networks where nodes need to communicate sporadically, a random access MAC is a better option. In a random access MAC, nodes transmit based on some heuristics to reduce collisions, but every now and then, collisions occur. For nodes equipped with multichannel modems (more than one receiving hydrophone), it may be possible to receive transmissions from multiple nodes simultaneously via clever signal processing. This is the aim of the blind source separation (BSS) research in this project.

There are two main families of random access MAC protocols in use today. The CSMA family of MAC protocols sense the channel and transmit if they do not hear anyone else transmitting. Transmissions are kept short to reduce chance of collision. Collisions are still possible since with propagation delays, knowledge of transmission by other nodes is delayed and hence not hearing anyone does not mean no one is transmitting. The MACA family of MAC protocols build on top of CSMA. They use CSMA to send an initial short handshake to indicate an intent to transmit. This handshake causes other nodes to reserve the channel and avoid collisions during the intended period of transmission. The short transmissions from CSMA or handshake from MACA typically are called CONTROL frames on UnetStack-based modems. If

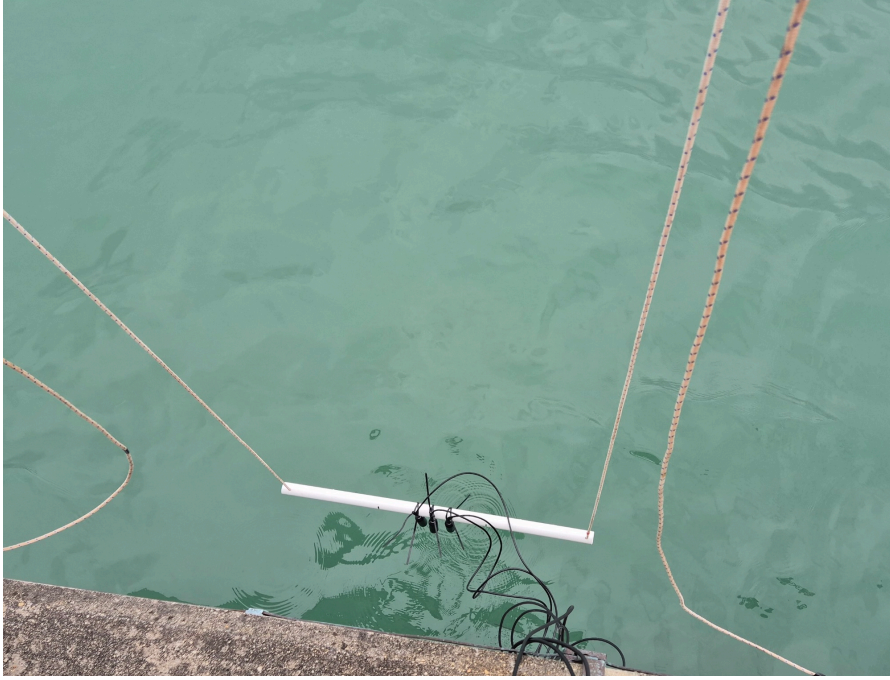


Figure 3.1: Horizontal array deployment at St. Johns Island jetty.

two CONTROL frames arrive at a node at the same time, being able to decode both would improve network performance. During the MAC reservation, higher speed DATA frames are used to exchange information. These are less likely to collide, as the MAC reservation would reduce the chance that other nodes transmit during the period. If DATA frames can also be received from multiple nodes simultaneously, network performance could be improved further.

Subnero modems typically use FH-BFSK modulation for CONTROL frames and OFDM modulation for DATA frames. FH-BFSK is a spread spectrum technique that has built-in resilience to collisions. When FH-BFSK frames collide, both frames see an increased BER, but the error correction code used for CONTROL frames is often able to correct the errors and successfully decode the frames. In this chapter, we show that the beamforming based BSS techniques recommended in deliverable D3 further improve the resilience of CONTROL frames. OFDM used in DATA frames, on the other hand, is very sensitive to collisions and rarely decodes successfully if a collision occurs. While beamforming based BSS improves the situation with OFDM greatly, we find that frame loss cannot be completely avoided using BSS. Non-blind techniques may allow us further improvements.

3.2 Experiment

To test BSS, we conducted an experiment at St. Johns Island in November 2024. A horizontal receiver array of 3 hydrophones was deployed from the jetty at about 4 m in a water depth of about 7–8 m. For the MF tests, the spacing between hydrophones was 4 cm, whereas for the VLF tests, the spacing was 15 cm, as recommended in deliverable D3. A photograph of the array being deployed is shown in Figure 3.1. Two transmitter modems were deployed in the area. One modem was deployed from another part of the jetty, while the other modem was deployed from a boat stationed in the bay. A rough geometry of the

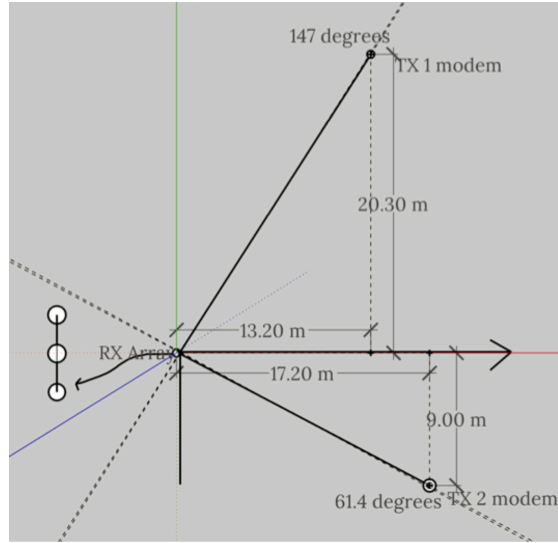


Figure 3.2: BSS experiment modem geometry.

setup is shown in Figure 3.2. The angular separation between the modems was roughly 86 degrees. Both modems transmitted continuously with similar power.

The signals were recorded on the hydrophone array and pre-processed using a beamforming based BSS as described in deliverable D3. Although D3 focused on delay-and-sum broadband beamforming, we subsequently explored the use of minimum-variance distortionless response (MVDR) beamformers and found them to be often more effective for BSS. The MVDR beamformer is able to listen to signals from one direction while simultaneously steering a null to mask signals from another direction. With only 3 hydrophones, however, its ability to mask signals from unwanted directions is limited. The recorded signal before BSS processing, and after BSS processing were fed to a Subnero modem, and the results were compared to see whether BSS processing yielded any improvement.

Figure 3.3 shows a spectrogram of a recorded signal during the MF OFDM BSS experiment. We see overlapping frames arriving from both the modems with similar receive power. Without BSS, we expect such frames to not decode well. In Figure 3.4, we see a beamformer energy plot that has two peaks corresponding to two modems transmitting the OFDM signals. By selecting a specific peak to beamform towards, the BSS receiver is able to listen to the signals from one modem and reject energy from the other modem.

3.2.1 MF FH-BFSK results

As FH-BFSK is resilient to collisions, we expect that many FH-BFSK frames would be detected successfully in spite of them colliding at the receiver. Even without BSS processing, we detect 24 frames and successfully decode 19 of them. After MVDR BSS processing, we detect 28 frames and are able to decode 25 of them successfully. In addition to MVDR BSS processing, we also test a time-domain broadband beamformer processing. In case of MF FH-BFSK, that results in 30 detections and 26 successful receptions. The results are summarized in Table 3.1.

Using BSS processing, we see a 40% extra CONTROL frames being successfully decoded as compared to the case without BSS processing.

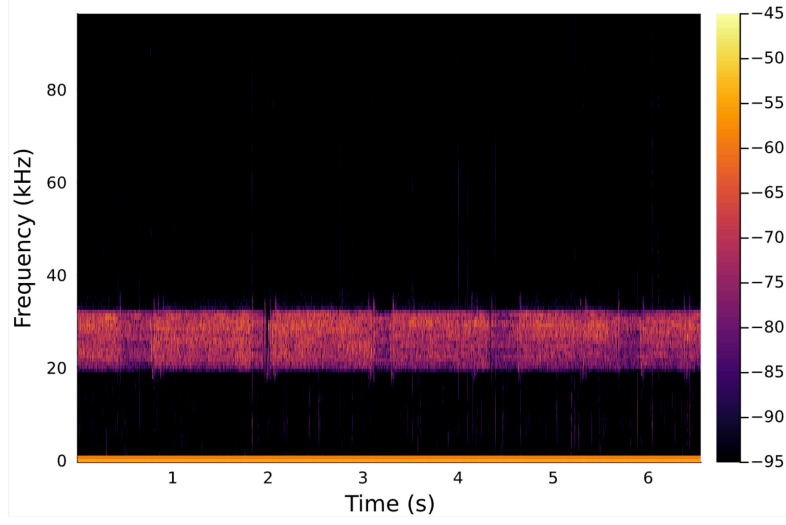


Figure 3.3: Spectrogram showing overlapping MF frames received from both transmitting modems.

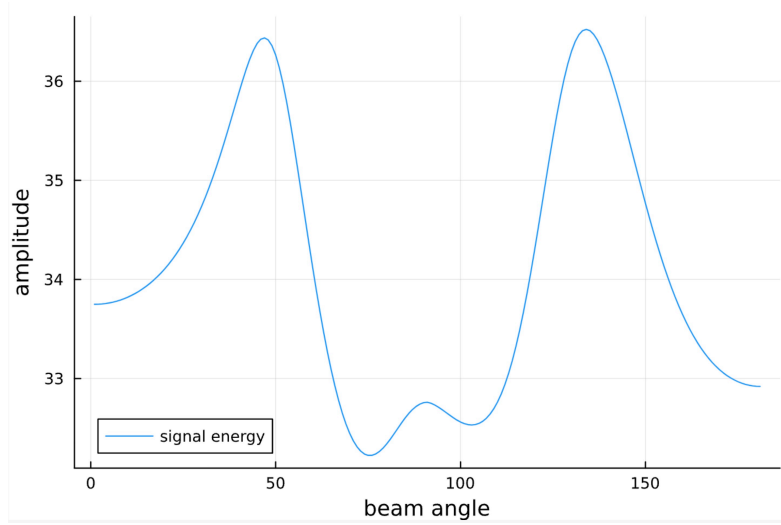


Figure 3.4: Beamformer energy plot showing energy arriving from two directions corresponding to the two transmitting modems.

3.2.2 VLF FH-BFSK results

In the VLF band, we use FH-BFSK for both CONTROL and DATA frames. The low data rate in this band makes frames very long and collisions highly likely. The resilience of FH-BFSK to collisions is important to ensure successful frame delivery. In a 84 second recording, without BSS processing, we detect 9 frames and successfully decode all of them. With time-domain broadband BSS processing that worked best with MF FH-BFSK, we detect all 10 transmitted frames over that period, and successfully decode all. The results are summarized in Table 3.2.

Table 3.1: MF FH-BFSK BSS results.

Method	Detected	Decoded	BER
No BSS processing	24	19	0.062
MVDR BSS processing	28	25	0.060
Time-domain BSS processing	30	26	0.059

Table 3.2: VLF FH-BFSK BSS results.

Method	Detected	Decoded	BER
No BSS processing	9	9	0.019
Time-domain BSS processing	10	10	0.009

While the number of frames successfully received with BSS is not much higher than the number of frames that could be received even without BSS, we see that the frames have a lower BER. This translates to a potential for a higher data rate that can be achieved using BSS by using an higher rate error correction code.

3.2.3 MF OFDM results

OFDM is much less resilient to collisions, and we expect that most OFDM frames fail when both modems transmit at the same time. Without BSS processing, we detected 116 frames. However, only 13 frames were successfully decoded. The MVDR beamformer based BSS yielded the best results for OFDM. We were able to detect 120 frames, with 37 frames successfully decoded. The results are tabulated in Table 3.3.

Table 3.3: MF OFDM BSS results.

Method	Detected	Decoded	BER
No BSS processing	116	13	0.19
MVDR BSS processing	120	37	0.18

With BSS processing, we see a 185% improvement in number of successfully decoded DATA frames!

3.3 Discussion

FH-BFSK is resilient to collisions and so is a great candidate for CONTROL frames used for short messages and handshaking in random access MAC protocols. We demonstrated that with BSS processing, the resilience of FH-BFSK can be further improved, and receiving CONTROL frames from two modems simultaneously is possible with high probability.

In case of OFDM, which is used for MF DATA frames, we got a large improvement from BSS processing. However, even with the improvement, 69% of the DATA frames were lost! OFDM is very sensitive to signal distortion, and while the BSS processing improves the signal to noise ratio (SNR), it is unable to retain the fidelity of the signal required for OFDM processing. The windowing performed during MVDR BSS processing likely destroys the orthogonality needed by OFDM and affects its performance.

Since our focus in this project was on *blind* source separation, where the receiver assumes no a priori knowledge of the source modulation and location, we were unable to improve the performance of MF OFDM BSS any further. If we assume knowledge of the source modulation, we may be able to process each OFDM block individually and avoid the windowing operation needed in the BSS processing. In addition, if more than 3 receive hydrophones are available, the improvement in SNR can be more, and the performance of OFDM source separation can be improved further.

CHAPTER 4

ACOMMS EXPERIMENT 2023

4.1 Introduction

AcommsEx 2023, a collaborative experiment conducted in Panama City, Florida, from the 14–17 November 2023, in conjunction with the Naval Research Lab (NRL), Naval Surface Warfare Center (NSWC), and supported by Popoto modems and the ARL (Singapore), aimed to investigate underwater acoustic communication in littoral waters. The study focused on assessing the reliability, data transmission rates, and overall performance of underwater acoustic communication systems when mounted on separate vessels, with a specific emphasis on the dynamic conditions introduced by the motion of the transmitter vessel. The experiment collected data that will help us get insights into the dynamic challenges posed by motion on underwater acoustic communication between vessels. It also provides us an opportunity to test some of our communication techniques in the Gulf of Mexico, where noise and environment variations be quite different from what we see in Singapore. We present a preliminary analysis of the data collected during the experiment.

4.2 Experiment

The AcommEx 2023 experiment was conducted over 3 days at sea. The receiver was moored and a transmitter was deployed from a boat that could move. During the first 2 days of the experiment, the transmitter boat cruised at various speeds starting approximately 2–3 km away from the receiver and passing the receiver until it was 2–3 km away from it in the other direction. This routine was repeated for every signal transmission. After each signal transmission was completed, another signal was loaded for

transmission, and the cycle was repeated. As such, each transmission has the full range of distance. In total there are 9 types of signals transmitted by all four groups (including ARL). The third day tested the same communication schemes in a static setup where the transmitter and receiver were both fixed.

The tracks of the boat are shown in Figure 4.1. The deployment diagram showing the location and the position of each element of the multi-hydrophone receiver system is shown in Figure 4.2. The Subnero modem was only used for the mobile tests, while the Popoto modems were used for both the mobile and static tests.

A summary of the data collection for the ARL signals is shown below:

- **Day 1:** 14 Nov 2023 – Mobile tests
 - 12:06 – 12:21 :: 4.86 kts mobile test
- **Day 2:** 16 Nov 2023 – Mobile tests
 - 09:11 – 09:26 :: 5.83 kts mobile test
 - 11:41 – 11:56 :: 6.83 kts mobile test
 - 11:58 – 12:13 :: 6.83 kts mobile test (repeated)
- **Day 3:** 17 Nov 2023 – Static tests
 - 07:57 – 08:02 :: static test (1 km range)
 - 08:54 – 08:59 :: static test (2 km range)
 - 09:38 – 09:43 :: static test (5 km range)
 - 10:27 – 10:32 :: static test (7.5 km range)
 - 11:17 – 11:22 :: static test (500 m range)

4.3 Signals

The Popoto modem used for transmission of signals during this experiment operates at a center frequency of 29 kHz and a bandwidth of 8 kHz. The effective band used for the experiment was therefore 25–33 kHz.

The signals designed by ARL for transmission during the AcommsEx 2023 included the channel probes, OFDM transmissions and windowed OFDM (WOFM) transmissions from the AcommEx 2022 in Singapore. While a few OSDM transmissions were included in AcommsEx 2022, they did not work well in Singapore. We therefore included a new set of OSDM signals to be tested in AcommsEx 2023 in Panama City. In addition, we included a few PSK signals which may potentially be useful for channel studies. A summary of signals is shown in Table 4.1.

Table 4.1: Summary of signals transmitted during AcommsEx 2023.

ID	Details
HFM-PROBES	5 42 ms 25-33 kHz hyperbolic up-chirps
OFDM-1024-BPSK	OFDM $N = 1024$, $L = 256$, BPSK, 10 blocks
OFDM-2048-BPSK	OFDM $N = 2048$, $L = 256$, BPSK, 5 blocks
OFDM-4096-BPSK	OFDM $N = 4096$, $L = 256$, BPSK, 3 blocks

ID	Details
OFDM-8192-BPSK	OFDM $N = 8192$, $L = 256$, BPSK, 2 blocks
OFDM-1024-QPSK	OFDM $N = 1024$, $L = 256$, QPSK, 10 blocks
OFDM-2048-QPSK	OFDM $N = 2048$, $L = 256$, QPSK, 5 blocks
OFDM-4096-QPSK	OFDM $N = 4096$, $L = 256$, QPSK, 3 blocks
OFDM-8192-QPSK	OFDM $N = 8192$, $L = 256$, QPSK, 2 blocks
WOFDM-1024-BPSK	WOFDM $N = 1024$, $L = 256$, BPSK, 10 blocks
WOFDM-2048-BPSK	WOFDM $N = 2048$, $L = 256$, BPSK, 5 blocks
WOFDM-4096-BPSK	WOFDM $N = 4096$, $L = 256$, BPSK, 3 blocks
WOFDM-8192-BPSK	WOFDM $N = 8192$, $L = 256$, BPSK, 2 blocks
WOFDM-1024-QPSK	WOFDM $N = 1024$, $L = 256$, QPSK, 10 blocks
WOFDM-2048-QPSK	WOFDM $N = 2048$, $L = 256$, QPSK, 5 blocks
WOFDM-4096-QPSK	WOFDM $N = 4096$, $L = 256$, QPSK, 3 blocks
WOFDM-8192-QPSK	WOFDM $N = 8192$, $L = 256$, QPSK, 2 blocks
OSDM-64-1-BPSK	OSDM $N = 64$, BPSK, 10 blocks
OSDM-32-1-BPSK	OSDM $N = 32$, BPSK, 10 blocks
OSDM-64-5-BPSK	OSDM $N = 64$, $Q = 5$, BPSK, 10 blocks
OSDM-32-5-BPSK	OSDM $N = 32$, $Q = 5$, BPSK, 10 blocks
OSDM-64-2-BPSK	OSDM $N = 64$, $Q = 2$, BPSK, 10 blocks
OSDM-32-2-BPSK	OSDM $N = 32$, $Q = 2$, BPSK, 10 blocks
GMSEQ11-BPSK	50 2047 chip generalized m-sequence
PSK01	Coded BPSK signal 1
PSK02	Coded BPSK signal 2
PSK03	Coded BPSK signal 3
PSK04	Coded BPSK signal 4
PSK05	Coded BPSK signal 5
PSK06	Coded BPSK signal 6
PSK07	Coded BPSK signal 7

We focus on the OFDM, WOFDM and OSDM signals. The implementation of the OFDM modulation scheme is based on the Subnero modems, with a shift in center frequency from 25 kHz to 29 kHz to adapt to the transmission band of Popoto modems. The WOFDM implementation was developed specifically for this project, but is derived from the Subnero OFDM implementation. The OSDM implementation is based on [2, 3].

4.4 Analysis

As noted in Dale Green's presentation during the meeting in February 2024, the data from the 500 m, 1 km, 2 km and 5 km static tests is of good quality. At 7.5 km, the signal is not audible – this is not surprising, and is consistent with expectations from known physics.



Figure 4.1: Tracks of the boat during the experiment

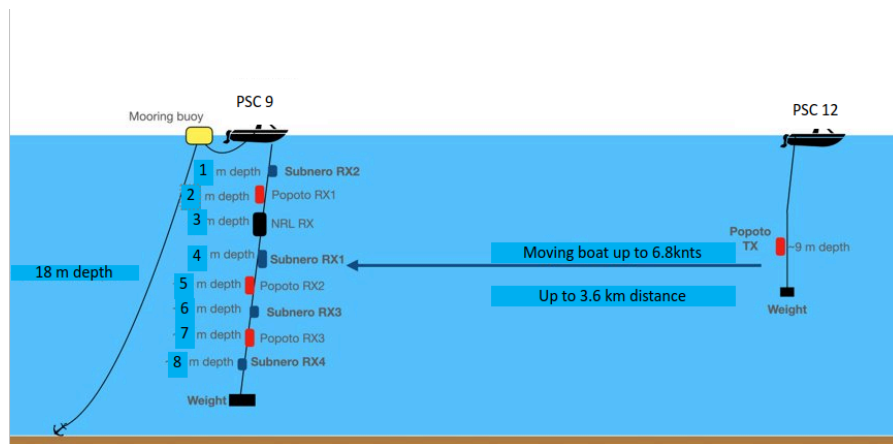


Figure 4.2: Deployment diagram of the multi-hydrophone receiver system

The data from the mobile tests is harder to interpret, as the towed source may have been uncontrolled throughout the tests and possibly very near the surface. A preliminary look at the data from the mobile tests suggests that the signals are clearly audible when the boat passes the closest point of approach (CPA) to the receiver, but the signal rapidly fluctuates, suggesting that the source is either uncontrolled or the bubbles from the towing boat are affecting the transmissions.

We only analyze the datasets from the static tests at 500 m, 1 km, 2 km and 5 km. They provide useful information into the performance of various modulation schemes under different test conditions.

4.4.1 Static test at 500 m range

At 500 m range, Popoto RX1 and Popoto RX2 were recording. The results based on recordings from each of the two receivers are shown below. Only modulation schemes with an uncoded bit error rate (BER) less than 0.2 are shown – at this BER, it is possible to use a forward error correction (FEC) code to correct all the bit errors.

The column :nbits shows the number of bits per frame. The column :bps shows the uncoded data rate in bits per second. At this data rate, the frame may have some bits in error. The column :ber shows the uncoded BER. A FEC code can be used to correct the bit errors. However, this reduces the effective data rate, as some bits are used as redundancy. The column :effective shows the estimated effective error-free data rate in bits per second.

Results from data recorded on Popoto RX1

	mod	nbits	bps	ber	effective
1	OFDM-1024-QPSK	20460	12788	0.1407	5311
2	WOFSM-2048-QPSK	10230	7104	0.0897	4012
3	WOFSM-4096-QPSK	12282	7526	0.1227	3486
4	OFDM-1024-BPSK	10230	6394	0.0983	3429
5	WOFSM-1024-QPSK	10220	6388	0.1208	2993
6	OFDM-2048-BPSK	10235	7108	0.1466	2853
7	WOFSM-2048-BPSK	5115	3552	0.0485	2560
8	WOFSM-1024-BPSK	5110	3194	0.0547	2217
9	WOFSM-4096-BPSK	6141	3763	0.087	2159
10	WOFSM-8192-BPSK	8190	3878	0.1556	1464

While the best performing scheme is OFDM with QPSK, we observe that more WOFSM schemes perform at an acceptable BER to yield robust communication. As expected, the windowing technique used in WOFSM improves robustness at the cost of a slightly lower data rate. As we see in the rest of this report, this finding is consistent across all the ranges and receivers tested.

Results from data recorded on Popoto RX2

	mod	nbits	bps	ber	effective
1	OFDM-1024-QPSK	20460	12788	0.1334	5616
2	OFDM-2048-QPSK	20470	14215	0.1805	4532
3	WOFDM-4096-QPSK	12282	7526	0.1058	3861
4	OFDM-1024-BPSK	10230	6394	0.0901	3622
5	WOFDM-2048-QPSK	10230	7104	0.1204	3459
6	OFDM-2048-BPSK	10235	7108	0.1314	3118
7	WOFDM-1024-QPSK	10220	6388	0.1239	2963
8	WOFDM-2048-BPSK	5115	3552	0.0368	2745
9	WOFDM-8192-QPSK	16380	7756	0.1684	2683
10	WOFDM-1024-BPSK	5110	3194	0.0548	2221
11	WOFDM-4096-BPSK	6141	3763	0.1272	1760

The log-likelihood ratios from multiple receivers can be combined to harness spatial diversity. Such combining can improve reception performance beyond that of each individual receiver. The results from combining the data from Popoto RX1 and Popoto RX2 are shown below:

Results from combining data from Popoto RX1 and Popoto RX2

	mod	nbits	bps	ber	effective
1	OFDM-1024-QPSK	20460	12788	0.0929	7324
2	OFDM-2048-QPSK	20470	14215	0.1367	6087
3	WOFDM-4096-QPSK	12282	7526	0.0468	5474
4	WOFDM-2048-QPSK	10230	7104	0.0408	5410
5	OFDM-4096-QPSK	24570	15055	0.1835	4703
6	WOFDM-1024-QPSK	10220	6388	0.05	4566
7	OFDM-2048-BPSK	10235	7108	0.0737	4411
8	OFDM-1024-BPSK	10230	6394	0.0769	4097
9	WOFDM-8192-QPSK	16380	7756	0.1046	4006
10	WOFDM-2048-BPSK	5115	3552	0.0077	3320
11	WOFDM-1024-BPSK	5110	3194	0.0089	2959
12	WOFDM-4096-BPSK	6141	3763	0.053	2696
13	OFDM-4096-BPSK	12285	7528	0.17	2577
14	WOFDM-8192-BPSK	8190	3878	0.1237	1784

We see that many more modulation schemes have a BER less than 0.2 when the data from both Popoto RX1 and Popoto RX2 are combined. We also see that the best performing schemes provide a higher effective data rate when the data from both receivers are combined.

4.4.2 Static test at 1 km range

At 1 km range, Popoto RX1, Popoto RX2 and Popoto RX3 were recording. The results based on recordings from each of the three receivers are shown below.

Results from data recorded on Popoto RX1

	mod	nbits	bps	ber	effective
1	OFDM-1024-QPSK	20460	12788	0.1371	5419
2	OFDM-2048-QPSK	20470	14215	0.1585	5259
3	OFDM-1024-BPSK	10230	6394	0.0767	3907
4	WOFDM-2048-QPSK	10230	7104	0.1176	3405
5	OFDM-2048-BPSK	10235	7108	0.1228	3288
6	WOFDM-4096-QPSK	12282	7526	0.1438	3055
7	WOFDM-1024-QPSK	10220	6388	0.1446	2615
8	WOFDM-2048-BPSK	5115	3552	0.052	2505
9	WOFDM-1024-BPSK	5110	3194	0.0629	2113
10	WOFDM-4096-BPSK	6141	3763	0.106	1939
11	OSDM-32-2-BPSK	2560	3200	0.1719	1082

Results from data recorded on Popoto RX2

	mod	nbits	bps	ber	effective
1	OFDM-1024-QPSK	20460	12788	0.1211	6004
2	OFDM-2048-QPSK	20470	14215	0.1462	5716
3	OFDM-1024-BPSK	10230	6394	0.0783	3865
4	WOFDM-2048-QPSK	10230	7104	0.1133	3483
5	OFDM-2048-BPSK	10235	7108	0.1155	3460
6	WOFDM-2048-BPSK	5115	3552	0.0286	2886
7	WOFDM-1024-QPSK	10220	6388	0.1324	2815
8	WOFDM-4096-BPSK	6141	3763	0.0646	2462
9	WOFDM-1024-BPSK	5110	3194	0.0672	2072

Results from data recorded on Popoto RX3

	mod	nbits	bps	ber	effective
1	OFDM-2048-QPSK	20470	14215	0.1142	6930
2	OFDM-1024-QPSK	20460	12788	0.1198	6164
3	WOFSM-2048-QPSK	10230	7104	0.0435	5270
4	WOFSM-4096-QPSK	12282	7526	0.075	4634
5	WOFSM-8192-QPSK	16380	7756	0.0839	4532
6	OFDM-4096-QPSK	24570	15055	0.1946	4350
7	WOFSM-1024-QPSK	10220	6388	0.0633	4213
8	OFDM-1024-BPSK	10230	6394	0.093	3700
9	OFDM-2048-BPSK	10235	7108	0.1172	3402
10	WOFSM-2048-BPSK	5115	3552	0.0256	2941
11	WOFSM-4096-BPSK	6141	3763	0.0484	2712
12	OFDM-4096-BPSK	12285	7528	0.1839	2345
13	WOFSM-1024-BPSK	5110	3194	0.0487	2297
14	WOFSM-8192-BPSK	8190	3878	0.115	1881
15	OSDM-64-2-BPSK	7680	5333	0.1728	1792
16	OSDM-32-2-BPSK	2560	3200	0.1152	1550

Results from combining data from Popoto RX1, Popoto RX2 and Popoto RX3

	mod	nbits	bps	ber	effective
1	OFDM-1024-QPSK	20460	12788	0.0503	9109
2	OFDM-2048-QPSK	20470	14215	0.0816	8427
3	WOFSM-2048-QPSK	10230	7104	0.0386	5428
4	OFDM-1024-BPSK	10230	6394	0.0221	5422
5	OFDM-4096-QPSK	24570	15055	0.1821	4748
6	WOFSM-1024-QPSK	10220	6388	0.0537	4484
7	WOFSM-4096-QPSK	12282	7526	0.0941	4139
8	OFDM-2048-BPSK	10235	7108	0.1142	3640
9	WOFSM-2048-BPSK	5115	3552	0.0067	3345
10	WOFSM-4096-BPSK	6141	3763	0.0239	3150
11	WOFSM-1024-BPSK	5110	3194	0.014	2856
12	OSDM-64-2-BPSK	7680	5333	0.1906	1585
13	OSDM-32-2-BPSK	2560	3200	0.1918	943

It is worth noting that OSDM provided acceptable performance at 1 km range, although it lagged behind the OFDM and WOFDM schemes. The best performing schemes at 1 km range are OFDM with QPSK, with WOFDM providing robust performance across various parameters.

4.4.3 Static test at 2 km range

At 2 km range, Popoto RX1 and Popoto RX2 were recording. The results based on recordings from each of the two receivers are shown below.

Results from data recorded on Popoto RX1

	mod	nbits	bps	ber	effective
1	OFDM-4096-QPSK	24570	15055	0.1203	7072
2	OFDM-1024-QPSK	20460	12788	0.1016	6773
3	OFDM-2048-QPSK	20470	14215	0.1441	6007
4	WOFDM-2048-QPSK	10230	7104	0.0465	5177
5	WOFDM-4096-QPSK	12282	7526	0.0646	4976
6	WOFDM-1024-QPSK	10220	6388	0.0625	4235
7	OFDM-1024-BPSK	10230	6394	0.0663	4203
8	OFDM-4096-BPSK	12285	7528	0.0953	4110
9	OFDM-2048-BPSK	10235	7108	0.1097	3782
10	WOFDM-8192-QPSK	16380	7756	0.1224	3684
11	WOFDM-2048-BPSK	5115	3552	0.0225	3010
12	WOFDM-4096-BPSK	6141	3763	0.0387	2897
13	WOFDM-1024-BPSK	5110	3194	0.0316	2548
14	OFDM-8192-BPSK	16382	7757	0.178	2516
15	WOFDM-8192-BPSK	8190	3878	0.09	2248
16	OSDM-64-2-BPSK	7680	5333	0.1543	2024
17	OSDM-32-5-BPSK	2560	3200	0.1484	1261
18	OSDM-32-2-BPSK	2560	3200	0.1582	1188

Results from data recorded on Popoto RX2

	mod	nbits	bps	ber	effective
1	OFDM-2048-QPSK	20470	14215	0.0937	7837
2	OFDM-1024-QPSK	20460	12788	0.1343	5618
3	OFDM-4096-QPSK	24570	15055	0.1575	5594
4	OFDM-1024-BPSK	10230	6394	0.0873	3661
5	WOFSM-2048-QPSK	10230	7104	0.11	3613
6	WOFSM-1024-QPSK	10220	6388	0.0945	3505
7	OFDM-4096-BPSK	12285	7528	0.1402	3126
8	WOFSM-2048-BPSK	5115	3552	0.0297	2867
9	OFDM-2048-BPSK	10235	7108	0.147	2826
10	WOFSM-1024-BPSK	5110	3194	0.0437	2367
11	WOFSM-8192-BPSK	8190	3878	0.1078	1965
12	OSDM-64-2-BPSK	7680	5333	0.1987	1497
13	OSDM-32-2-BPSK	2560	3200	0.1887	964

Results from combining data from Popoto RX1 and Popoto RX2

	mod	nbits	bps	ber	effective
1	OFDM-4096-QPSK	24570	15055	0.0596	10148
2	OFDM-1024-QPSK	20460	12788	0.0551	9023
3	OFDM-2048-QPSK	20470	14215	0.104	8086
4	WOFSM-2048-QPSK	10230	7104	0.0162	6271
5	OFDM-2048-BPSK	10235	7108	0.0351	5550
6	WOFSM-1024-QPSK	10220	6388	0.0439	4826
7	OFDM-1024-BPSK	10230	6394	0.0658	4351
8	WOFSM-4096-QPSK	12282	7526	0.1131	3694
9	WOFSM-2048-BPSK	5115	3552	0.0022	3473
10	WOFSM-8192-QPSK	16380	7756	0.143	3163
11	WOFSM-1024-BPSK	5110	3194	0.0026	3110
12	OFDM-4096-BPSK	12285	7528	0.149	2997
13	WOFSM-8192-BPSK	8190	3878	0.0711	2562
14	OSDM-64-2-BPSK	7680	5333	0.1521	2053
15	WOFSM-4096-BPSK	6141	3763	0.1096	1886
16	OSDM-32-2-BPSK	2560	3200	0.1438	1299

The results at 2 km range are consistent with the results at 1 km range. The best performing schemes are OFDM with QPSK, with WOFSM providing robust performance across various parameters. Some

OSDM schemes provided acceptable performance, but they lagged behind the OFDM and WOFDM schemes.

With 2-receiver spatial diversity combining, an effective data rate of over 10 kbps could be achieved at 2 km range. Without diversity combining, both receivers were able to achieve about 7 kbps.

4.4.4 Static test at 5 km range

At 5 km range, Popoto RX1 and Popoto RX2 were recording. The results based on recordings from each of the two receivers are shown below.

No OFDM, WOFDM or OSDM schemes had a BER less than 0.2 at 5 km range when recorded on Popoto RX1.

Results from data recorded on Popoto RX2

	mod	nbits	bps	ber	effective
1	WOFDM-1024-BPSK	5110	3194	0.1818	1009

Results from combining data from Popoto RX1 and Popoto RX2

	mod	nbits	bps	ber	effective
1	OFDM-1024-BPSK	10230	6394	0.1966	1822
2	WOFDM-1024-BPSK	5110	3194	0.1601	1167

At a range of 5 km, OFDM and WOFDM with BPSK modulation and 1024 carriers performed acceptably with spatial diversity combining. Without spatial diversity, WOFDM was still able to provide an effective data rate of over 1 kbps.

4.4.5 Summary

We summarize the findings from the static tests in Figure 4.3.

If adaptive modulation is used to choose the best performing scheme at each range, we can achieve a effective data rate of almost 12 kbps at 2 km range and close to 2 kbps at 5 km range with spatial diversity combining. Without spatial diversity, the effective data rate is about 7 kbps at 2 km range and over 1.5 kbps at 5 km range.

Without adaptive modulation, we are forced to choose a single modulation scheme for all ranges and receiver depths. If we choose OFDM-1024-BPSK, we get the most consistent performance of about 5 kbps at ranges up to 2 km and 2 kbps at 5 km range with spatial diversity combining. Without spatial diversity, the effective data rate is about 4 kbps at 2 km range and 1.3 kbps at 5 km range.

While we observe that WOFDM generally provides more consistent performance irrespective of choice of parameters, WOFDM-1024-BPSK generally yielded a slightly lower performance than OFDM-1024-BPSK. However, at 5 km without spatial diversity combining, WOFDM-1024-BPSK is the only tested modulation scheme that provided robust communication.

While OSDM provided acceptable performance at some ranges and receiver depths, its performance was always inferior to the OFDM and WOFDM schemes. Since OSDM can be tuned to be the same as OFDM with appropriate choices of parameters, this finding simply reflects the need for very careful tuning of OSDM parameters to achieve robust communication.

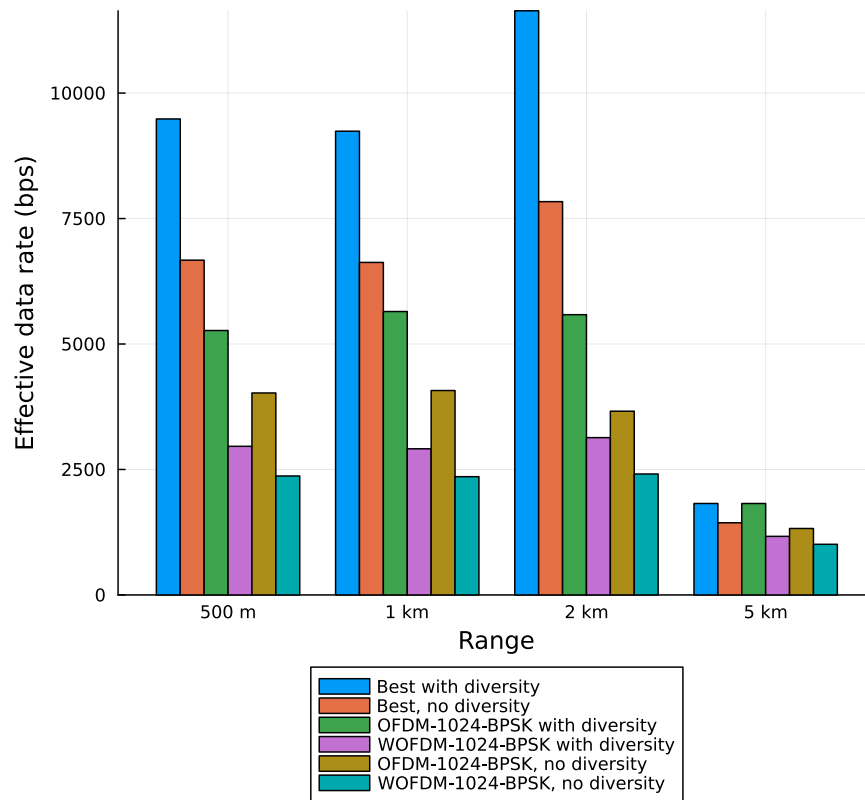


Figure 4.3: Summary of performance of various modulation schemes at different ranges. For non-spatial diversity combining, the Popoto RX2 modem in mid-water column was used.

CHAPTER 5

VLF WINDOWED OFDM

5.1 Introduction

OFDM is a communication technique that is bandwidth efficient, but quite sensitive to channel variability. In deliverable D2, we introduced windowed-OFDM (W-OFDM) – a technique that trades off data rate for robustness to channel variability, and suggested that this technique might be useful in channels where variability is significant. In MF tests during AcommsEx 2022 (deliverable D2) and AcommsEx 2023 (Chapter 4), we found that while W-OFDM was indeed more robust than OFDM, OFDM generally outperformed W-OFDM in terms of data rate. The recommendation is for W-OFDM to be used only in cases where the channel variability causes OFDM to fail.

Since both AcommsEx 2022 and 2023 did not use VLF modems, the W-OFDM technique was untested in the VLF band. In this chapter, we report findings from an experiment in Singapore waters in February 2025 where we tested OFDM and W-OFDM in the VLF band at a range of 2.5 km.

5.2 Experiment

A VLF transmitter was deployed at a depth of about 5 m and a range of about 2.5 km from a vertical receiver array. The spacing between the hydrophones on the receiver array was about 1 m and the top hydrophone was at about 10 m depth. The water depth was between 20–50 m.

OFDM signals with differing schemes (number of carriers, prefix length and windowing) were transmitted over a period of 20 minutes. The received signals were processed to estimate BER and *goodput*. Recall from deliverable D2 that the number of bits of user information successfully delivered per unit

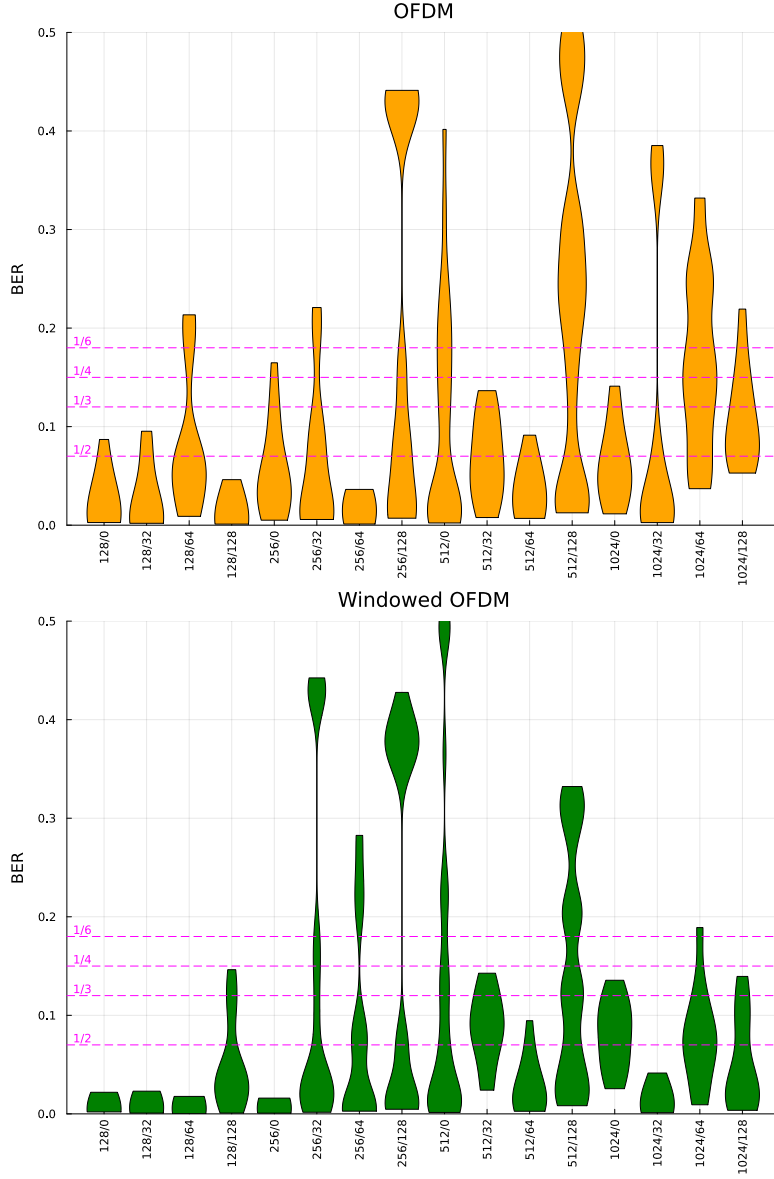


Figure 5.1: Comparison of BER between OFDM and W-OFDM for various OFDM parameters in the VLF band. The labels on the x -axis are of the format: number of carriers / prefix length. The horizontal lines are code rate boundaries, and show maximum BER at which an LDPC error correcting code with the labeled code rate is likely to correct all errors.

time, and is computed by optimizing the code rate to strike a balance between coded data rate and low BER (to guarantee successful error correction of channel errors). Mathematically:

$$\text{Goodput}_i = \max_p [D_i p \times p_i(p)], \quad (5.1)$$

Number of carriers	Prefix length	W-OFDM Goodput	OFDM Goodput
128	0	333	556
128	32	267	444
128	64	222	333
128	128	167	222
256	0	250	375
256	32	0	111
256	64	113	267
256	128	125	222
512	0	139	117
512	32	64	0
512	64	111	160
512	128	117	233
1024	0	167	222
1024	32	135	323
1024	64	209	392
1024	128	111	259

Table 5.1: Goodput in bps of various OFDM communication schemes with and without windowing.

where ρ is the chosen code rate, D_i is the signaling rate of the communication scheme i in bits/second, and $p_i(\rho)$ is the probability of success of a packet (zero errors in the packet) given the scheme i and code rate ρ .

The BER results for various communication schemes with and without windowing are shown in Figure 5.1. We see that the BER is quite sensitive to the number of carriers and prefix length, with generally a smaller number of carriers performing better than a larger number of carriers. More importantly, the W-OFDM BER is usually lower than the corresponding OFDM scheme with the same number of carriers and prefix length. Both observations are consistent with the idea that channel variability has a strong effect on OFDM performance. OFDM blocks with less carriers are shorter in duration than the ones with more carriers. They therefore see less variability, and perform better. W-OFDM is less sensitive to degradation due to channel variability, and so the BER for W-OFDM is generally lower than that of OFDM.

Although W-OFDM performs better in terms of BER, its signaling rate is half of OFDM as alternate carriers are left empty to avoid inter-carrier interference (ICI) due to windowing. By choosing an appropriate code rate, OFDM could potentially outperform W-OFDM if the reduced BER of W-OFDM does not lead to a code rate that is at least double than of OFDM. This trade-off is clearly visible by tabulating the goodput for OFDM and W-OFDM in Table 5.1. We see that generally the goodput from OFDM is higher than that from W-OFDM. The only exception is 512 carriers with a prefix length of 32. For this scheme, OFDM was unable to deliver any error-free frames and resulted in a data rate of 0 bps. W-OFDM was robust and was able to maintain a modest data rate of 64 bps. This lower goodput but higher robustness is consistent the results obtained for W-OFDM in the MF band.

5.3 Discussion

The VLF band is suitable for long-range low-rate communication. In the “long-range robust underwater communication” project, ARL has shown that an incoherent FH-BFSK modulation scheme can be used to robustly communicate to more than 20 km. At long distances, the scintillation due to inhomogeneities in the water column makes phase coherent communication (e.g. OFDM) difficult. Moreover, the narrow bandwidth available in the VLF band makes further sub-division of the band into sub-carriers undesirable. While the OFDM data presented in this chapter yielded reasonably good performance, the OFDM transmissions at a similar range during the MIMO experiment (Chapter 2) suffered a poorer performance. This inconsistent performance of OFDM in the VLF band, even at relatively short ranges where SNR is high, makes OFDM unsuitable for long-range robust communication in this band. We generally recommend incoherent FH-BFSK for this purpose. However, in mid-range applications, if a higher data rate communication in the VLF band is required, W-OFDM might provide a reasonable alternative with more robustness to channel variability and phase jitter than OFDM¹.

¹At mid-ranges, a modem operating at a higher frequency band (9-16 kHz LF band) may provide better communication. During a recent experiment, Subnero demonstrated OFDM data rates of over 1 kbps at 5 km and about 500 bps at 10 km using the LF modem (achievable only when the channel conditions are favorable). Robust communication was consistently demonstrated at about 100 bps using incoherent FH-BFSK at various ranges up to 10 km.

CHAPTER 6

DIFFERENTIABLE FORWARD ERROR CORRECTION

6.1 Introduction

Typical methods for handling fast time-varying channels rely on pilots or training sequences to monitor channel fluctuations over time, while forward error correction (FEC) codes are used to reduce bit errors and maintain a robust communication link. However, these two processes – channel compensation and error correction – often operate independently, without leveraging each others outputs. This approach is suboptimal, as the structure of error-correcting codes contains valuable channel information. In addition to recovering the original bit sequence, these code structures can be exploited to help reconstruct the channel, and vice-versa, giving rise to the joint channel estimation and decoding problem.

The complexity of this joint approach has thus far prevented the development of closed-form solutions. To tackle this complexity, we formulate a differentiable cost function and employ automatic differentiation (AD) as a key enabler. As a core feature of leading deep learning frameworks like PyTorch and TensorFlow, AD automatically derives gradients directly from the implementation, facilitating optimization methods based on gradient descent. This approach of using AD to solve the joint channel estimation and decoding problem by reformulating the original combinatorial problem as a differentiable loss function is termed as Differentiable Forward Error Correction (DFEC).

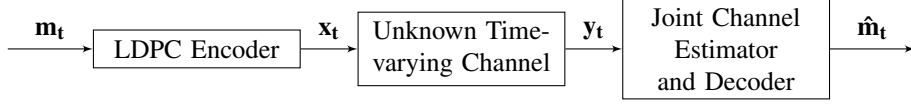


Figure 6.1: A schematic diagram of the joint channel estimation and decoding flow. Low Density Parity Codes (LDPC) are used. A brief description is provided below.

6.2 System model

6.2.1 LDPC Encoder

Low-Density Parity-Check (LDPC) codes, along with Turbo codes, are considered capacity-achieving codes and have been widely adopted across various industry standards. In this work, we focus on LDPC codes. An LDPC encoder maps the message $\mathbf{m}$ a k -dimensional binary vector into a codeword $\mathbf{C}$, another n -dimensional binary vector where $n > k$. The remaining $n - k$ bits are parity bits. A typical structure of the codeword is as shown in (6.1)

$$\mathbf{C} = \underbrace{p_1, p_2, \dots, p_{n-k}}_{\mathbf{p}, \text{ parity bits}}, \underbrace{m_1, m_2, \dots, m_k}_{\mathbf{m}, \text{ message bits}} \quad (6.1)$$

where $\mathbf{C}$ here represents the set of all valid codewords. The parity-check matrix, $\mathbf{P}$ is a binary matrix of $\mathbf{P} \in \mathbb{F}_2^{m \times n}$, where m is the number of check nodes and n is the length of the code and $\mathbb{F}_2$ is the Galois field of 2. The binary linear code associated with $\mathbf{P}$, denoted by $\tilde{\mathbf{C}}(\mathbf{P})$, is defined as the set of all binary vectors $\mathbf{b} \in \mathbb{F}_2^n$ that satisfy the following parity-check equation:

$$\tilde{\mathbf{C}}(\mathbf{P}) = \{\mathbf{b} \in \mathbb{F}_2^n \mid \mathbf{P} \mathbf{b}^T = \mathbf{0}\}$$

This condition ensures that the vector $\mathbf{b}$ satisfies all the parity-checks imposed by the matrix $\mathbf{P}$, and hence, is a valid codeword in the code defined by $\mathbf{P}$. The *parity-check matrix*, $\mathbf{P}$ structured as follows::

$$\mathbf{P} = \begin{bmatrix} \mathbf{p}_1 \\ \mathbf{p}_2 \\ \mathbf{p}_3 \\ \dots \\ \mathbf{p}_{n-k} \end{bmatrix} \quad (6.2)$$

$\mathbf{p}_i$, each row of the $\mathbf{P}$ is a row vector of length n , representing one parity-check equation. There are $n - k$ rows, each corresponding to its respective element in parity bit vector $\mathbf{p}$ and Let $\mathbf{b} = b_0, b_1, \dots, b_n$ be a codeword in $\mathbf{C}$, the inner product of $\mathbf{p}_i$ and $\mathbf{b}$ is the must be an even number (zero in the modulo 2 domain) if the codeword is indeed a valid codeword.

$$\sum_j^n p_j^i b_j \pmod{2} = 0 \quad \forall i = 0 \quad (6.3)$$

In matrix form, this condition is commonly written as:

$$\mathbf{P} \mathbf{b}^T = \mathbf{0} \quad (6.4)$$

It can be easily computed with:

$$\langle \mathbf{P} \mathbf{b}^T \rangle = \sum_{i=1}^m \left(\sum_{j \in N_i} b_j \right) \pmod{2} \quad (6.5)$$

where the neighborhood set N_i for the i -th row is defined as:

$$N_i = \{j \mid P_{ij} = 1, j \in \{1, 2, \dots, n\}\}$$

where P_{ij} represents the element in the i -th row and j -th column of $\mathbf{P}$: also:

- j is the index of a column in the parity-check matrix $\mathbf{P}$.
- $P_{ij} = 1$ indicates that the j -th bit x_j is included in the parity-check equation for the i -th row.
- N_i is the set of all such indices j where the j -th bit of the codeword x_j participates in the i -th parity-check equation.

When the number of ones in $\mathbf{P}$ is much lesser than the zeros in the matrix, the code is defined as a low parity density check code. The mapping of coded bits b_k to transmitted symbols employs Binary Phase Shift Keying (BPSK), is defined by:

$$x_k = M(b_k) = 2b_k - 1 \quad (6.6)$$

where x_k is the transmitted signal in the baseband.

6.2.2 Unknown Time-varying Channel

The encoded signal passes through an unknown time-varying channel, which introduces phenomena like fading and noise. In a wireless scenario, a defining characteristic is that the channels gain and phase vary over time (and frequency). Fading causes the signal amplitude and phase to fluctuate due to multipath propagation and Doppler shifts from movement. In addition, the channel adds random noise.

Mathematically, a time-varying channel can be modeled as a linear time-varying filter with impulse response $h_t(\tau)$ at time t . The output is the convolution of the input signal with the impulse response. In continuous time, the received signal $y(t)$ can be expressed as:

$$y(t) = \int_{-\infty}^{\infty} h(t, \tau) x(t - \tau) d\tau + n(t)$$

where $x(t)$ is the transmitted signal waveform corresponding to the codeword $\mathbf{x}_t$, $h(t, \tau)$ is the channels impulse response at time t , and $n(t)$ is the additive noise. This integral representation means that an input impulse at time $t - \tau$ is scaled by $h(t, \tau)$ and contributes to the output at time t . In a discrete-time equivalent model, this relation becomes a convolution sum, for example:

$$y_t = \sum_{\ell} h_t[\ell] x_{t-\ell} + n_t$$

where $h_t[\ell]$ are the time-varying channel tap gains (which can change with t). The phase and amplitude fluctuations are encapsulate in the complex nature of the channel taps.

6.2.3 Joint Decoding and Channel Estimation

The Maximum A-Posterior (MAP) decoder is of the form:

$$\hat{\mathbf{V}} = \underset{\forall \mathbf{b} \in \mathbf{C}}{\operatorname{argmax}} \prod_{i=1}^N P(\mathbf{b} = b_{1,\dots,n}) | \mathbf{Y} = y_{1,\dots,n} \quad (6.7)$$

where $P(\mathbf{b}|\mathbf{y})$ is the *a-posterior* probability function of the random vector of $\mathbf{b}$ and $\mathbf{y}$. The argument is evaluated over the interval of the set of all possible codewords, $\mathbf{C}$. Since $\mathbf{b}$ and $\mathbf{x}$ have a one to one

mapping, estimating $\mathbf{b}$ is equivalent to estimating $\mathbf{x}$. $\mathbf{y}$ is the received signal vector, $\mathbf{y} = y_1, \dots, y_n$, $\mathbf{x} = x_1, \dots, x_n$ is the transmitted BPSK symbol vector and

$$y_n = \sum_{k=0}^L h_k x_{n-k} + \eta_n \quad (6.8)$$

is a convolution of the transmitted symbols with the channel, h_k , corrupted by some noise η_n , and L is the length of the channel. It is well known that the MAP decoder reduces to the Maximum-Likelihood (ML) decoder under AWGN assumptions:

$$\hat{\mathbf{X}} = \underset{\forall \mathbf{X} \in \mathbf{C}}{\operatorname{argmax}} \prod_{i=1}^N P(\mathbf{X}|\mathbf{Y}) = \underset{\forall \mathbf{X} \in \mathbf{C}}{\operatorname{argmin}} \prod_{i=1}^N -P(\mathbf{Y}|\mathbf{X}) \quad (6.9)$$

The likelihood function, $P(\mathbf{Y}|\mathbf{X})$ is the joint probability density function received data vector conditioned on $\mathbf{h}$ and $\mathbf{X}$.

$$P(\mathbf{Y}|\mathbf{h}, \mathbf{X}) = \frac{1}{2\pi\sigma^2} \exp\left(-\frac{1}{2\sigma^2} \sum_{n=1}^N \left|y_n - \sum_{k=0}^L h_k x_{n-k}\right|^2\right) \quad (6.10)$$

Since the scaling term are the same $\frac{1}{2\pi\sigma^2}$ for each symbol, it can be omitted from the optimization, further, minimizing the product of exponents is equivalent to minimizing the sum of the argument of the exponential.

$$\hat{\mathbf{x}}, \hat{\mathbf{h}} = \underset{\forall \mathbf{x} \in \mathbf{C}, \mathbf{h}}{\operatorname{argmin}} \sum_{i=1}^N -P(y_i|\mathbf{x}, \mathbf{h}) = \underset{\forall \mathbf{x} \in \mathbf{C}, \mathbf{h}}{\operatorname{argmin}} \|\mathbf{y} - \mathbf{H}\mathbf{x}\|_2^2 \quad (6.11)$$

where, $\mathbf{H}$ is the channel Toeplitz matrix,

$$\mathbf{H} = \begin{bmatrix} h_0 & h_1 & h_2 & \cdots & h_{L-1} & 0 & \cdots & 0 \\ 0 & h_0 & h_1 & \cdots & h_{L-2} & h_{L-1} & \cdots & 0 \\ 0 & 0 & h_0 & \cdots & h_{L-3} & h_{L-2} & \cdots & h_{L-1} \\ \vdots & \vdots & \vdots & \ddots & \vdots & \vdots & \ddots & \vdots \\ 0 & 0 & 0 & \cdots & h_0 & h_1 & \cdots & h_{L-1} \end{bmatrix}$$

If we do not know the channel and the transmitted bits, and we have to estimate them then this would be a hard problem, as we have to estimate both of them from the received data. However, we have the help from the code structure, which is embedded in $\forall \mathbf{x} \in \mathbf{C}$, we can write

$$\hat{\mathbf{x}}, \hat{\mathbf{h}} = \underset{\forall \mathbf{x} \in \mathbf{C}, \mathbf{h}}{\operatorname{argmin}} \|\mathbf{y} - \mathbf{H}\mathbf{x}\|_2^2 = \underset{\mathbf{x}, \mathbf{h}}{\operatorname{argmin}} \|\mathbf{y} - \mathbf{H}\mathbf{x}\|_2^2 + \beta \langle \mathbf{P} M^{-1}(\mathbf{x})^T \rangle \quad (6.12)$$

the inner product of the second term can also be effectively calculated with:

$$\langle \mathbf{P} \mathbf{b}^T \rangle = \sum_{i=1}^m \left(\sum_{j \in N_i} b_j \right) \mod 2 \quad (6.13)$$

Instead of estimating the bits in $\mathbb{F}_2$ (Galios Field of 2) domain, we found that it is more efficient to estimate the bits in the Log Likelihood Ratio (LLR) domain. Where the LLR for a binary random variable is:

$$L(\mathbf{Y} = y|\mathbf{X}) = \ln\left(\frac{P(\mathbf{Y}|\mathbf{X} = 1)}{P(\mathbf{Y}|\mathbf{X} = -1)}\right)$$

In AWGN the LLR is a function of only the symbol x_i and is given by:

$$L(x_i) = \frac{4x_i}{\sigma^2} \quad (6.14)$$

and the modulo-2 addition between two values in the LLR domain is defined as:

$$L(d_1) \boxplus L(d_2) = \ln \left[\frac{e^{L(d_1)} + e^{L(d_2)}}{1 + e^{L(d_1)} e^{L(d_2)}} \right]$$

the modulo-2 summation between N values in the LLR domain is written as:

$$\sum_i^N L(x_i) = L(x_1) \boxplus L(x_2) \boxplus L(x_3) \boxplus L(x_4) \dots \boxplus L(x_N) \quad (6.15)$$

The inner modulo 2 summation over GF2 in (6.13) can be replaced by modulo-2 addition in the LLR domain, $\sum_{j \in N_l} L(x_i)$ in (6.15). The minimization can still be performed in both cases because as the summation in (6.5) leads to a solution close to zero, the modulo-2 summation leads to a solution that is as negative as possible. Which is what a zero would be represented in the LLR domain. So, we write,

$$\phi(\mathbf{L}(\mathbf{x})) = \sum_{l=1}^{n-k} \sum_{j \in N_l} L(x_i)$$

and the optimization problems becomes:

$$\hat{\mathbf{x}}, \hat{\mathbf{h}} = \underset{\mathbf{x}, \mathbf{h}}{\text{minimize}} \quad \|\mathbf{y} - \mathbf{H} \cdot \mathbf{x}\| + \beta \bar{\phi}(\mathbf{L}(\mathbf{x})) \quad (6.16)$$

where β is a regularized parameter, used to adjust the weights given to fulfill the constraints set by the code structure as compared to fulfilling the constraints set by the channel. If we write the cost function to the minimization problem as: $g(\mathbf{x}, \mathbf{h})$. And we seek the gradient to obtain the optimal solution, where the gradient $\nabla g(\mathbf{x}, \mathbf{h})$ is:

$$\nabla g(\mathbf{x}, \mathbf{h}) = \left[\frac{\partial g}{\partial x_1}, \frac{\partial g}{\partial x_2} \dots \frac{\partial g}{\partial x_n}, \frac{\partial g}{\partial h_1}, \frac{\partial g}{\partial h_2} \dots \frac{\partial g}{\partial h_n} \right]$$

We used Forward Mode Automatic Differentiation to obtain the gradient of the cost function, and obtain the optimal solution by performing a gradient descend.

$$\mathbf{L}^{(i+1)} = \mathbf{L}^{(i)} - \alpha \nabla g(\mathbf{L}^{(i)})$$

Where $\mathbf{L}^{(i)}$ is the combination of the estimated bits and channel taps, $\mathbf{L} \in \{\mathbf{x}, \mathbf{h}\}$.

6.3 Transmission frames

Figure 6.2 illustrates the packet-based transmission structure of a communication system employing LDPC encoding and BPSK modulation. The data undergoes LDPC encoding to introduce error correction before being modulated using binary phase-shift keying (BPSK) for transmission. The packet stream consists of multiple blocks (blk. 1, blk. 2, ..., blk. T) with a preamble and training sequence preceding the data to facilitate synchronization and channel estimation. Below blk. 1, a bit-level representation is depicted, where each packet consists of data and pilot bits. The gray compartments indicate pilot bits, which are inserted at regular intervals for channel estimation and equalization, while the white compartments represent data bits, which contain the actual transmitted information. The pilots are known bits, which

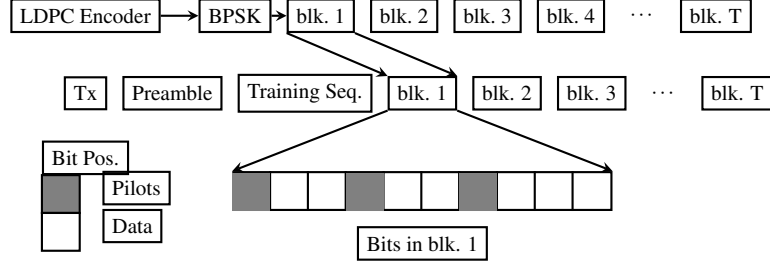


Figure 6.2: Frame organization

do not contain any information. In DFE + FEC, the pilots in the beginning help to refine the equalizer taps before going into Decision-directed mode. For DFEC, the pilots are added as an additional constrain in the optimization problem, with more constrains, the higher possibility that the final estimation is the correct one. In this setup, we let the number of pilots be a dependent variable to be varied and let the BER be the independent variable in which we want to examine.

6.3.1 Pilots

The transmitted signal $\mathbf{x}$ is a combination of data symbols $d[n]$ and pilot symbols $p[n]$, where:

$$\mathbf{x} = [x_1, x_2, \dots, x_N],$$

with $x_n \in \{p, d\}$ depending on the symbol type. Pilots can be placed randomly or at fixed positions. For the AD scheme, we used random pilot positions, and for the benchmark scheme which is a decision feedback equalizer followed by a FEC decoding scheme. The pilots has to be placed in the beginning to provide training for the equalizer. Let $\mathcal{P} \subseteq \{1, 2, \dots, N\}$ be the set of indices where pilot symbols are placed, and let $|\mathcal{P}| = M$, the number of pilot symbols. The positions of the pilots are chosen uniformly at random from the block indices:

$$\mathcal{P} = \{i_1, i_2, \dots, i_M\}, \quad i_k \sim \text{Uniform}(\{1, 2, \dots, N\}),$$

subject to the constraint that:

$$i_j \neq i_k \quad \text{for } j \neq k.$$

This ensures that each pilot index is unique. The complement of the pilot set is thus the data set indices $\mathcal{D}$ which is written as the complement of $\mathcal{P}$:

$$\mathcal{D} = \{1, 2, \dots, N\} \setminus \mathcal{P}.$$

The transmitted signal $x[n]$ can then be expressed as:

$$x[n] = \begin{cases} p[n], & \text{if } n \in \mathcal{P}, \\ d[n], & \text{if } n \in \mathcal{D}. \end{cases}$$

6.4 Experiment

We present experimental results for mid-frequency (MF) and very low frequency (VLF) bands, covering both the November 2024 experiment at St. John Island and the field experiment conducted in February 2025. Table 6.1 provides the dataset IDs and details.

ID	Frequency band	Location	Distance	Experiment Date
MF-KUSU	MF	Kusu Island	1.6 km	Feb 2025
VLF-KUSU	VLF	Kusu Island	2.7 km	Feb 2025
MF-SJI	MF	St John Island	0.5 km	Nov 2024
VLF-SJI	VLF	St John Island	0.5 km	Nov 2024

Table 6.1: Summary of experimental data.

The comparison is made between the DFEC methods developed in this work and a benchmark approach that performs equalization and decoding separately (DFE + FEC). Decoding is carried out using the Belief Propagation algorithm. The sampling rate, f_s , is 24 kSa/s for MF and 6 kSa/s for VLF. The number of samples per symbol, s_{ps} , is 3 for MF and 6 for VLF. An LDPC code with a code rate of $k/n = 1/2$ is used. The overhead, χ , defined as:

$$\chi = \frac{\text{gaps} + \text{training sequence} + \text{preamble}}{\text{frame length}}$$

is 20% for MF and 30% for VLF. The Pilot Ratio (PR), η_{pr} , is varied. The resulting data rate is given by:

$$\text{Data Rate} = f_s \times s_{ps} \times \frac{k}{n} \times (1 - \chi) \times \eta_{pr}.$$

Figure 6.4 presents the average BER of all transmitted frames as a function of the Pilot Ratio. Since other variables affecting the data rate remain constant, the data rate and pilot ratio have a one-to-one relationship. The left column shows results for all experimental settings, where we observe that DFEC consistently outperforms DFE + FEC in terms of BER, except in the case of VLF-SJI at high pilot ratios. In the right column, we illustrate the operating regions of both schemes. The region is lower bounded by the throughput based on the frame success rate, assuming that any frames with even a single bit error is discarded. In practice, one may use an outer FEC code to correct such errors. The code rate of such an outer code is bounded by the channel capacity. Hence, the upper bound of the region is determined by the channel capacity of the Binary Symmetric Channel (BSC):

$$C = 1 - H(p),$$

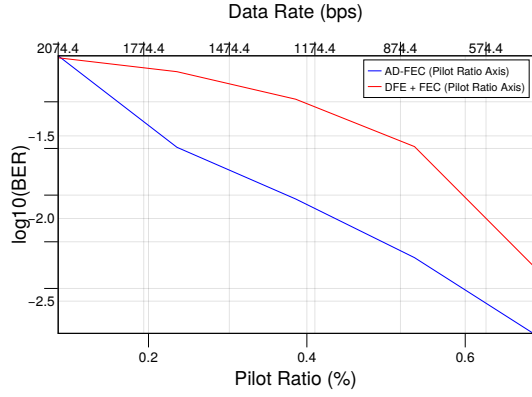
where the binary entropy function is given by:

$$H(p) = -p \log_2(p) - (1 - p) \log_2(1 - p).$$

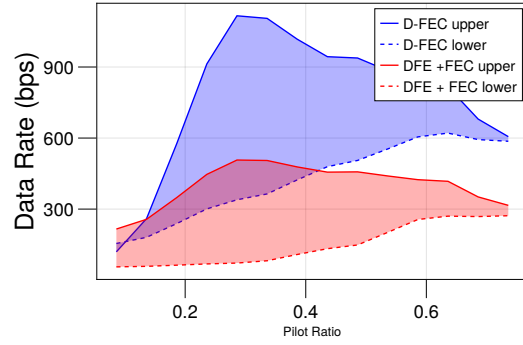
We tabulate the optimal pilot ratios and the equivalent achievable data rates based on Figures 6.3 and 6.4 in Table 6.2. Across all experimental settings, it is evident that DFEC outperforms DFE + FEC. This indicates that DFEC is a strong candidate for achieving higher performance in future modems.

6.5 Discussion

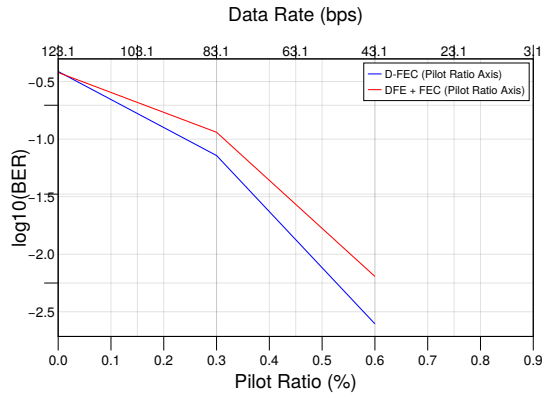
We have demonstrated that DFEC outperforms the benchmark DFE + FEC scheme in both MF and VLF bands during several experiments. The DFEC technique allows us to jointly perform decoding and equalization, which may lead to the development of novel communication techniques in the future. While the achieved MF data rates with DFEC seem lower than the data rates achieved using OFDM + LDPC, we



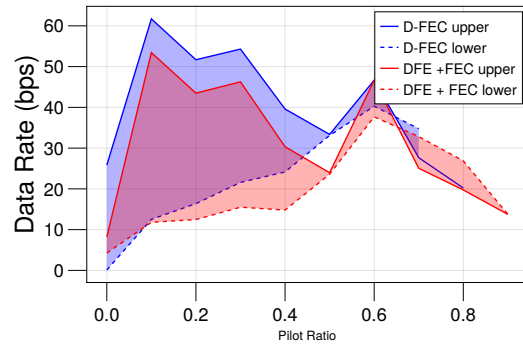
(a) BER Vs Pilot Ratio for MF-KUSU



(b) Potential operating regions for DFEC and DFE + FEC for MF-KUSU



(c) BER Vs Pilot Ratio for VLF-KUSU



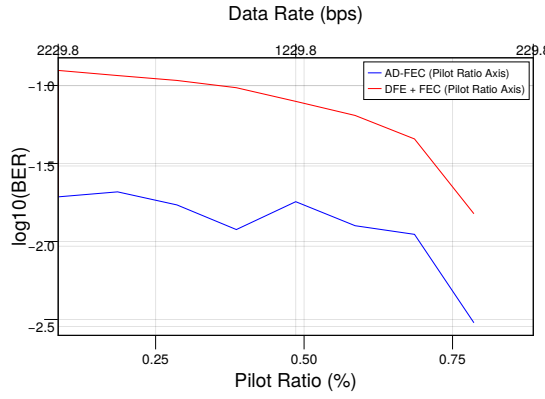
(d) Potential operating regions for DFEC and DFE + FEC for VLF-KUSU

Figure 6.3: DFEC performance in KUSU experiments. In the left column are all the BER vs PR for each experiment settings. In the right column is the potential operating regions for the benchmark and DFEC scheme. The blue region is the operating region for DFEC and the red region is for the benchmark DFE + FEC.

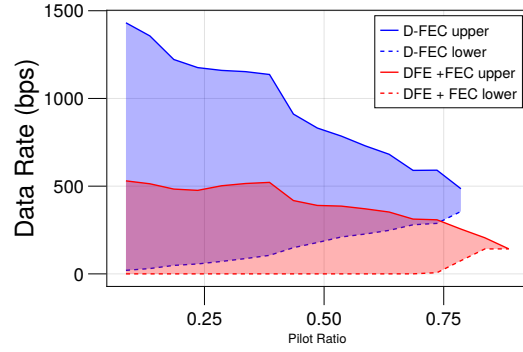
cannot directly compare them. Instead, we should compare the performance of DFEC against separate equalization and decoding with a standard DFE + FEC scheme.

DFEC is an error correction technique that we applied to single carrier communication systems not optimized for Singapore waters. We chose a single carrier system as a first step in development of DFEC, as the systems use equalizers to operate – and DFEC lends itself to combining with equalizers to highlight the benefit of differentiability. OFDM, on the other hand, has been optimized by us over the years for use in Singapore waters, but we avoid equalization in OFDM at the cost of a cyclic prefix. An equalized single carrier or OFDM system can achieve better data rate than cyclic prefix OFDM if the equalization and error correction is done optimally. DFEC opens the doors for this possibility. Further development of DFEC may include optimization of single carrier systems and equalized OFDM systems – both of these have the potential for higher speed communication than possible with our OFDM systems today.

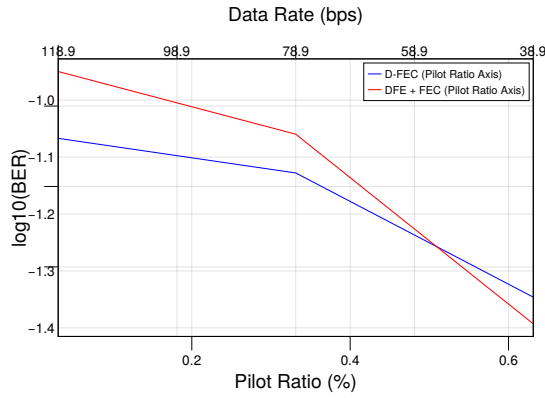
A current limitation of DFEC is the high computational cost of the optimization, making this scheme unsuitable for real-time operation in small devices such as modems. As computing power in embedded devices is rapidly increasing, and GPUs are being added to such devices, we expect that DFEC will



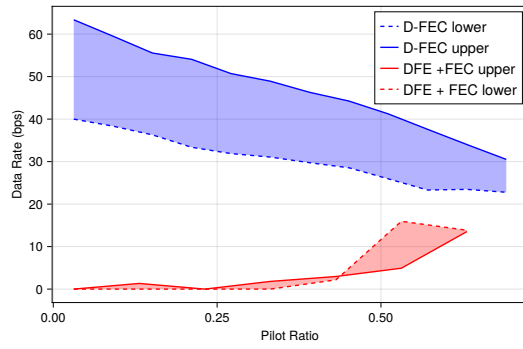
(a) BER Vs Pilot Ratio for MF-SJI



(b) Potential operating regions for DFEC and DFE + FEC for MF-SJI



(c) BER Vs Pilot Ratio for VLF-SJI



(d) Potential operating regions for DFEC and DFE + FEC for VLF-SJI

Figure 6.4: DEFC performance in SJI experiments. In the left column are all the BER vs PR for each experiment settings. In the right column is the potential operating regions for the benchmark and DFEC scheme. The blue region is the operating region for DFEC and the red region is for the benchmark DFE + FEC.

be implementable on modems in the near future. Moreover, in this project, we focused on the theoretical development and demonstration of DFEC, and did not focus on optimizing its computational performance. We expect that, with some effort, we should be able to improve the implementation of DFEC to make it suitable for use on modems.

Scheme	MF-KUSU	VLF-KUSU	MF-SJI	VLF-SJI
DFEC				
Pilot ratio	0.3	0.1	0.0	0.0
Data rate	1.1 kbps	62 bps	1.4 kbps	62 bps
DFE + FEC				
Pilot ratio	0.3	0.1	0.4	0.6
Data rate	500 bps	52 bps	500 bps	15 bps

Table 6.2: The optimal pilot ratios and achievable data rates (BSC channel capacity) for all experiment setting

CHAPTER 7

DIGITAL TWIN FOR LINK ADAPTATION

7.1 Introduction

Link adaptation, sometimes termed as adaptive modulation and coding (AMC), is key to achieving good communication performance in dynamic wireless environments. While terrestrial wireless systems such as WiFi perform link adaptation by testing various modulation parameters by transmitting frames through air, the same approach underwater is inefficient. Long frame durations make testing of acoustic frames through water expensive in terms of time and energy. Long propagation delays to obtain feedback further exacerbates the problem. If a *digital twin* of the acoustic environment was available, one might perform link adaptation digitally using the twin, without expending time or energy needed for acoustic transmissions and obtaining feedback. However, since each ocean environment is different and often dynamic, we cannot use a fixed digital twin to model the ocean. Instead, we require a digital twin that can mimic the operating environment by making just a few quick measurements through water.

One option to build a digital twin is *direct replay* [4]. The key idea of this method is to transmit a *probe* signal through the water, record the receiver signal, and estimate a time varying impulse response (TVIR) using an adaptive channel estimator. The TVIR can then be used as a digital twin. Any communication signal to be transmitted can be convolved with the TVIR digitally, and then noise added. This faithfully mimics transmission through water, and generates a signal similar to one that would have been received had the communication signal been transmitted through water instead of the probe. The procedure can be rapidly repeated for many communication signals to find a modulation and coding scheme that would work best in the environment. While attractive, the direct replay method has some shortcomings. If a single probe is used as a digital twin, it does not adequately represent the time-varying nature of the ocean. Using a large number of probes solves this problem, but incurs substantial time and energy costs.

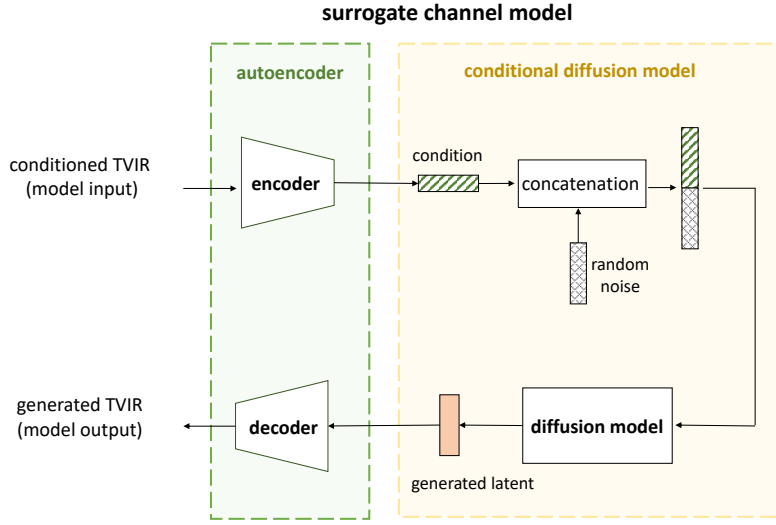


Figure 7.1: An overview of the architecture of our proposed digital twin.

An alternative explored in literature is *stochastic replay* [5]. However, the channels produced by stochastic replay do not usually capture the temporal characteristics of the channel adequately.

Recently generative AI techniques have been wildly successful for many tasks. In this project, we set out to explore if such techniques could be adapted to generate realistic TVIRs for use as a digital twin for link adaptation. Given a measured TVIR as a *condition* to seed a generative AI, could we generate a large number of novel TVIRs with similar characteristics as the measured TVIR but variability as one might expect in the ocean? We described a method based on conditional latent diffusion for generating static channel impulse responses (CIR) in deliverable D3. We extend this method to generate TVIRs in this chapter, and experimentally explore its usability for the purposes of link adaptation.

The high dimensionality of TVIRs makes direct training with the original data inefficient for both the link tuning and conditional diffusion models. To overcome this, we propose using an autoencoder to compress TVIRs into a compact latent space, significantly reducing computational complexity. These latent representations are then used to generate channel realizations with the diffusion model and serve as inputs to the adaptive link tuning model for selecting candidate communication schemes.

7.2 Generative AI based digital twin

7.2.1 Overview of system architecture

Figure 7.1 illustrates the architecture of the proposed digital twin, which consists of two submodels: an autoencoder and a conditional diffusion model. We explored various model architectures, each with its own advantages and limitations.

Based on our experience in Singapore waters, we assume that environmental changes occur at a Doppler spread of no more than 10 Hz and a delay spread of about 20 ms. We use 20 CIRs to represent a TVIR spanning 1 second, in accordance with the Nyquist sampling rate. Each CIR consists of 250 complex values, sampled at 12 kHz to cover a 20 ms delay spread, resulting in each TVIR input having 20×250 complex-valued elements. Figure 7.8 presents a simulated TVIR, showing its amplitude variations over 1 second.

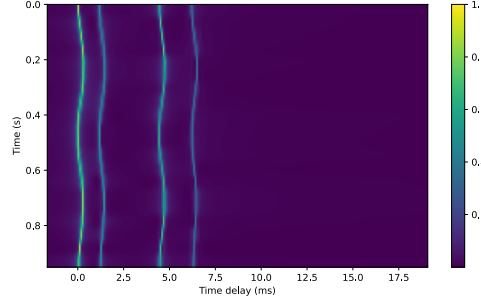


Figure 7.2: An example of simulated TVIR over 1 second.

7.2.2 Autoencoder

The autoencoder is designed to reduce the dimensionality of the TVIRs while preserving its key features. TVIRs consist of a sequence of CIRs, and the time-varying nature of the underwater acoustic channel introduces correlations across consecutive CIRs. Long Short-Term Memory (LSTM) networks are well-suited for modeling such time-varying patterns, as they can retain long-term memory, making them ideal for encoding and decoding sequential data [6]. The autoencoder compresses the original 20×250 complex-valued data in a TVIR into a single 128-dimensional real-valued latent representation, achieving a compression ratio of approximately $78 \times$ between autoencoder input and output. In the communication process, we first downsample the channel responses by $600 \times$ (from 12 kHz to 20 Hz) before encoding them with the autoencoder. After reconstruction, we upsample the data back to the original sampling rate for communication evaluation. The results show that the communication performance remains comparable to that of the original TVIR sampled at 12 kHz. This demonstrates that our autoencoder, combined with downsampling, effectively achieves an overall compression ratio of $46,800 \times$ while preserving communication performance.

We evaluate two LSTM autoencoder architectures, where the encoder and decoder share identical network structures, differing only in their operational direction. Method 1 pre-encodes the CIR inputs with an additional encoder before using the LSTM encoder, which simplifies feature learning for LSTM networks but struggles to reconstruct fine details. In contrast, Method 2 directly processes the TVIR with the LSTM autoencoder, achieving better reconstruction accuracy. By finding the most robust input representation of the TVIR, Method 2 enables the real-valued LSTM to process it appropriately, overcoming the limitations of Method 1.

7.2.2.1 Method 1 Since LSTM layers in PyTorch [7]¹ cannot process complex-valued inputs directly, and the large input size complicates LSTM autoencoder training, we first use a CIR autoencoder to reduce the size of each CIR before passing it into the LSTM network. In report D3, Chapter 2.2.2, we processed the real and imaginary parts of each CIR separately using two distinct autoencoders, followed by a separate network to fuse the encoded amplitude and phase into the final latent representation for each CIR. While this approach simplified training by making both autoencoder inputs real-valued, it resulted in a loss of the inherent relationship between amplitude and phase, leading to suboptimal reconstruction accuracy, particularly for the phase component. To address this limitation, we use complex-valued neural networks

¹PyTorch is a popular open-source machine learning library, built on the Python programming language and the Torch library. It is extensively used for applications in artificial intelligence, particularly deep learning, offering powerful tools for automatic differentiation and GPU acceleration [7].

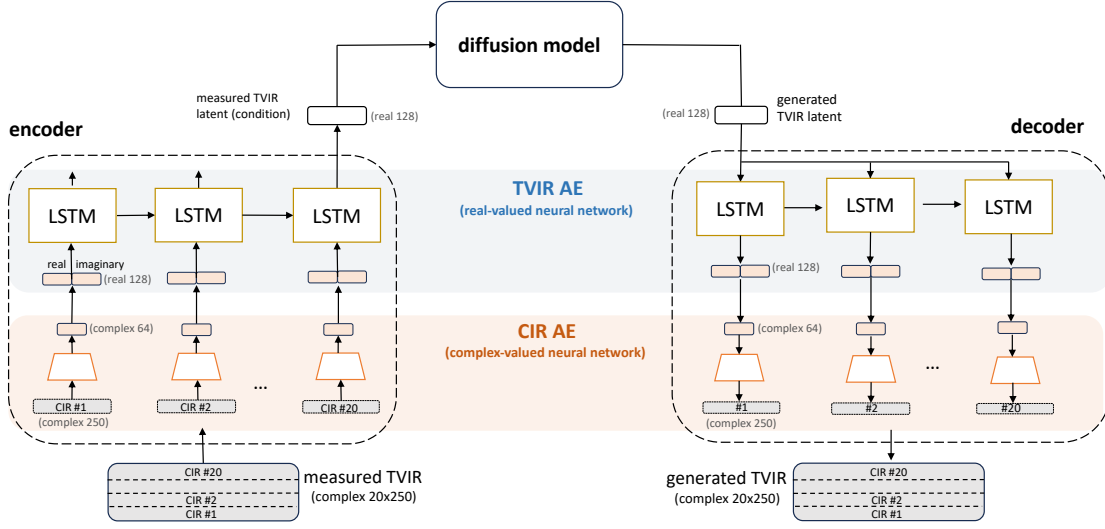


Figure 7.3: An overview of the autoencoder design for Method 1, with the data types and dimensions specified at each step.

in this report. These networks extend traditional architectures by directly processing complex numbers, incorporating complex weights, biases, and activation functions to handle both real and imaginary components simultaneously [8]. This is especially useful in signal processing, communications, and physics, where complex numbers are inherent. By maintaining the mathematical relationships between the real and imaginary parts, this design ensures accurate modeling and reconstruction of complex-valued data. Applying this approach to complex-valued CIRs preserves the physical coherence between amplitude and phase, resulting in more accurate representations and reconstructions compared to separately processing the real and imaginary components.

Specifically, we use a complex-valued neural network to compress each 250-element complex-valued CIR into 64 complex latents. The real and imaginary components are then concatenated into a 128-dimensional real-valued latent per CIR. A sequence of 20 such compressed CIRs, each with 128 real-valued components, is then sequentially processed by the LSTM encoder, producing a compact 128-dimensional real-valued latent representation that captures the original 20×250 complex-valued TVIR matrix. Figure 7.3 illustrates the detailed design of Method 1.

The complex-valued CIR autoencoder uses one complex-valued convolutional layer followed by three complex-valued dense layers. The details of these layers are provided below:

Layer	Filter size	Activation function	Kernel size	Stride	Padding
Convolution	24	LeakyReLU	3	2	1
Flatten	-	-	-	-	-

Layer	Activation function	Output size
Dense	LeakyReLU	1024
Dense	LeakyReLU	1024
Dense	-	64

PyTorch does not natively support complex-valued neural networks, so we implement custom layers and activation functions using real-valued operations. Complex numbers are represented as pairs of real-valued tensors for the real and imaginary components, which are processed separately while adhering to complex arithmetic rules. This approach allows complex-valued models to be trained using standard PyTorch optimizers while preserving key mathematical properties. The following details the mathematical operations used to implement complex functions with real-valued tensors.

In a fully connected dense layer, if we have a complex weight matrix $w = w_r + iw_i$, complex bias $b = b_r + ib_i$ and complex input $x = x_r + ix_i$, the output of the dense layer can be expanded as:

$$\begin{aligned} wx + b &= (w_r + iw_i)(x_r + ix_i) + (b_r + ib_i) \\ &= (w_r x_r - w_i x_i + b_r) + i(w_r x_i + w_i x_r + b_i). \end{aligned} \quad (7.1)$$

Thus, the real and imaginary parts of the output are:

$$z_r = w_r x_r - w_i x_i + b_r, \quad (7.2a)$$

$$z_i = w_r x_i + w_i x_r + b_i. \quad (7.2b)$$

We can implement this complex operation using real-valued tensors to compute the real and imaginary parts separately. A complex-valued 1D convolutional network follows a similar approach to a complex-valued dense layer, but the kernels must follow to complex arithmetic operations. For a complex ReLU, we apply ReLU separately to real and imaginary parts:

$$\text{ReLU}(z_r + iz_i) = \text{ReLU}(z_r) + i\text{ReLU}(z_i). \quad (7.3)$$

Complex-valued neural networks inherently preserve the relationship between amplitude and phase while naturally accounting for the fact that phase components with zero corresponding amplitudes do not affect overall multipath effects. This allows the loss function to directly minimize reconstruction loss without requiring manual weighting of different elements. We apply the absolute value of the mean square error (MSE) loss function to the complex form of CIRs:

$$\mathcal{L}_{\text{CAE}}(x) = \sum_{i=1}^{250} |\text{CAE}(x)_i - x_i|^2, \quad (7.4)$$

where x denotes the complex-valued CIR in the original data space, while CAE refers to the complex-valued autoencoder model designed to process individual CIRs within the TVIR, as illustrated in Figure 7.3, the notation $\text{CAE}(x)_i$ represents the i^{th} element of the CIR reconstructed by the CIR autoencoder proposed in Method 1. This loss function is applied to individual data points and summed across the entire batch for batch training.

Once the CIR autoencoder is trained, the encoded CIR sequence from the TVIRs serves as input to the LSTM autoencoder to capture temporal variations. LSTM, a type of recurrent neural network (RNN), is designed to process sequential data by maintaining long-term dependencies through its gated architecture. It uses input, forget, and output gates to regulate the flow of information, ensuring that relevant data is retained while irrelevant details are discarded. This mechanism helps mitigate vanishing gradient issues common in traditional RNNs, making LSTMs highly effective for modeling long-term dependencies in complex sequence. Figure 7.4 illustrates the standard architecture of an LSTM cell, while Figure 7.5 depicts the structure of the LSTM autoencoder, which consists of three stacked LSTM layers in both the encoder and decoder.

We concatenate the real and imaginary parts of the CIR autoencoder's output, which consists of 64 complex-valued elements, to form a 128-dimensional real-valued input for the LSTM network. We then implement the LSTM layers using PyTorch's built-in functions. To improve the LSTMs ability to capture

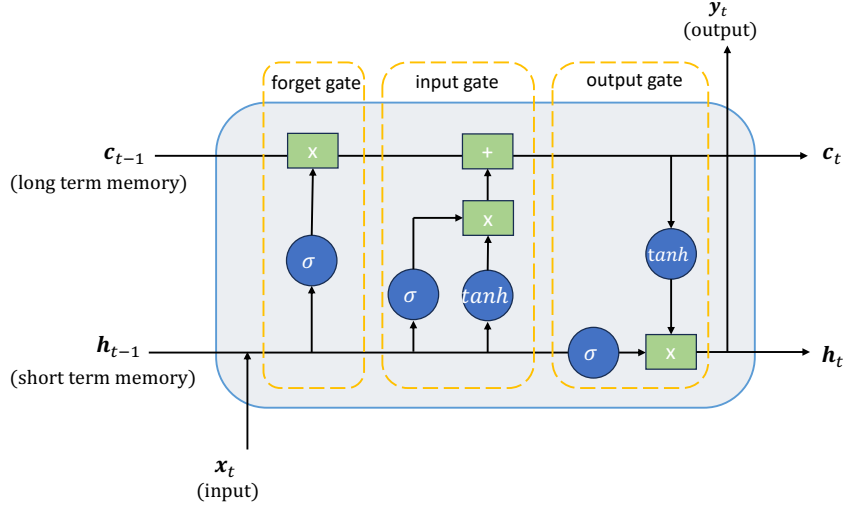


Figure 7.4: Illustration of an LSTM cell.

complex temporal relationships, we stack three LSTM layers in both the encoder and decoder, with initial cell states set to zero. During encoding, the first encoded CIR latent of the TVIR is fed into the first LSTM cell, and subsequent encoded CIR latents are passed through the LSTM, with cell states carried forward across time steps. The final encoded TVIR latent is taken from the last time steps state. In the decoding stage, the real-valued latent, either the encoded latent for autoencoder training or the latent generated by the conditional diffusion model during inferring, is used as input at all time steps of the LSTM cells. The initial cell states are again set to zero, and cell states from the second time step onward are carried forward. The sequence of CIR latents collected from each time step serves as the output of the LSTM. The 64 complex-valued outputs are then reformatted, with the first 64 elements representing the real part and the last 64 representing the imaginary part. After decoding each CIR, we reverse the sequence to reconstruct the final TVIR in complex form. The loss function used to train the LSTM autoencoder in Method 1 minimizes the MSE on the encoded latent sequence for each TVIR:

$$\mathcal{L}_{\text{LAE1}}(\mathbf{Y}) = \sum_{i=1}^{20} \sum_{j=1}^{128} |\text{LAE1}(\mathbf{Y})_{i,j} - \mathbf{Y}_{i,j}|^2, \quad (7.5a)$$

$$\mathbf{Y} = \begin{bmatrix} \mathbf{y}_1 \\ \mathbf{y}_2 \\ \vdots \\ \mathbf{y}_{20} \end{bmatrix} = \begin{bmatrix} E_{\text{CAE}}(\mathbf{x}_1) \\ E_{\text{CAE}}(\mathbf{x}_2) \\ \vdots \\ E_{\text{CAE}}(\mathbf{x}_{20}) \end{bmatrix}, \quad (7.5b)$$

where LAE1 is the proposed LSTM autoencoder in Method 1, E_{CAE} is the encoder of the CIR autoencoder that encodes CIR $\mathbf{x}_i$, and $\mathbf{Y}$ contains the sequence of 20 encoded CIR latents in a TVIR, with each encoded CIR latent consisting of 128 elements.

Unfortunately, while the proposed two-stage autoencoder model achieves good reconstruction performance for significant arrivals, it struggles with weak arrivals². These weak arrivals, usually found at

²Weak arrivals refer to those with strong path loss due to multiple reflections, and they typically appear with larger delays.

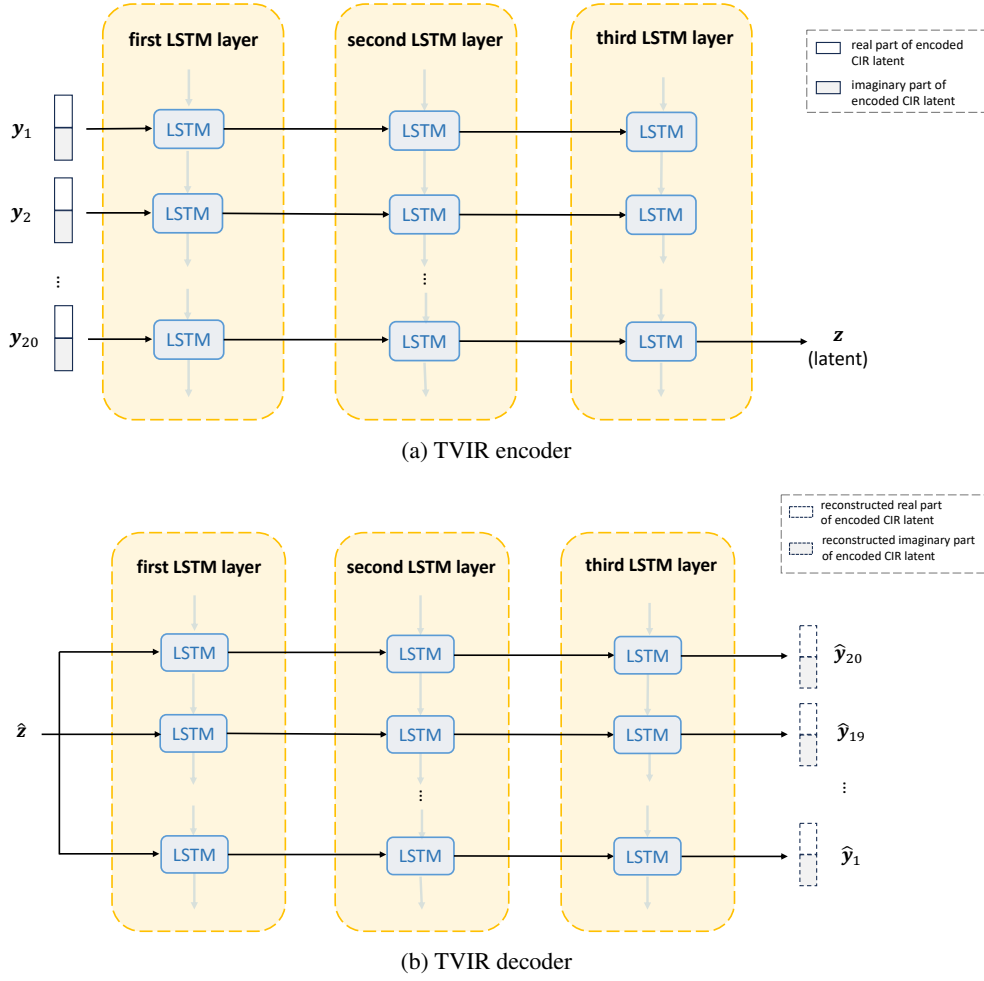


Figure 7.5: Model structure of the LSTM autoencoder used in Method 1. Here, y_i denotes the i^{th} CIR latent output from the CIR encoder, while z represents the final latent representation of a TVIR, consisting of 128 real-valued elements. During autoencoder training, $\hat{z}$ is identical to z , but when the encoder and decoder are used separately, $\hat{z}$ can be a generated latent output from the diffusion model. The notation $\hat{y}_i$ refers to the i^{th} reconstructed CIR latent at the output of the LSTM decoder.

later indices in each CIR, are often poorly reconstructed. This issue is likely caused by training the CIR autoencoder and LSTM autoencoder separately, which disrupts the temporal continuity of the encoded latent representation in the TVIR. Although the CIR autoencoder reconstructs individual CIRs well, its encoded latent may not preserve the temporal correlations necessary for the LSTM autoencoder to learn the sequential dependencies it is good at. While training both models end-to-end could help mitigate this issue, it is computationally expensive. A common strategy for large models is to pre-train each component separately before co-training, but determining the optimal stopping point for pre-training is heuristic, which led to suboptimal results in our case.

7.2.2.2 Method 2 We propose Method 2 to address the limitations of Method 1. In this approach, the TVIR matrix is directly fed into the LSTM autoencoder, allowing the LSTM cells to capture the temporal dependencies inherent in the TVIR data and generate a compact 128-dimensional latent representation. To accommodate the complex-valued TVIR matrix using real-valued LSTM layers in PyTorch, a common approach is to concatenate the normalized amplitude and phase along the feature dimension. The phase of a specific arrival in a TVIR can experience sudden jumps due to phase wrapping within the range of $[-\pi, \pi]$, where even a small change in phase near π or $-\pi$ can cause the phase to undergo a rapid shift. This discontinuity occurs because phase is a cyclical quantity, and minor changes in phase values near the boundaries can result in a large variation in the observed phase. Consequently, the phase components of CIRs in a TVIR do not always change smoothly over time, which makes it challenging for the LSTM networks to learn the underlying patterns effectively. Therefore, a key challenge is normalizing both the amplitude and phase to a similar range across all training data while ensuring smooth transitions over time without sudden jumps. The amplitude can be easily normalized to a relaxed range of $[0, 1]$ by ensuring that the maximum amplitude of the first CIR in each TVIR is 1³. Phases can be unwrapped to eliminate discontinuities, ensuring a smooth representation over time. However, the range of the unwrapped phase can vary significantly depending on the specific case, making it difficult to preset a phase normalizing factor for all training data. If the unwrapped phase has a much larger range compared to the amplitude, the learning process could become dominated by the phase, rather than achieving a balanced accuracy between phase and amplitude. This imbalance can potentially hinder effective learning.

To smooth the phase while keeping its range manageable, we represent it using Euler's formula with sine and cosine:

$$e^{i\phi} = \cos \phi + i \sin \phi. \quad (7.6)$$

This ensures smooth transitions, as sine and cosine functions are continuous, and their values are bounded within $[-1, 1]$. The input to the LSTM encoder is then a concatenation of the normalized amplitudes and the normalized sine and cosine of the phases. This results in an input of 750 elements per CIR. All concatenated inputs are normalized to a relaxed range of $[0, 1]$ for consistency. For the LSTM autoencoder, the objective is to learn how to encode each input sequence (a TVIR) consisting of 750 elements and a sequence length of 20 into a compact real-valued latent representation of size 128. In the decoded CIRs of a TVIR, the first 250 elements correspond to the normalized amplitude, the next 250 elements represent the normalized sine of the phase, and the remaining 250 elements represent the normalized cosine of the phase. We can extract phase ϕ from its cosine and sine by:

$$\phi = \tan^{-1} \left(\frac{\sin \phi}{\cos \phi} \right). \quad (7.7)$$

In our initial experiments with single-directional LSTMs, the model struggled to capture dependencies well enough, as it only considered past time steps. To address this, we use bi-directional LSTM (Bi-

³The relaxed range allows the largest amplitude in each CIR to exceed 1 due to the time-varying channel. However, we normalize the maximum amplitude of the first CIR to 1, and while other CIRs may have different amplitudes, their maximum values should not deviate much from 1, as the TVIR spans only 1 second.

LSTM) layers in both the encoder and decoder, enabling the model to capture dependencies from both past and future time steps. This is particularly beneficial for TVIRs, where correlations extend across the entire sequence. Bi-LSTMs allow the encoder to create a more comprehensive latent representation and help the decoder generate more accurate reconstructions by leveraging the full temporal context. The architecture of the proposed Bi-LSTM autoencoder is illustrated in Figure 7.6.

We have experimented with various loss functions to optimize reconstruction performance. Initially, we use MSE on the complex-valued representation of the reconstructed TVIR and the ground truth TVIR, as this inherently preserves the relationship between amplitude and phase. However, we find that using standard MSE on the concatenated real-valued form of amplitudes and the sine and cosine of the phases results in better reconstruction performance. Additionally, we test weighting the MSE on the sine and cosine of the phases by their corresponding amplitudes. This approach emphasizes significant arrivals while reducing the influence of phase components where the corresponding amplitude is near zero, given the sparsity of CIRs. This weighted approach yields the best results. The final loss function for the LSTM autoencoder in Method 2 is defined as follows:

$$\mathcal{L}_{\text{LAE2}} = \mathcal{L}_{\text{amp}} + \mathcal{L}_{\text{phase}}, \quad (7.8a)$$

$$\mathcal{L}_{\text{amp}} = \sum_{i=1}^{20} \sum_{j=1}^{250} (|A_{i,j} - \hat{A}_{i,j}|^2), \quad (7.8b)$$

$$\mathcal{L}_{\text{phase}} = \sum_{i=1}^{20} \sum_{j=1}^{250} \left(|A_{i,j} \sin(\phi_{i,j}) - \hat{A}_{i,j} \sin(\hat{\phi}_{i,j})|^2 + |A_{i,j} \cos(\phi_{i,j}) - \hat{A}_{i,j} \cos(\hat{\phi}_{i,j})|^2 \right), \quad (7.8c)$$

where $A_{i,j}$ and $\phi_{i,j}$ denote the ground truth amplitude and phase of the j^{th} element in the i^{th} CIR, $\hat{A}_{i,j}$, $\sin(\hat{\phi}_{i,j})$ and $\cos(\hat{\phi}_{i,j})$ represent the reconstructed amplitude and the sine and cosine of the phases, respectively, output from the LSTM autoencoder model proposed in Method 2. This loss function is computed for a single data point and then summed over the entire batch during training to facilitate batch-wise optimization.

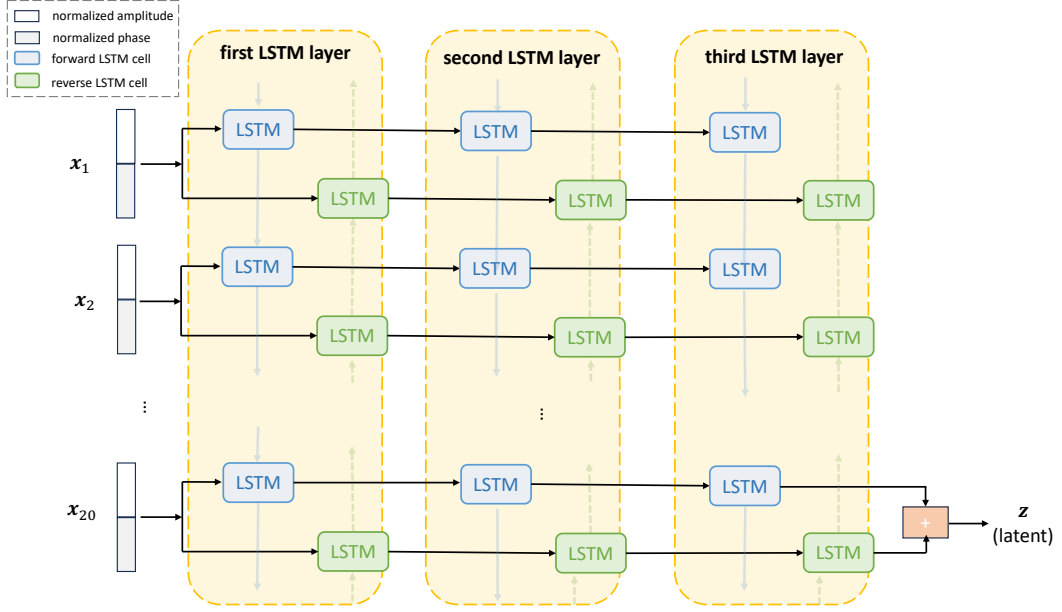
Compared to Method 1, the autoencoder in Method 2 achieves better reconstruction performance, even for weak arrivals. Therefore, we use the Method 2 autoencoder to encode measured TVIRs for both adaptive link tuning models and the conditional diffusion model.

7.2.3 Conditional diffusion model

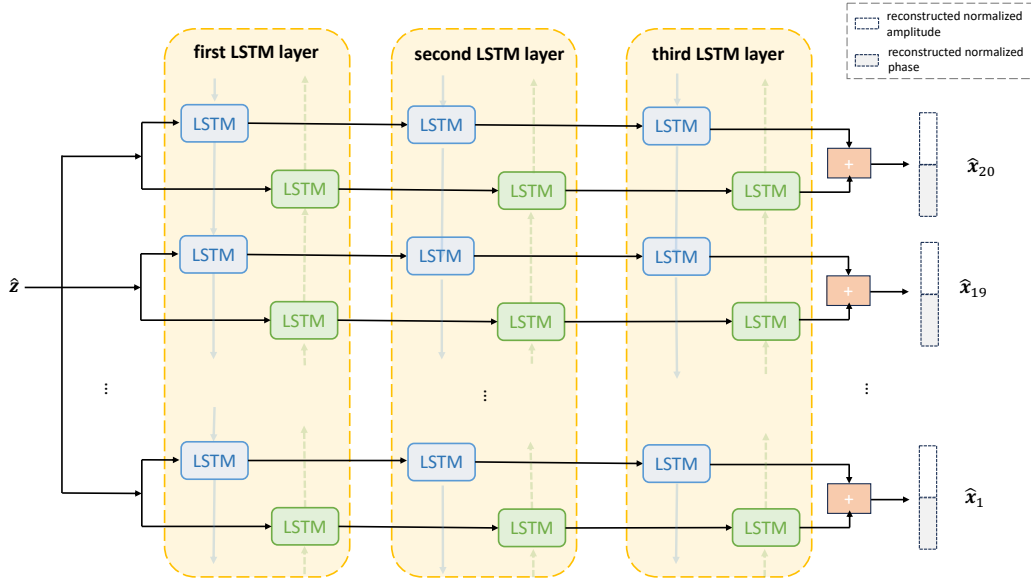
We use the conditional diffusion model as the core technique to generate realistic channel responses. The detailed introduction of a standard conditional diffusion model is provided in the D3 report. Please refer to Report D3, Chapter 2.2.3 for more information. To improve the generation performance of the diffusion model, we have explored several enhancements. In our latent conditional diffusion model proposed in this report, we implement skip connections to facilitate the direct flow of information across layers. This approach helps mitigate the vanishing gradient problem, allowing the model to learn better while retaining essential features from both the input and intermediate representations. Additionally, we incorporate layer normalization for each fully connected layer to stabilize training and improve convergence. The model structure of our proposed conditional diffusion model is illustrated in Figure 7.7.

7.3 Simulation

Data-driven machine learning models, such as LSTM autoencoders and diffusion models, typically require large amounts of high-quality training data. However, obtaining large experimental datasets with a high signal-to-noise ratio is often challenging. To tackle this, we first pre-train our LSTM autoencoder and



(a) Bi-directional TVIR encoder



(b) Bi-directional TVIR decoder

Figure 7.6: Bi-directional LSTM autoencoder is used in Method 2. Here, x_i represents the i^{th} CIR input of a TVIR to the LSTM encoder, while z denotes the final latent representation of a TVIR, consisting of 128 real-valued elements. During autoencoder training, $\hat{z}$ is identical to z , but when the encoder and decoder are used separately, $\hat{z}$ can be a generated latent output from the diffusion model. $\hat{x}_i$ is the i^{th} reconstructed CIR at the output of the LSTM decoder. The weights of the three stacked LSTM layers are shared across different step sequences.

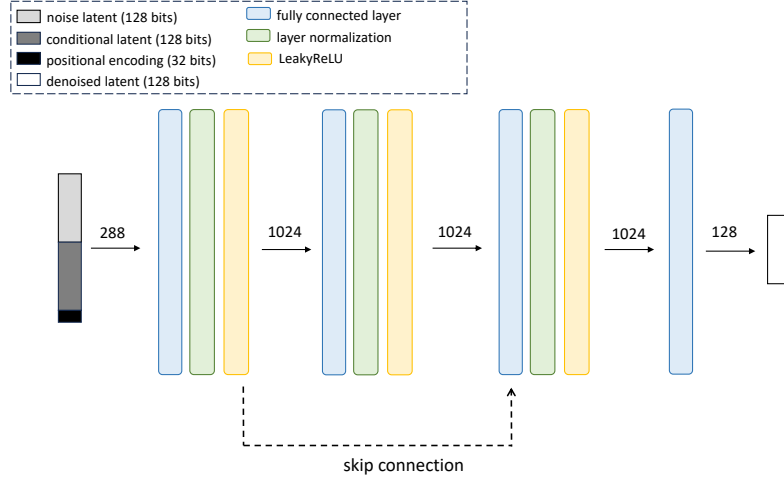


Figure 7.7: Model architecture of proposed latent conditional diffusion model, with output sizes labelled at each step.

diffusion model on a large set of simulated data, allowing them to learn essential underlying features. We then fine-tune the models using a small amount of sea-measured data to capture realistic phenomena. This approach enables a thorough evaluation of the proposed method in this section, while the next section, Section 7.4, focuses on experimental validation using real-world data.

7.3.1 Data generation

Underwater acoustic channel responses are generated using a physics-based propagation model by defining environmental parameters. We simulated 1,000,000 underwater environments by randomly sampling parameters within the ranges specified in report D3 Chapter 2.3.1. For each environment, we used the Ray Tracer from the UnderwaterAcoustics.jl package [9] to generate a nominal CIR with a 20 ms delay spread. We used an acoustic channel simulator [10] to extend the CIRs into TVIRs by incorporating random time-varying environmental parameters, simulating dynamic changes in the underwater environment. Figure 7.8 shows a nominal CIR generated by a ray tracer and the corresponding TVIR produced by the channel simulator.

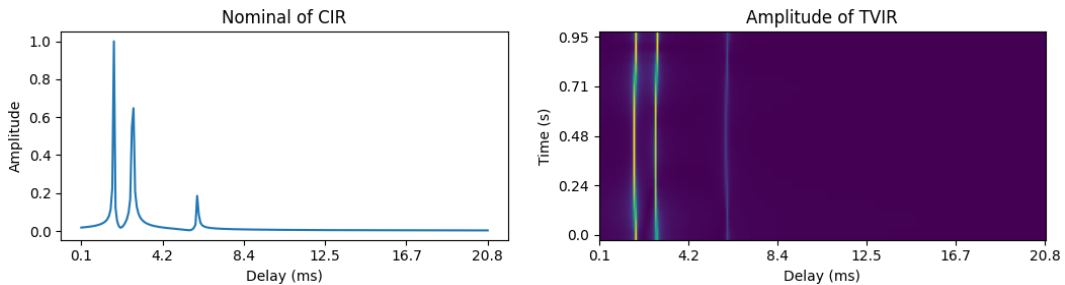


Figure 7.8: An example of nominal CIR and its corresponding TVIR generated by the channel simulator.

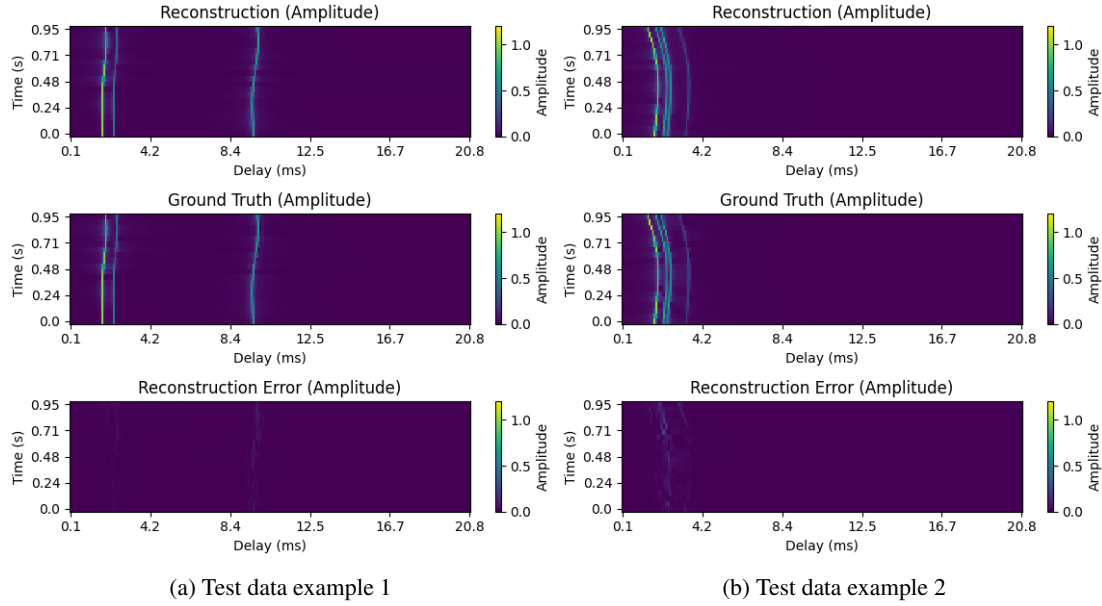


Figure 7.9: Two simulated test data are used to evaluate the reconstruction performance of the pre-trained LSTM autoencoder proposed in the Method 2.

7.3.2 Autoencoder performance

We pre-train the LSTM autoencoder model proposed in Method 2 using clean simulated data to learn key channel features. The training data consists of simulated TVIRs from 900,000 environments, with 99,500 environments used for validation and the last 500 reserved for testing. The initial learning rate is set to 0.001, reducing by a factor of 10 if the validation error does not improve for 3 consecutive epochs, with early stopping implemented. Training terminates when the learning rate falls below 10^{-7} . This pre-training stage typically takes a few hours on GPU. Figure 7.9 shows two examples of reconstruction performance using the pre-trained autoencoder on a clean test dataset. The autoencoder successfully captures key variations in amplitude, even when there is significant time variability and generalizes well to unseen environments.

7.3.3 Diffusion model performance

To pre-train the diffusion model on simulated data, we prepare both the conditioned latents and the corresponding ground truth latents. Using the channel simulator, we generate 40 consecutive CIRs per TVIR over a 2-second duration from nominal CIRs. The first 20 CIRs form the conditioned TVIR, while the last 20 CIRs represent the ground truth of generated TVIR. The goal is to train the latent diffusion model to generate TVIR latents that are statistically similar to the conditioned ones when decoded back into the original signal space using the LSTM decoder. Similar to the pre-training of the LSTM autoencoder, we use 900,000 simulated conditioned and ground truth latent pairs, encoded by the pre-trained autoencoder, for training, with 99,500 reserved for validation and 500 for testing. Training starts with a learning rate of 0.001. If the validation error does not improve over 5 consecutive epochs, the learning rate is reduced by a factor of 10. Early stopping is also applied to halt training if no further improvements are observed and learning rate drops below 10^{-7} . We set the number of denoising steps to 100 and use a Sigmoid-based noise scheduler. This pre-training stage typically takes one to two hours on GPU.

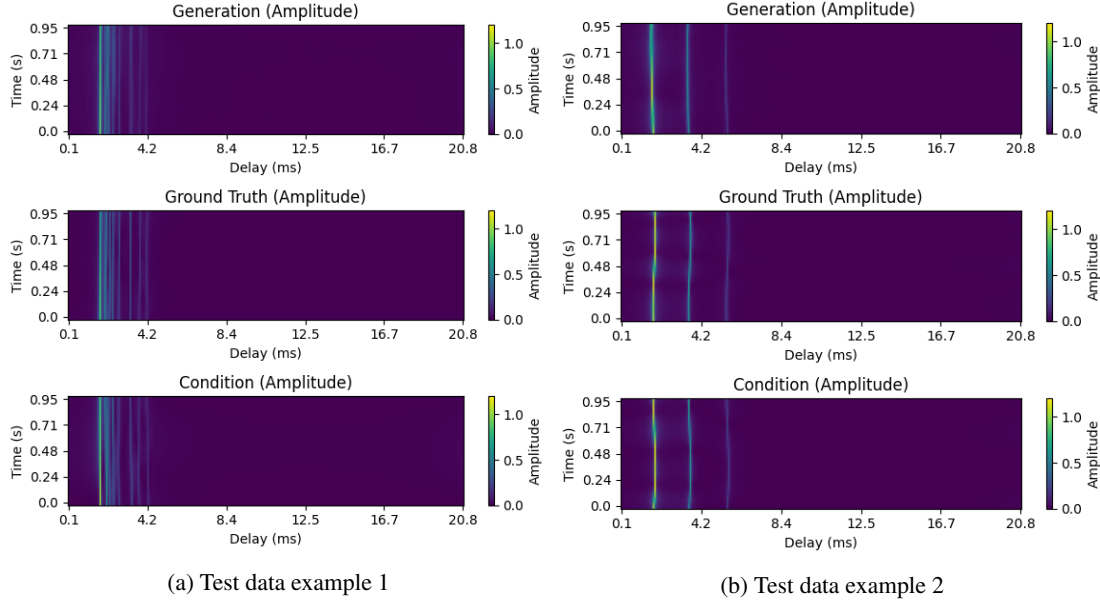


Figure 7.10: Two examples of clean simulated test data used to evaluate the pre-trained conditional diffusion model.

Figure 7.10 shows two examples generated by the pre-trained diffusion model using simulated data. Most of the time, the diffusion model generates realistic extensions from the conditioned latent, and when decoded into signal space, the resulting channels remain realistic. However, since the model generates samples from random noise and learns a potential statistical distribution, minor deviations from the ground truth can occur.

7.4 Experimental validation

7.4.1 St. John November 2024

7.4.1.1 Experiment setup In November 2024, we conducted a preliminary trial to record channel soundings using mid-frequency (MF) modems at St. John Island. The recordings allow us to test and fine-tune our pre-trained models using experimental data. In the experiment, we placed the receiver modem at the jetty and the transmitter modem on a boat, which was anchored at various locations in the bay and allowed to drift while continuously transmitting. We transmitted 8,191-bit m-sequences, repeated five times, with a 24 kSa/s sampling rate and a 24 kHz center frequency for channel sounding. At each location, several consecutive transmissions were performed. A total of 28 recordings were made, each with a duration of approximately one minute or slightly longer, throughout the trial.

7.4.1.2 Channel estimation We utilize `AdaptiveEstimators.jl` [11] to estimate the TVIRs from raw recordings using adaptive filters. Figure 7.11 shows two estimated channel responses from different recordings as examples. It clearly demonstrates the channel variations in delay caused by the transmitter boat drifting during the transmission.

We want to fine-tune the pre-trained autoencoder and diffusion models with experimental data to capture the channel characteristics observed in Singapore waters. To ensure a clean test dataset, we randomly

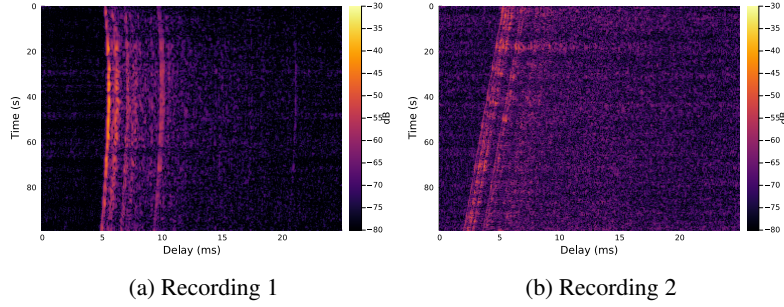


Figure 7.11: Examples of estimated channel responses from two recordings.

select 25 recordings for training and keep 3 recordings for testing before extracting the channel responses into TVIR blocks. For the autoencoder, we sample CIRs at 20 Hz per recording, forming TVIR blocks that capture one-second durations. This results in 1,440 training TVIRs and 148 test TVIRs. For the diffusion model, both conditional and ground truth TVIRs are generated by sampling CIRs at 20 Hz and constructing TVIR blocks over consecutive two-second intervals. The first 20 CIRs form the conditional TVIR, while the next 20 CIRs serve as the ground truth TVIR, yielding 700 training datasets and 72 clean test datasets.

7.4.1.3 Augmented simulation data The measured channels at St. John Island differ significantly from our simulated data due to real-world noise. Testing the pre-trained LSTM autoencoder and diffusion models on this data shows that, while they capture key features, their performance is limited because they are only trained on clean simulated data. To improve robustness, we fine-tune the models using simulated data augmented with real background noise from the sea, prior to conducting experimental fine-tuning. This approach helps the models learn features common to both clean and noisy data, enhancing their adaptability to real-world conditions.

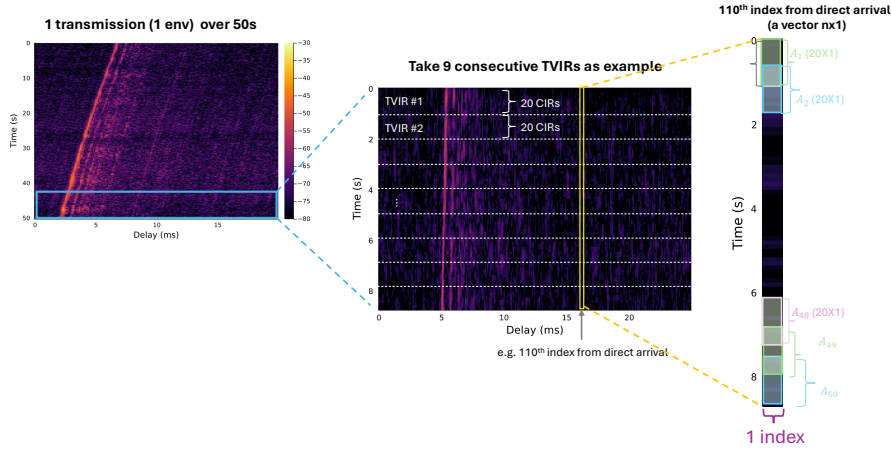


Figure 7.12: Illustration of the covariance matrix calculation for background noise based on experimentally measured channel responses. The covariance matrix is computed from 50 observed noise vectors, A_1 to A_{50} .

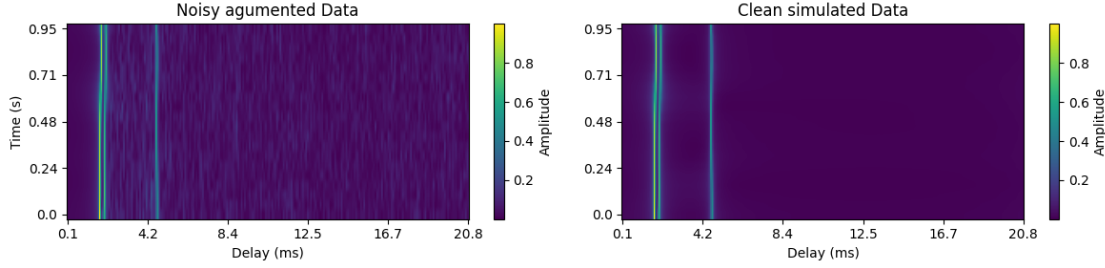


Figure 7.13: An example of clean TVIR alongside its noise-corrupted counterpart, generated using experimentally extracted noise profiles.

To implement this, we first process the raw channel recordings by extracting TVIRs over the entire duration and splitting them into individual TVIR blocks. Our goal is to model the background noise distribution so we can generate random noise accordingly. We choose a multivariate Gaussian distribution with zero mean and extract the complex covariance matrix from the experimental recordings to model the noise characteristics. Since background noise varies across delay indices, we build a noise covariance profile database by computing the covariance matrix at each index across all recordings. To ensure consistency, delay indices are normalized relative to the direct path arrival index. Each recording provides one noise profile, with its covariance matrix extracted across all indices. Figure 7.12 shows an example of a covariance matrix from a recording at the 110th index relative to the direct path, computed using 50 sets of 20-element noise vectors collected over a consecutive period. To build a complete noise profile, we compute the covariance matrix at all indices within each recording and repeat this process for all 28 recordings.

To augment the simulated data with realistic noise, we follow this approach: For each clean simulated TVIR, we randomly select one of the 28 experimental noise profiles. Then, for each delay index from the direct path arrival onward, we sample a 20-element noise vector from the multivariate Gaussian distribution with zero mean and the corresponding covariance matrix from the selected noise profiles. This noise is added to the clean TVIR at the same relative delay indices. To ensure variability, a different noise profile is randomly chosen for each clean TVIR, allowing diverse noise realizations across training samples. This process is applied to all clean TVIRs, generating a dataset with noise characteristics closely matching real-world measurements. Figure 7.13 illustrates an example of a clean TVIR and its corresponding noisy version.

7.4.1.4 Model fine-tuning We first fine-tune our pre-trained models using augmented noisy simulated data, allowing them to learn how to handle background noise. Starting with the pre-trained autoencoder trained on clean simulated data, we use a small initial learning rate of 0.0001 and set a patience of 5 for reducing the learning rate. After this fine-tuning, we further refine the model using a small amount of experimentally measured TVIRs, helping it adapt to real-world data. This experimental fine-tuning takes about one minute on GPU, with an initial learning rate of 0.01 and a patience of 10. This approach significantly improves the autoencoders reconstruction performance on experimental data. Figure 7.14 shows the reconstruction results for unseen test data. The model captures key arrivals in the compact latent while smoothing out random background noise. This indicates that the compact latent representations preserve essential channel information, making them suitable as input for linking the tuning model to identify candidate communication schemes.

We fine-tune the pre-trained diffusion model using the latent representations generated by the LSTM encoder, which has been fine-tuned with noisy simulated data. We then further refine the diffusion model using latent representations of the experimentally collected data, encoded by the experimentally fine-

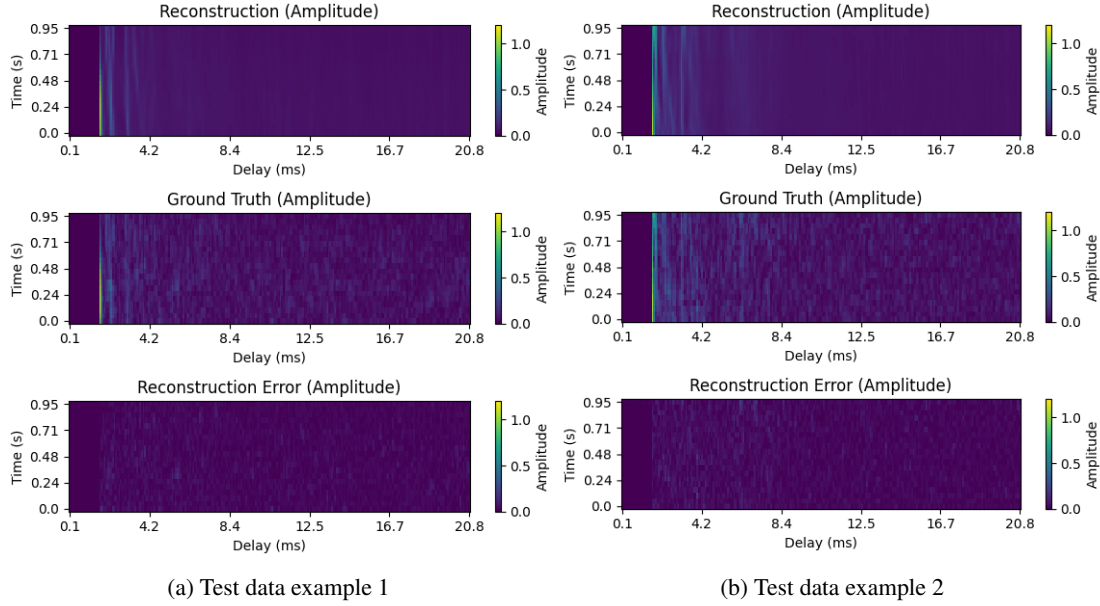


Figure 7.14: Two examples of TVIRs reconstructed by the experimentally fine-tuned autoencoder using clean sea-measured TVIRs from St. John November 2024 trial.

tuned LSTM encoder. Figure 7.15 illustrates the generation performance of the experimentally fine-tuned diffusion model on experimental test data collected during the St. John November 2024 trial. The generated latents closely match the conditioned ones and retain key arrivals when decoded into signal space. This aids in testing candidate communication schemes in simulation using the generated channel responses.

7.4.2 Kusu February 2025

We conducted a second extensive experiment from February 24 to 27, 2025 at Kusu Island, with the transmitter and receiver modems placed on separate boats at sea, loosely anchored. During the experiment, we transmitted a 16,383-bit m-sequence for MF channel sounding and an 2,047-bit m-sequence for very low frequency (VLF) channel sounding. As both the transmitter and receiver were deployed on drifting boats near a busy shipping lane, leading to substantial variations in channel characteristics.

7.4.2.1 MF channel performance Figures 7.16 and 7.17 present two examples showcasing the reconstruction performance of the proposed autoencoder and the generation performance of the diffusion model on data from this experiment. Notably, these models are not fine-tuned on this dataset. Instead, they are pre-trained on clean simulated data, fine-tuned on noisy simulated data, and further refined using experimental data collected in St. John November 2024 trial, as discussed in Section 7.4.1.4. This experiment introduces significantly different conditions compared to the St. John November 2024 trial. The models are tested on data they have never encountered before. Despite these challenges, the models are robust in handling real-world scenarios.

7.4.2.2 VLF channel performance Most of our work on link tuning focuses on the MF band, as the band is suitable for high rate communication with appropriately tuned OFDM and FEC parameters. However, we briefly explore how the digital twin may be extended to lower frequency modems. The VLF

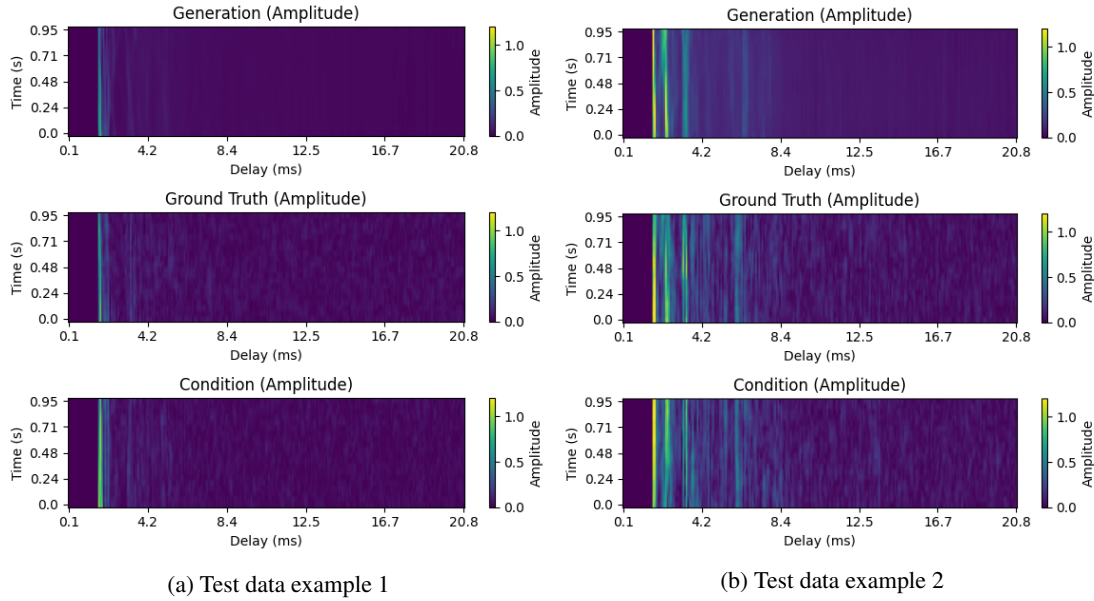


Figure 7.15: Two examples of TVIRs generated by the experimentally fine-tuned diffusion model, conditioned on clean sea-measured TVIRs collected from St. John November 2024 trial.

probe signals collected during the February 2025 trial comprise 194 channel soundings across four channels within the 4.5 kHz to 5.5 kHz frequency band. As lower-frequency acoustic signals propagate farther than higher-frequency ones, we extend the delay spreads to 250 ms, represented by 250 elements, with a sampling rate of 1,000 Hz to capture key channel responses.

As the VLF channel characteristics differ significantly from the MF channels on which the models are originally trained, we fine-tune the models using 140 VLF TVIR samples for training, 30 for validation, and the remaining 24 for testing. The fine-tuning is performed using pre-trained models that have been experimentally adapted based on data collected during the St. John November 2024 trial. Figure 7.18 and Figure 7.19 illustrate the reconstruction performance of the autoencoder and generation performance of diffusion model on the VLF test data. With a limited amount of fine-tuning, the MF model was successfully adapted to capture most of key features of VLF channels.

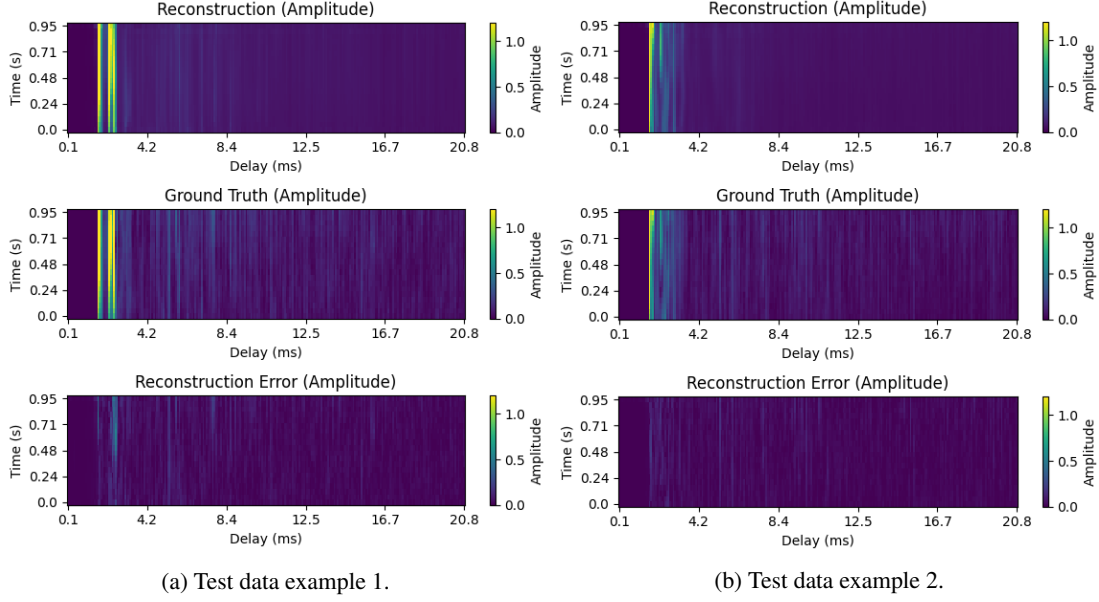


Figure 7.16: Two examples illustrating the reconstruction performance of the proposed autoencoder on data collected from the Kusu February 2025 trial.

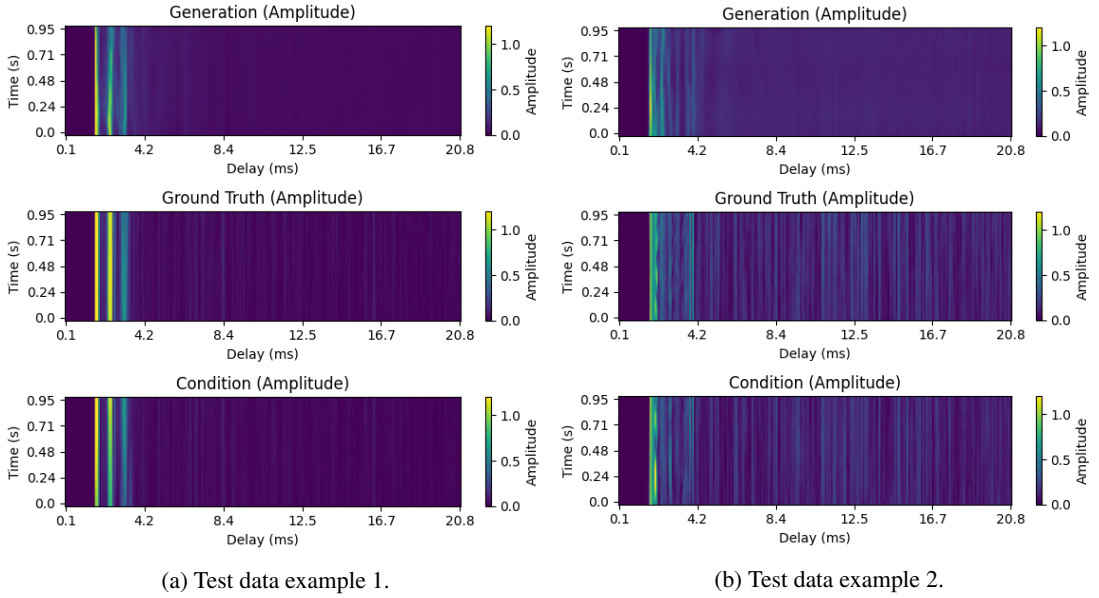


Figure 7.17: Two examples illustrating the reconstruction performance of the proposed diffusion model on data collected from Kusu February 2025 trial.

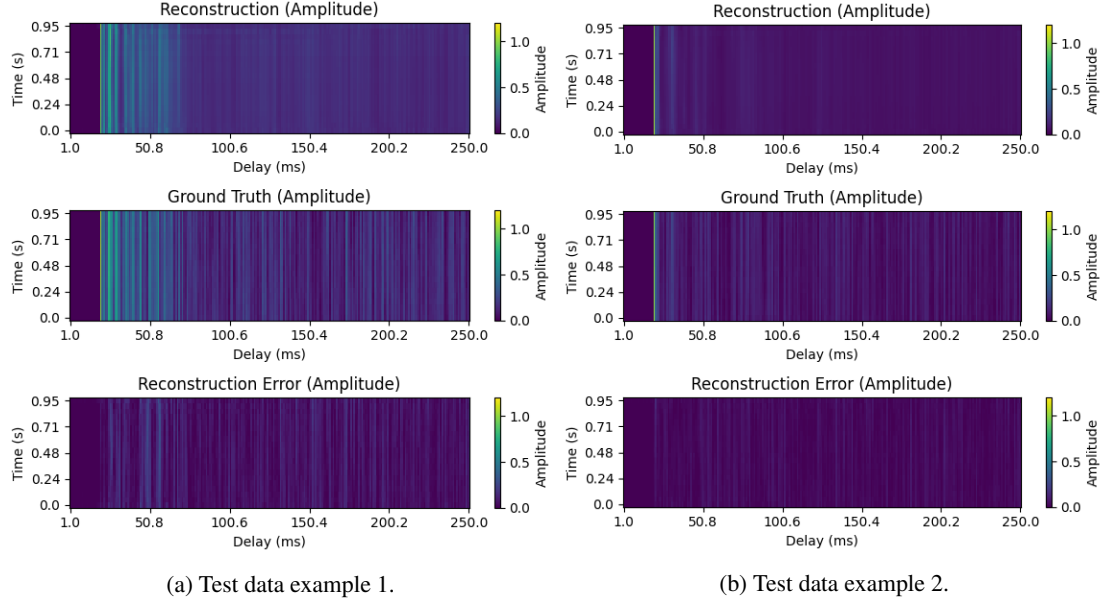


Figure 7.18: Two examples illustrating the reconstruction performance of the proposed autoencoder on VLF data collected from the Kusu February 2025 trial.

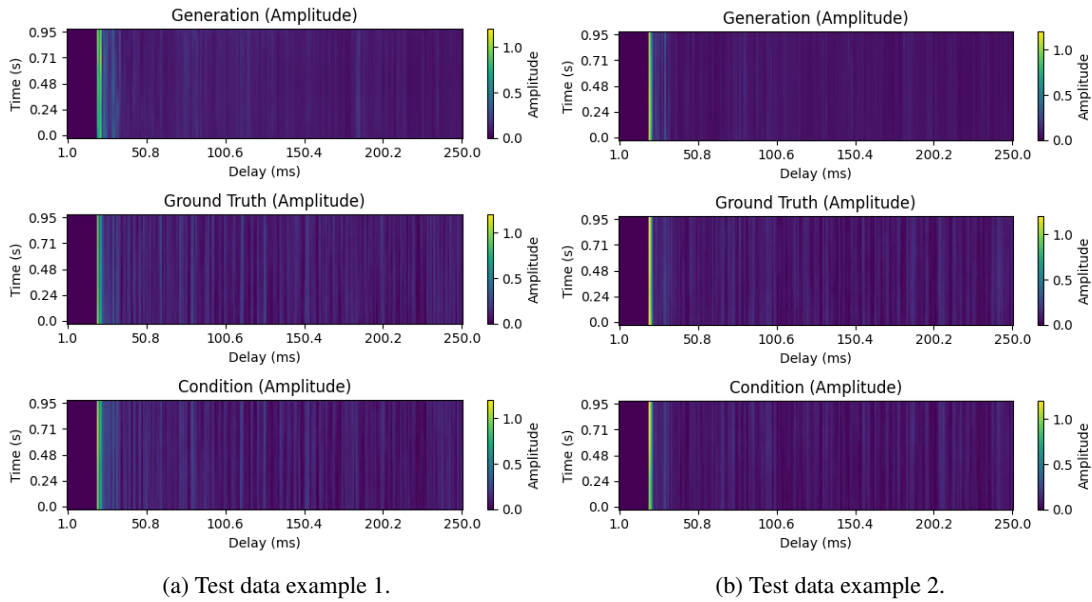


Figure 7.19: Two examples illustrating the generation performance of the proposed diffusion model on VLF data collected from the Kusu February 2025 trial.



Figure 7.20: Digital twin experiment location at Keppel marina. The transmitter is marked as TX, and the receiver array is marked as RX. The receiver array was bottom mounted with hydrophones within 30 cm of each other (in a tetrahedral shape). The transmitter was a surface-deployed modem.

7.5 Communication performance

While the testing of the autoencoder and diffusion model in the last section is convincing, the tests were focused on accurate reconstruction of the TVIR and generation of novel plausible TVIRs. Since we want to use these TVIRs for communication performance prediction, we would like to know whether they are suitable for that task, as they were not trained specifically with communication performance as the target.

In order to test the ability of direct replay based and generative AI (diffusion model) based digital twins to predict the performance of underwater communication schemes, we need a controlled dataset in a slow-varying oceanic channel where we can transmit probes and various communication scheme packets over the coherence time of the channel. We obtained such a dataset at Keppel marina. A transmitter was deployed at a distance of 150 m from a 4-channel receiver array in a water depth of about 5 m. In each cycle, the transmitter sent a probe signal (about 2 seconds in duration) followed by 3 OFDM frames (1 second each) with one set of OFDM parameters. The parameters were changed and the cycle was repeated. A total of 49 cycles with different OFDM parameters were sent, and all the signals recorded.

The probe sent just before the OFDM frames is used to build a direct replay digital twin, which in turn is used to predict the BER of the OFDM frames. This BER is compared against the actual measured BER when the 3 OFDM frames are subsequently sent. The probe is then used as a condition for the diffusion model to generate 3 novel TVIRs. These are used as digital twins for the channel to predict BER distribution for the OFDM frames. Again, the predictions are compared with the actual measurements. Figure 7.21 shows a comparison of the measured BER, direct replay based digital twin predictions, and

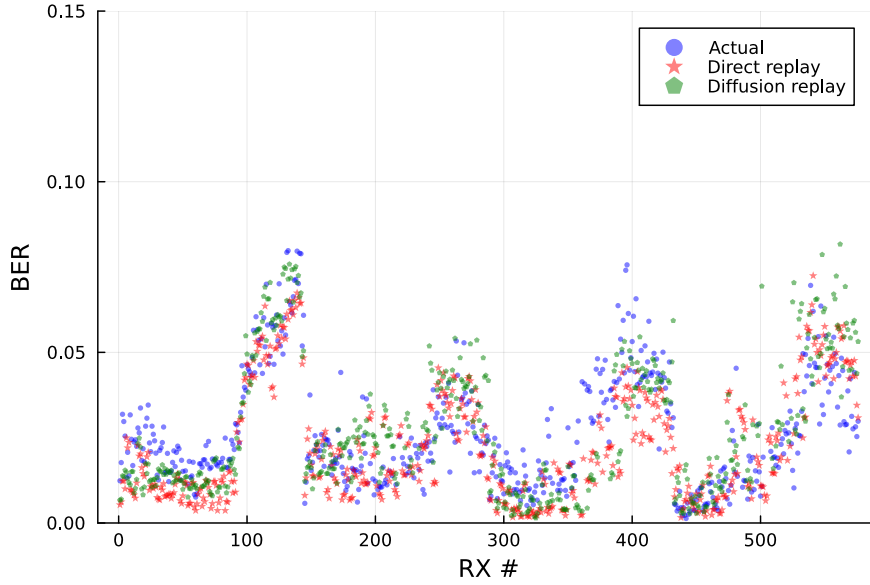


Figure 7.21: Comparison of BER predictions from in-water transmissions (actual), direct replay based digital twin, and generative AI (diffusion model) based digital twin. Each reception (RX #) is an OFDM frame with parameters changed every 3 frames. 49 parameters sets from 4 hydrophones are shown here.

diffusion model based digital twin predictions. We see a close match between the 3, giving us confidence that the digital twins can indeed model the real world well!

The advantage of the diffusion model over the direct replay is that the diffusion model can be used to model the distribution by drawing many samples from it. On the other hand, the direct replay can only be used to make a single prediction.

We show median BER prediction errors using direct replay and diffusion replay in Figure 7.22. For most OFDM parameters, we see that the predictions are within $\pm 1.5\%$ of the actual BER. A few predictions (2048/0, 4096/0, 4096/0/w) are out of the plot axis limits, where the predictions are poor. We suspect that for these cases, the actual BER is high due to specific transient events during those transmissions.

7.6 Discussion

We developed a digital twin comprising an LSTM-based autoencoder and a latent conditional diffusion model to simulate realistic time-varying underwater communication channels. By leveraging the autoencoder to compress high-dimensional TVIRs into a compact latent space, we significantly reduce computational complexity, enabling the efficient training of both the conditional diffusion model and the link tuning model. The digital twin generates TVIRs that closely resemble measured channels, providing a strong foundation for the link tuning model to select optimal modulation schemes in simulation, thereby eliminating the need for actual transmission. The models are validated through both simulations and two experimental datasets, demonstrating their adaptability and ability to generalize to highly dynamic channel conditions. The models were also shown to be able to predict communication BER for various OFDM modulation parameters within about 1.5%. As we will see in Section 7.5, and in Chapter 8, the autoen-

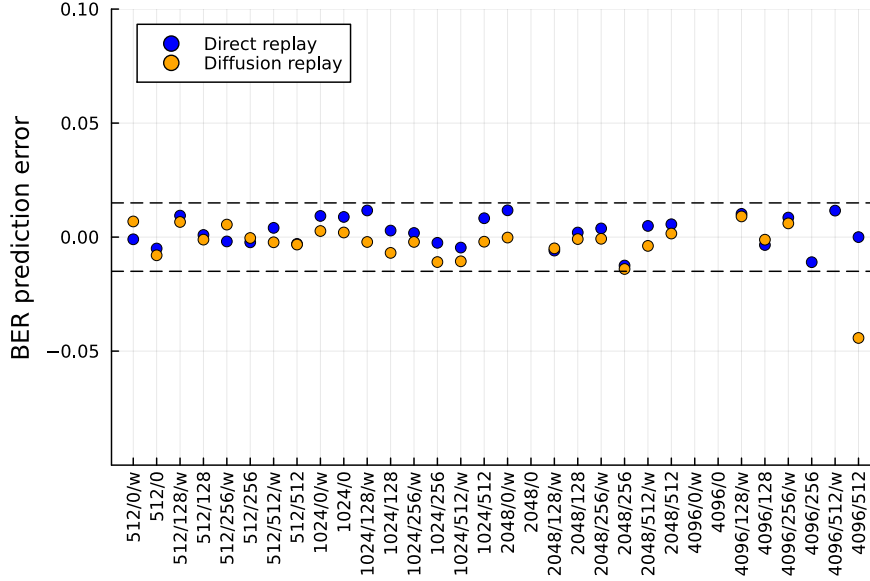


Figure 7.22: BER prediction errors using direct replay based digital twin and generative AI (diffusion model) based digital twin. The x -axis is labeled with the OFDM parameters in the format: number of carriers / prefix length / windowed. The horizontal dashed lines are at $\pm 1.5\%$ BER error, showing that most predictions are within this margin of error. A few predictions (2048/0, 4096/0, 4096/0/w) are out of the plot axis limits, where the predictions are poor.

coder and diffusion models have been successfully used at sea for predicting communication performance and tuning link parameters.

While the model performs well in Singapore, it has been trained on environmental characteristics specific to local waters. To improve generalization across different underwater environments and frequency bands, several enhancements can be explored:

1. The autoencoder is trained on TVIRs with a fixed sequence length of 20, optimized for the available training data. Although LSTM networks can handle sequences of varying lengths, our model has not yet leveraged this capability.
2. The time delays for TVIRs in the training data are sampled at 12,000 Hz, resulting in each CIR containing 250 elements to capture a 20 ms delay spread for MF data. While this is effective for Singapore waters, the models generalizability could be improved by training on datasets with variable sequence lengths, longer delay spreads, and more diverse experimental data with different background noise conditions.
3. Communication performance models can be incorporated into the training process for both the autoencoder and diffusion model by evaluating losses based on communication performance rather than solely on channel responses. This approach could help preserve critical features in channel responses that are essential for communication quality.

CHAPTER 8

LINK ADAPTATION

8.1 Introduction

Limited bandwidth, frequency-selective fading, long propagation delays and time variability makes underwater acoustic communication (UAC) very challenging [12, 13]. To overcome these challenges, we developed software-defined modems, which are now commercially available through Subnero. These modems allow the user to change modulation types, modulation parameters, and error correction codes dynamically, enabling performant communication in a dynamic environment. When the channel physics permits, parameters can be set to achieve high data rates. When the channel conditions are unfavorable, parameters can be adjusted to achieve robust lower rate communication. Currently this process has to be undertaken by a trained human operator. Although some basic parameter tuning capability has recently been introduced in the Subnero modems, this process is based on heuristics and requires a large number of transmissions in the water. Since transmissions take time and energy, only a small portion of the parameter space can be explored using in-water *link tuning*, and the procedure does not always yield a performant link.

The idea of adapting link parameters to get good performance is not new. In wireless communication, this is termed as adaptive modulation and coding (AMC). However, traditional wireless communication devices only support a few modulation and coding schemes (MCSs). This, in concert with short frame duration and low propagation delay, makes a brute-force search for the best MCS feasible. In UAC, this simply isn't possible, and we require more sophisticated methods to select a good MCS from among millions of possibilities. While researcher (including ARL) have explored ideas to address this need [14–18], effective methods to fully address the challenges remain elusive. In this chapter, we explore a few ideas to leverage the digital twin from Chapter 7 to reduce the need to transmit through the water during

the search. We also explore modern machine learning algorithms to aid with narrowing down the search for good MCS, thereby reducing the time and energy taken in the search.

The digital twins developed in Chapter 7 provide us a way to estimate MCS performance without having the transmit signals through water. This is a powerful ability that we can leverage to rapidly search for candidate MCS without incurring the time and energy for underwater transmission, and without interfering with other nodes who may want to use the channel while we conduct the search. In section 8.2, we demonstrate the use of direct replay and diffusion replay based digital twins for MCS selection.

Link tuning based on digital twins still requires an extensive search to find an appropriate MCS. This cost of the search is reduced by doing the search digitally instead of using in-water transmissions, but the search still takes time and energy, and limits the parameter space that can be explored. Would it be possible to learn a function that maps a channel measurement directly to a good set of MCS? Given sufficient training data, machine learning could be used to learn such a function. Unfortunately, getting such a dataset is impractical, and so purely data driven machine learning cannot solve this problem. Channel physics can inform MCS selection, particularly in OFDM systems where channel parameters such as delay spread, coherence bandwidth, and Doppler spread influence key MCS parameters like subcarrier count (n_c), prefix length (n_p), and bandwidth allocation [19]. Accurate measurement of these channel parameters is difficult in a time-varying channel. Even if one could measure channel parameters reliably, the relationship to MCS parameters is only partially understood. Could one use machine learning to discover these relationships and utilize them to narrow down the MCS with just a few transmissions through water? We leverage the autoencoder developed in Chapter 7 to distill information from channel measurements, and show that it is possible to learn a mapping between the distilled information and good MCS in section 8.3.

8.2 Digital twin based link tuning

In Chapter 7, we developed two digital twins. The direct replay digital twin uses a TVIR estimated by transmitting a probe signal through water as the basis for the twin. To test a communication signal through the twin, the TVIR is applied to the acoustic signal and noise is added. The resulting distorted signal is then demodulated and decoded to estimate communication performance. The diffusion replay digital twin uses a generative AI based TVIR conditioned on the measured TVIR, instead of using the measured TVIR directly to distort the signal. The advantage of this method is that we can estimate the distribution of BERs as a measure of communication performance, rather than a single BER as in the case of direct replay. In this section, we use both digital twins to help narrow down MCS for testing in water, thereby making the link tuning process more efficient. The key steps in link tuning between nodes A and B using the digital twin are outlined below:

1. Node A sends a request to node B for a probe signal. The request is sent over a robust CONTROL scheme that provides a low data rate and is not tuned.
2. Node B sends a special probe signal (typically a pseudo-noise sequence with good autocorrelation properties for channel estimation) to node A.
3. Node A uses the received probe signal to estimate the TVIR of the channel. The TVIR is used to build a digital twin, either using direct replay or diffusion replay.
4. Node A does a grid search over various OFDM parameters (number of carriers, prefix length, band of operation, etc) by generating signals with those parameters, passing them through the digital twin, demodulating the signals, and estimating communication performance in terms of BER. The BER informs error correction code selection using Table 2.1, which in turn yields estimates of error-free data rate through the digital twin.

5. The top performing N OFDM parameter sets are shortlisted for testing in water. For each of these parameters, node A sends a request to node B for M test frames with BPSK and QPSK modulation on each carrier.
6. Node B changes the modulation parameters to the requested values and sends M test frames with known bits. These frames do not use any error correction coding.
7. On receiving the $2N \times M$ test frames, node A measures the BER for each of the $2N$ OFDM parameter sets. The BER informs error correction code selection using Table 2.1, which in turn yields estimates of error-free data rate through water.
8. The top performing OFDM parameter set, together with the corresponding error correction code recommendation, is selected as the MCS.
9. Node A requests node B to set this MCS and transmit M test frames to verify that the MCS achieves error-free communication. Node B transmits the frames and node A demodulates and decodes them to verify that tuning was successful.

We tested this algorithm at sea using Subnero MF SISO modems with $N = 3$ and $M = 3$ at a range of 2.7 km. The results are summarized below.

8.2.1 Direct replay

The first test was conducted near Kusu island on 25th February 2025 at a range of 2.7 km with water depth varying between 20–40 m using the direct replay digital twin. The transmitter was deployed at a depth of about 5 m and the receiver was deployed at about 10 m. The test commenced at 14:41 with a probe request, and a recommendation of MCS was made in about 3 minutes after a grid search and subsequent channel measurements. The 6 modulation schemes (top 3 with BPSK and QPSK) shortlisted are summarized in Table 8.1.

Table 8.1: Shortlisted MCS during direct replay link tuning.

nc	np	bw	dc0	psk	frame Length	Digital Twin BER	In-water BER	FEC code	Coded data rate (bps)
2048	0	0.3	-204	2	344	0.062	0.049	ldpc-1/2	3326
2048	0	0.3	-204	4	459	-	0.105	ldpc-1/3	3837
1024	0	0.3	-102	2	344	0.067	0.058	ldpc-1/2	3316
1024	0	0.3	-102	4	459	-	0.084	ldpc-1/3	4211
2048	32	0.3	-204	2	344	0.053	0.045	ldpc-1/2	3267
2048	32	0.3	-204	4	459	-	0.059	ldpc-1/2	5642

Parameter nc is the number of OFDM carriers, np is the prefix length, bw is the fractional bandwidth (multiply by 24,000 for bandwidth in Hz), dc0 is the frequency offset in number of carriers, psk is the modulation order (2 for BPSK, 4 for QPSK), and frameLength is the number of bytes per OFDM frame. After testing the 6 shortlisted MCS, the link tuner made a recommendation for the MCS with a data rate of 5.642 kbps.

8.2.2 Diffusion replay

The second test was conducted at the same location shortly afterwards, at a range of 2.8 km, using the diffusion replay digital twin. The test commenced at 14:44 with a probe request, and a recommendation of MCS was made in about 9 minutes after a grid search and subsequent channel measurements. The longer search time is due to the generation of new TVIRs by the diffusion model. The 6 modulation schemes (top 3 with BPSK and QPSK) shortlisted are summarized in Table 8.2.

Table 8.2: Shortlisted MCS during diffusion replay link tuning.

nc	np	bw	dc0	psk	frame Length	Digital Twin BER	In-water BER	FEC code	Coded data rate (bps)
1024	32	0.5	0	2	286	0.108	0.122	ldpc-1/2	2677
1024	32	0.5	0	4	0	-	0.195	NA	0
2048	32	0.4	-102	2	306	0.115	0.090	ldpc-1/3	2912
2048	32	0.4	-102	4	460	-	0.131	ldpc-1/4	3772
1024	32	0.4	-51	2	229	0.108	0.124	ldpc-1/4	2142
1024	32	0.4	-51	4	0	-	0.178	NA	0

After testing the 6 shortlisted MCS, the link tuner made a recommendation for the MCS with a data rate of 3.772 kbps.

8.3 Physics-informed AMC

While the digital twin based link tuning works well, it involves a heuristic search that takes time and limits the parameter space that can be explored. In this section, we explore ways to learn a mapping between channel measurements and a set of good MCS, and use that mapping to narrow down the search space. Learning such a map just from data is impractical. We leverage known channel physics to guide our learning.

8.3.1 Tunable parameters

In this project, the transmitter (TX) and receiver (RX) nodes utilize UnetStack-based WNC-M25MSS3 modems [20] from Subnero. UnetStack is an agent-based network stack, where each agent operates similarly to a layer in a traditional communication network stack but offers flexibility to optimize resources such as bandwidth and energy. The stack includes a phy agent that provides physical layer services and supports two logical channels:

- CONTROL channel (phy[CONTROL]) – Designed for low-rate, robust data transmission.
- DATA channel (phy[DATA]) – Configured for higher-rate data transmission.

For high data-rate link tuning, OFDM is used as the modulation technique on phy[DATA] channel, enabling dynamic adjustment of multiple transmission parameters through indexed properties of phy[DATA].

In an OFDM system, several parameters can be tuned to optimize performance. One key parameter is the number of carriers used to modulate the baseband data, specified via n_c . The Subnero M25M modem operates with a typical bandwidth of 12 kHz, spanning from 20 kHz to 32 kHz, half of the total available

bandwidth ($W = 24$ kHz). The parameter bw controls the reduction in this bandwidth. For example, setting $bw = 0.5$ reduces the bandwidth by half, resulting in a frequency range from 20 kHz to 32 kHz.

The center frequency can be adjusted to avoid interference or optimize transmission for specific channel conditions. Changing the center frequency affects how the carriers are distributed across the spectrum. The parameter $dc0$ controls the starting carrier number. If $dc0 = 0$, the center frequency f_c is 24 kHz, with no shift. When $dc0 = 85$, along with $bw = 0.5$ and $n_c = 1024$, the center frequency is shifted by 2 kHz where $\delta f_c = 2$ kHz, from 24 kHz to 26 kHz. The relationship between δf_c and $dc0$ is given by:

$$dc0 = \frac{\delta f_c}{W} n_c. \quad (8.1)$$

This shift must respect the constraints of the available bandwidth:

$$20000 + bw \times \frac{W}{2} \leq 24000 + \delta f_c \leq 32000 - bw \times \frac{W}{2}. \quad (8.2)$$

The cyclic prefix length, adjustable via n_p , mitigates ISI from multipath propagation. A longer cyclic prefix helps reduce interference but reduces the overall data rate, as more transmission time is allocated to redundancy. Thus, balancing the cyclic prefix length is essential to optimizing both error resilience and throughput.

The modulation order per subcarrier, defined by psk , determines the number of bits transmitted per symbol. Higher-order modulations increase data throughput but require a higher Signal-to-Noise Ratio (SNR) for reliable transmission. Additionally, the coding rate ρ , adjustable via fec , is crucial for error correction. This fec , i.e., the Forward error correction (FEC) technique, enables the receiver to recover lost or corrupted data, enhancing communication reliability. However, increasing the coding rate reduces the effective data rate, so finding the optimal coding rate is key to balancing error protection with throughput.

The power level, controlled by $plvl$, is another tunable parameter. Excessive power can cause distortion and interference with neighboring systems, while insufficient power can lead to poor reception. Therefore, it is crucial to properly adjust the power to ensure efficient transmission. For transmissions in Singapore sea waters, the power level is typically tuned between -10 dB and 0 dB beforehand to establish a functional communication link between the modems.

By adjusting parameters such as the number of subcarriers, cyclic prefix length, modulation order, center frequency offset, bandwidth ratio, and coding rate, the OFDM system can be fine-tuned to balance data rate, reliability, and robustness in underwater acoustic communication.

8.3.2 Characterizing the channel

The modem has several tunable MCS parameters, represented as $a = \{n_c, n_p, dc0, bw, psk, fec\}$. As detailed in [19], certain channel physics parameters in the OFDM system can directly inform MCS selection. For example, the cyclic prefix duration of an OFDM symbol must exceed the channel delay spread to mitigate ISI, thus ensuring the condition $\frac{n_p}{B} > \tau_{ds}$, where τ_{ds} denotes the delay spread. Given that the CIR and TVIR carry these critical channel physics information essential for MCS selection, we aim to propose a link tuning method based on TVIR/CIR.

A TVIR from Singapore waters is plotted in Figure 8.1, alongside a CIR plot sampled from the TVIR in time and frequency domains shown in Figure 8.2. Usually, the delay spread, coherence bandwidth, Doppler spread, multipath effects, and path loss attenuation can be extracted from CIR. TVIR extends the CIR by capturing the temporal dynamics of the channel, including coherence time and fading characteristics, further refining MCS selection in dynamic environments. In Figure 8.1 and Figure 8.2a, the x-axis represents delays of arrivals, showing how the impulse response evolves over time. In Figure 8.1, the y-axis represents how 20 CIRs varying in one TVIR within 1 second and the color intensity indicates signal strength, with brighter colors representing stronger reflections. Focusing on one CIR, as shown

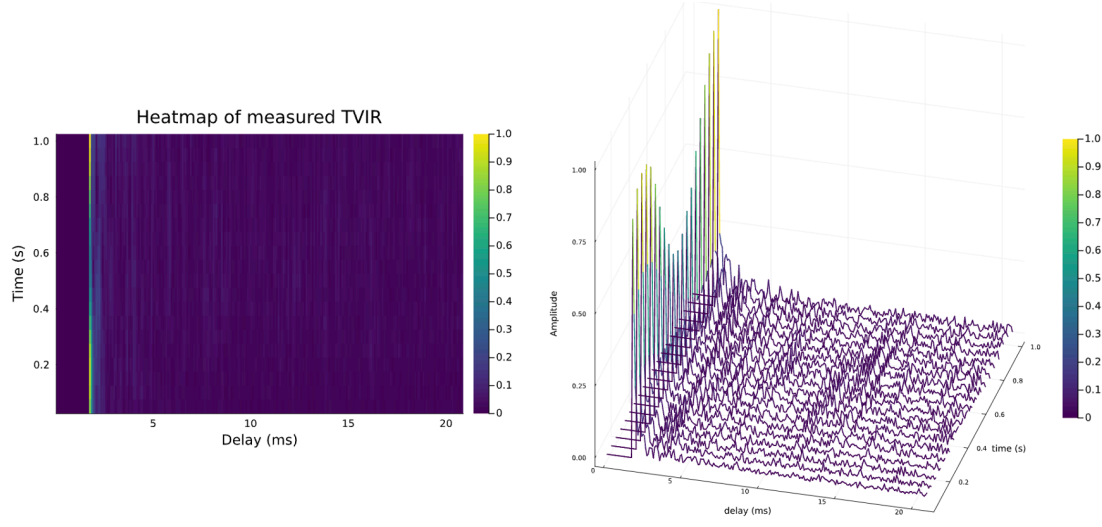


Figure 8.1: Sample TVIR.

in Figure 8.2a, the y-axis represents different multipath components (signal arrival paths), which align with the color intensity distribution observed in the TVIR. In Figure 8.2b, the PSD of the CIR gives a frequency-domain representation of the channel's characteristics. It shows how power is distributed across different frequencies in the received signal. The following section will explore how these channel physics parameters, derived from both CIR and TVIR, guide MCS selection in more detail.

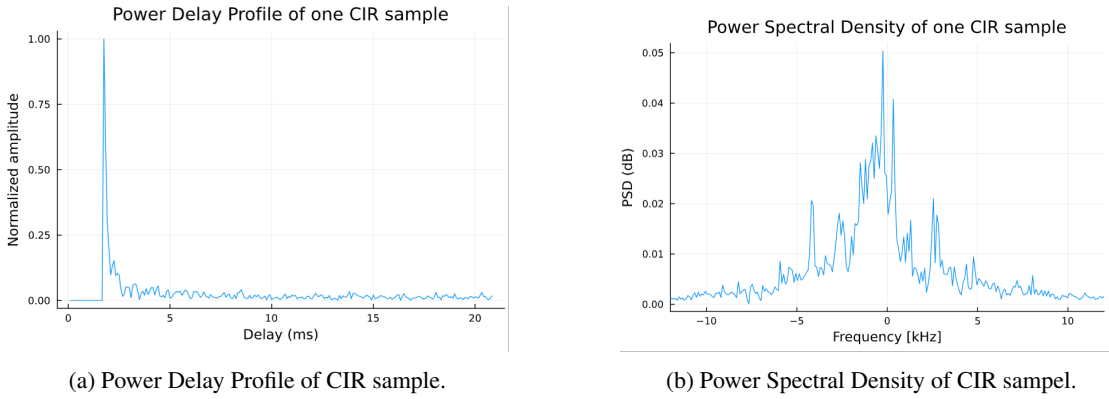


Figure 8.2: Sample CIR.

8.3.2.1 Doppler spread & channel coherence time. The TVIR contains information about the time-varying behavior of the channel impulse response. Channel variations cause Doppler spread, where narrowband signals broaden over time. Doppler spread and channel coherence time (T_c) are inversely related: a higher Doppler spread results in faster channel changes and a shorter coherence time, while a lower Doppler spread leads to a more stable channel and a longer coherence time.

By analyzing the variations of the TVIR, we can observe how the channel changes over time, which directly relates to the Doppler shift and spread. Additionally, the CIR Power Delay Profile (PDP) gives

insight into the time dispersion of the multipath components, which is linked to the coherence time of the channel, while the Power Spectral Density (PSD) helps quantify the frequency spread due to relative motion, aiding in the estimation of Doppler spread. Rapid fluctuations in the impulse response suggest a short coherence time and high Doppler spread, while slow variations indicate a long coherence time and low Doppler spread. By examining the variation in these patterns, we can roughly estimate Doppler spread and coherence time.

In an OFDM system, the symbol duration T_s is determined by the bandwidth $B = bw \times W$ and the sum of subcarriers (n_c) and pilot symbols (n_p):

$$T_s = \frac{n_c + n_p}{B}.$$

To ensure reliable transmission, the symbol duration should be shorter than the coherence time to minimize inter-carrier interference. Balancing n_c and n_p can enhance Doppler robustness. For a short T_c , the channel varies quickly, so using a smaller T_s (larger n_c) helps maintain symbols within the coherence time. More guard symbols (n_p) may also be required to reduce time variations and ISI. Therefore, the insights provided by the TVIR and CIR are essential for selecting appropriate values for n_c and n_p .

8.3.2.2 Channel coherence bandwidth. The TVIR and CIR provide valuable information about the channel's frequency-selective behavior. Frequency selectivity occurs when signal components at different frequencies experience varying degrees of fading, a phenomenon characterized by the channel's coherence bandwidth. The coherence bandwidth B_c represents the frequency range over which the channel response remains highly correlated. A wide coherence bandwidth indicates that the channel is relatively flat, meaning all frequency components undergo similar fading. In contrast, a narrow coherence bandwidth suggests frequency-selective fading, where different subcarriers experience independent fading.

From Figure 8.1, we can have a rough idea of the coherence bandwidth by analyzing how the delay spread of multipath components evolves in the TVIR. A long delay spread corresponds to a narrow coherence bandwidth, whereas a short delay spread indicates a wider coherence bandwidth. Since coherence bandwidth is inversely related to delay spread, a channel with severe multipath effects (longer delay spread) exhibits more frequency selectivity, impacting subcarrier spacing in an OFDM system.

Understanding the channel coherence bandwidth from the impulse responses can help guide the choice of subcarrier spacing in an OFDM system. The number of subcarriers n_c and the bandwidth B determine the subcarrier spacing Δf , which is given by:

$$\Delta f = \frac{B}{n_c}. \quad (8.3)$$

For reliable transmission, the subcarrier spacing should be smaller than the coherence bandwidth to ensure that each subcarrier experiences approximately flat fading. If B_c is small (high frequency selectivity), increasing n_c (reducing Δf) allows each subcarrier to remain within the flat-fading region. On the other hand, if B_c is large, fewer subcarriers (larger Δf) may be sufficient to avoid excessive redundancy.

8.3.2.3 Delay spread. The CIR and TVIR provide information about the delay spread by showing how multipath components of the signal arrive at different times, with longer delays indicating a larger spread. The cyclic prefix length n_p in an OFDM system is closely related to the channel's delay spread τ_{ds} . To mitigate ISI and maintain orthogonality between subcarriers, the cyclic prefix duration T_p should be at least as long as the delay spread:

$$T_p = \frac{n_p}{B} \geq \tau_{ds}. \quad (8.4)$$

A larger τ_{ds} (i.e., a highly dispersive channel) requires a longer cyclic prefix (n_p) to prevent ISI, ensuring that delayed multipath components do not interfere with subsequent symbols. However, increasing

n_p reduces spectral efficiency since a larger portion of each OFDM symbol is occupied by redundant guard time. Therefore, optimizing n_p involves balancing robustness against ISI and efficient bandwidth utilization.

8.3.2.4 Optimal frequency band. The time-varying impulse response TVIR also provides insight into the frequency-selective behavior of the channel, which is crucial for selecting the optimal transmission frequency band. Different frequency bands experience varying levels of attenuation and multipath fading due to environmental factors such as absorption, scattering, and interference.

By analyzing the CIR PSD, as shown in Figure 8.2b, we can identify the most suitable operating frequency range and apply frequency offset correction for transmission. The PSD reveals the distribution of signal energy across frequencies, helping to determine regions with minimal attenuation and stable channel conditions. Identifying frequency ranges where the power is relatively high, stable, and exhibits minimal fluctuation can provide a rough estimate of the optimal frequency band. However, due to channel variations and interference, additional factors such as coherence bandwidth, Doppler effects, and environmental conditions should also be considered to refine the selection of the most reliable transmission band.

In summary, key channel physics parameters such as Doppler spread, coherence time, delay spread, and coherence bandwidth can be extracted from the TVIR and CIR. These parameters provide essential insights into the time- and frequency-selective behavior of the channel, which directly impacts system design choices, including subcarrier spacing, symbol duration, and frequency band selection. Analyzing CIR PSD and PDP helps in estimating the coherence bandwidth and delay spread, which influence how signals experience fading and dispersion. While these channel characteristics play a fundamental role in selecting an appropriate MCS, their relationship with MCS parameters is not always straightforward. Additional correlations such as the interaction between Doppler effects, multipath conditions, and coding rate may be crucial for optimizing link performance. Given the complexity of these dependencies, a more efficient approach is needed to integrate channel physics into MCS selection. Instead of relying on direct parameter mapping, advanced models or learning-based frameworks can better capture the underlying patterns and make adaptive link tuning more effective.

8.3.3 Mapping from channel to MCS

In UAC systems, the TVIR provides crucial insights into channel characteristics, which, as previously discussed, are essential for selecting the optimal MCS as we mentioned before. However, real-time extraction of channel physics from CIR or TVIR for MCS selection is challenging due to high computational costs, and the variability of underwater environments. Meanwhile, the relationship between channel physics and MCS selection is highly complex, involving nonlinear dependencies that make manual mapping difficult.

To address these challenges, we propose an ML-driven approach that directly utilizes TVIR data for MCS prediction, eliminating the need for manual parameter extraction. To efficiently capture the rich temporal and spatial information in TVIR, we employ an LSTM autoencoder to encode it into a 128-length latent vector, preserving key channel characteristics while reducing dimensionality. By applying ML to this latent representation, we can learn complex relationships between channel conditions and optimal MCS choices without requiring explicit parameter estimation. This ML-based approach enables rapid adaptation to dynamic environments, enhancing communication efficiency and reliability. Furthermore, ML models can generalize across diverse scenarios, improving robustness against uncertainties in underwater acoustic channels.

The proposed ML framework leverages the channel physics embedded in the TVIR to optimize link tuning adaptively. To manage the complexity of interdependent channel parameters, we decompose the problem into marginal models, each targeting a specific, physically interpretable parameter. This decomposition allows for separate optimization while preserving interactions among parameters, making the

overall process more structured and efficient. The TVIR inherently encodes critical characteristics such as delay spread, coherence bandwidth, and spectral occupancy, which directly guide MCS selection.

For instance, the coherence bandwidth B_c , which can be inferred from the TVIR latent representation $\hat{h}$, informs the number of subcarriers n_c , expressed as

$$n_c = f_1(g_1(\hat{h})), \quad \text{where} \quad B_c = g_1(\hat{h}).$$

To model this mapping, we introduce a marginal model M_1 using a Multi-Layer Perceptron (MLP). Similarly, the delay spread τ_{ds} dictates the cyclic prefix length n_p , requiring another marginal model M_2 built by MLP. Spectral features extracted from the TVIR drive the selection of center frequency f_c and bandwidth ratio bw , which are modeled using a new MLP M_3 . Additionally, modulation order (e.g., BPSK or QPSK) is estimated by MLP M_4 . By structuring the problem into these targeted models within a unified framework, we can effectively capture the nonlinear relationships between channel conditions and optimal MCS choices while reducing model complexity.

The overview of our MCS sector mode is illustrated in Figure 8.3. We represent n_c and psk as one-hot vectors since they take on discrete values. In contrast, n_p spans a continuous range, so we predict its mean and standard deviation under the assumption that its optimal values follow a Gaussian distribution. For modem tuning, we predict bandwidth ratio and center frequency to determine bw and dc_0 . The parameter f_l represents the lower bound of the optimal frequency range, while $B = bw \times W$ specifies how much bandwidth is utilized for frame transmission. The structure of each marginal model is shown in Figure 8.4.

This structure balances specificity and coordination: each output head specializes in predicting a parameter grounded in channel physics, while shared feature learning ensures consistency across interdependent decisions. By aligning the architecture with domain knowledge (e.g., delay spread $\rightarrow n_p$, spectral occupancy $\rightarrow f_c$), the design enhances both interpretability and adaptability to dynamic channel conditions, ultimately improving the efficiency of MCS selection in underwater acoustic communication systems. The detailed structure of our marginal models M_1 to M_4 is shown from Figure 8.4.

The advantages of designing the model in this manner are multifaceted:

- **Physically interpretable optimization:** Each parameter is optimized based on physically interpretable channel characteristics, such as delay spread and coherence bandwidth, leading to more accurate and reliable MCS selection.
- **Enhanced interpretability and reduced training effort:** By aligning the model architecture with domain knowledge, the framework improves interpretability. We also experimented with a single model that directly maps TVIR to all MCS parameters simultaneously. However, when using the same number of parameters as the marginal models, this approach required significantly more training data to achieve comparable performance, highlighting the benefits of decomposition.
- **Adaptability to dynamic conditions:** The framework's ability to dynamically adjust to varying channel conditions ensures robust performance in real-world communication scenarios. This adaptability is particularly important in wireless systems where channel conditions can change rapidly.

8.4 Simulation

To evaluate the performance of the proposed MCS selector model in the realistic UAC system, we develop a simulation framework in a virtual UAC environment. This framework utilizes the `UnderwaterAcoustic.jl` and `VirtualAcousticOcean.jl` packages to generate synthetic channel environments and simulate end-to-end communication scenarios. Designed to replicate real-world conditions, the simulations also provide a robust training dataset for the MCS selector model. The key steps in the simulation pipeline are outlined below.

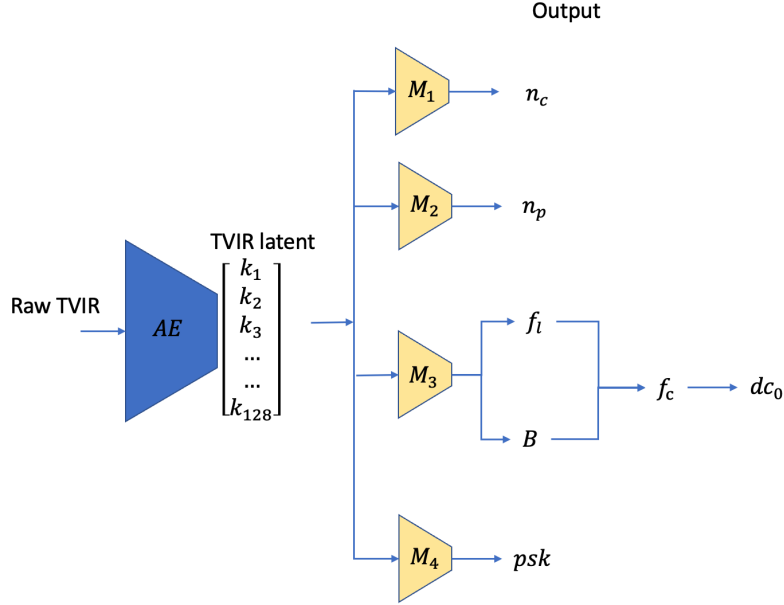


Figure 8.3: Model structure.

8.4.1 MCS selector training

The training dataset is constructed from raw TVIRs collected during sea trials conducted in Singapore waters in December, 2024. We employ the LSTM autoencoder to encode each TVIR into a 128-dimensional latent space for a compact representation. In total, we process 1500 raw TVIRs, denoted as h_i , and obtain their corresponding latent representations $\hat{h}_i$, where $i = 1, \dots, 1500$.

8.4.1.1 Training data collection. To determine the optimal MCSs for a given TVIR latent $\hat{h}_i$ in the training dataset, we employ the channel replay module. This module reconstructs a baseband replay channel using the raw TVIRs h_i . For each possible MCS configuration, a signal is generated by a modem, which can be either a virtual model or a real device. After upconversion, the signal is transmitted through the baseband replay channel, where the channel effects modeled by the TVIR are applied, along with optional noise. The received signal is then downconverted and decoded using the selected decoding techniques, allowing us to approximate the BER and assess transmission performance. Based on this, the coded data rate r of each MCS configuration is computed as:

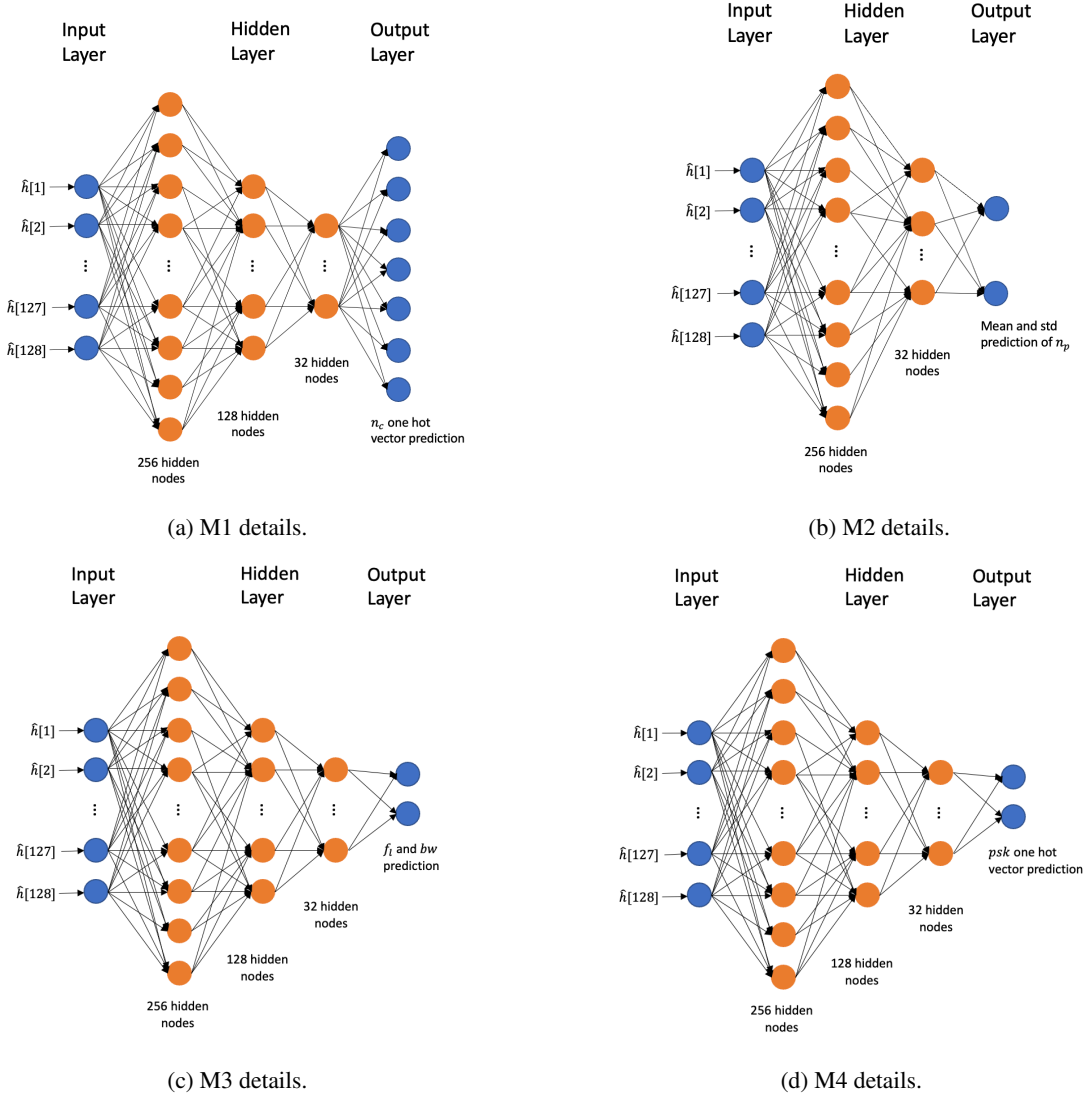
$$r = d\rho, \quad (8.5)$$

where d represents the uncoded data rate of the MCS, and ρ is the selected FEC rate based on the BER, as shown in Table 2.1.

The knowledge of the coded data rate for each available MCS under specific channel conditions, i.e., TVIR, enables the identification of the best-performing MCSs for each TVIR instance. This process ensures that the training dataset includes the TVIR latents $\hat{h}_i$ and their corresponding optimal MCS configurations. Thus, the training set is defined as

$$D = \{(\hat{h}_i, \mathcal{A}_i^*)\},$$

where $\mathcal{A}_i^*$ represents the set of optimal MCS configurations for the given TVIR latent $\hat{h}_i$.

Figure 8.4: Structure of marginal models M_1 to M_4 .

8.4.1.2 MCS selector training. The MCS selector model is trained using a dataset generated from the simulation studies. The model takes a 128-dimensional latent representation of the TVIR as input and outputs a probability distribution of each MCS parameter with respect to the coded data rate.

To effectively train the model, we define different loss functions tailored to each MCS parameter type: The modulation order n_c and modulation order psk are treated as categorical variables and predicted as one-hot vectors. We use cross entropy loss to measure the difference between the predicted probabilities and the actual one-hot encoded labels.

$$L_{\text{BCE}} = -\sum y \log \hat{y} + (1 - y) \log(1 - \hat{y}).$$

This ensures that the model assigns high probabilities to correct categories while minimizing incorrect classifications. We model n_p as a Gaussian-distributed variable, where the model predicts both the mean

(μ_p) and standard deviation (σ_p). The Gaussian negative log-likelihood loss is used:

$$L_{n_p} = \frac{1}{2} \log(2\pi\sigma_p^2) + \frac{(n_p - \mu_p)^2}{2\sigma_p^2}.$$

This allows the model to not only predict n_p accurately but also quantify uncertainty in its predictions. We also introduce compensatory loss terms to account for channel physics that were not explicitly captured in our previous model, such as the interdependencies between n_c and n_p . We introduce an additional Mean Squared Error (MSE) loss term:

$$L_{\text{inter}} = \text{MSE}((n_c + n_p)_{\text{pred}}, (n_c + n_p)_{\text{true}}).$$

This ensures that the model captures the inherent correlation between these two parameters. When designing the loss term for frequency band and bandwidth, the loss function for these parameters consists of:

- MSE loss for the lower bound frequency (f_l) and bandwidth (B), ensuring the model learns optimal frequency allocations.
- A regularization term to prevent excessive bandwidth selection while maintaining valid transmission ranges.

Additionally, we introduce an MSE loss for the center frequency (f_c):

$$L_{f_c} = \text{MSE}(f_{c,\text{pred}}, f_{c,\text{true}}),$$

where the predicted center frequency is computed as:

$$f_c = f_l + \frac{bw \times W}{2}.$$

In a nutshell, the total loss function is the weighted sum of all these individual losses:

$$L = \lambda_1 L_{\text{BCE}, n_c} + \lambda_2 L_{\text{BCE}, psk} + \lambda_3 L_{\text{NLL}, n_p} + \lambda_4 L_{\text{MSE}, f_l, B} + \lambda_5 L_{\text{inter}} + \lambda_6 L_{f_c}, \quad (8.6)$$

where $\lambda_1, \lambda_2, \lambda_3, \lambda_4, \lambda_5, \lambda_6$ are hyperparameters which are all set to 1 that equally balance the contribution of each loss term. By designing the loss function in this way, we ensure that the model optimally predicts modulation parameters while maintaining flexibility and robustness in dynamic underwater channel conditions.

8.4.2 Simulation results

As shown in Figure 8.5, the proposed framework systematically evaluates and selects MCSs for channel transmission by leveraging a combination of channel sounding, latent compression, MCS selection, BER-based filtering, and FEC optimization. The key steps in this process are as follows:

1. **Channel sounding:** A set of m-sequence signals is transmitted to measure the TVIR of the channel. This raw TVIR provides an essential characterization of the channel conditions.
2. **TVIR latent extraction:** An autoencoder processes the raw TVIR and extracts a TVIR latent representation, a compressed vector that captures essential channel features. This latent serves as input for MCS selection.

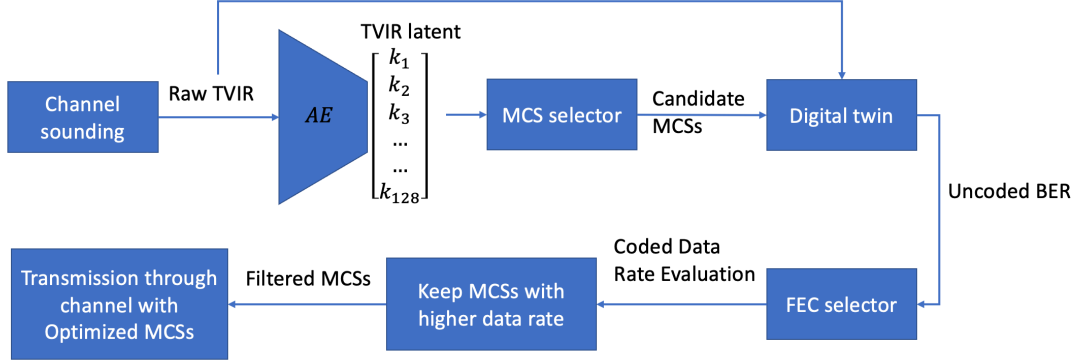


Figure 8.5: End-to-end system.

3. **MCS selector:** Based on the TVIR latent, the MCS selector proposes a set of MCS candidates which possibly achieve higher coded data rate. These selected MCSs are then verified through a simulated transmission process.
4. **Digital twin evaluation:** The proposed MCS candidates are tested through digital twin, which is modeled based on the collected raw TVIR. This step enables an offline evaluation of the BER for each candidate under realistic channel conditions.
5. **FEC selector and coded data rate calculation:** The FEC selector is introduced to determine the best FEC scheme based on the BER results obtained from the replay channel. For each MCS, the coded data rate is calculated given the BER and the uncoded data rate by Equation (8.5).
6. **MCS selection for transmission through channel:** The system ranks all tested MCSs based on their coded data rate performance from the replay channel and retains the top N MCS candidates for real modem transmission. These selected MCSs are expected to provide the highest achievable data rate while maintaining acceptable error performance.

There are 9 virtual modems available in the virtual UAC environment. We utilize modems 1101, 1102, and 1103 for our simulation tests. We begin by conducting a 1 km link tuning test between modems 1101 and 1102 and then a 2 km link tuning test between modems 1101 and 1103. Before transmitting frames, we first send a series of 1 second m-sequence signals for TVIR measurement, as shown in Figure 8.6 and Figure 8.7 for the 1 km and 2 km channels respectively. Our AE learns and reconstructs the TVIR latent.

Given the measured TVIR, we employ the digital twin for possible MCS testing, eliminating the need for real-time transmission. To make the simulation more realistic, we introduce an SNR of 6 dB, accounting for the inherent noise and distortions found in real-world communication systems. Our MCS selector then processes the compressed TVIR latent representation and outputs the optimal selection order for each MCS parameter. Following these recommendations, we configure the MCS parameters accordingly and evaluate the coded data rate performance of the 10 recommended MCSs through digital twin. The results are presented in Table 8.3 and Table 8.4 where NA means no FEC rate available for this MCS. We then try MCSs which perform well through digital twin for transmission through channel and the final coded data rates we can achieve are listed in Table 8.3 and 8.4.

The default DATA link in the Subnero modems achieves a coded data rate of 1 kbps at ranges up to 2 km, and works well in simulated channels. The link tuner in the modem consistently achieves a coded data rate of 4.5 kbps by setting $n_c = 128$, $n_p = 30$, $bw = 0.4$, $dc_0 = 5$, $psk = 4$ and FEC rate as LDPC- $\frac{1}{3}$

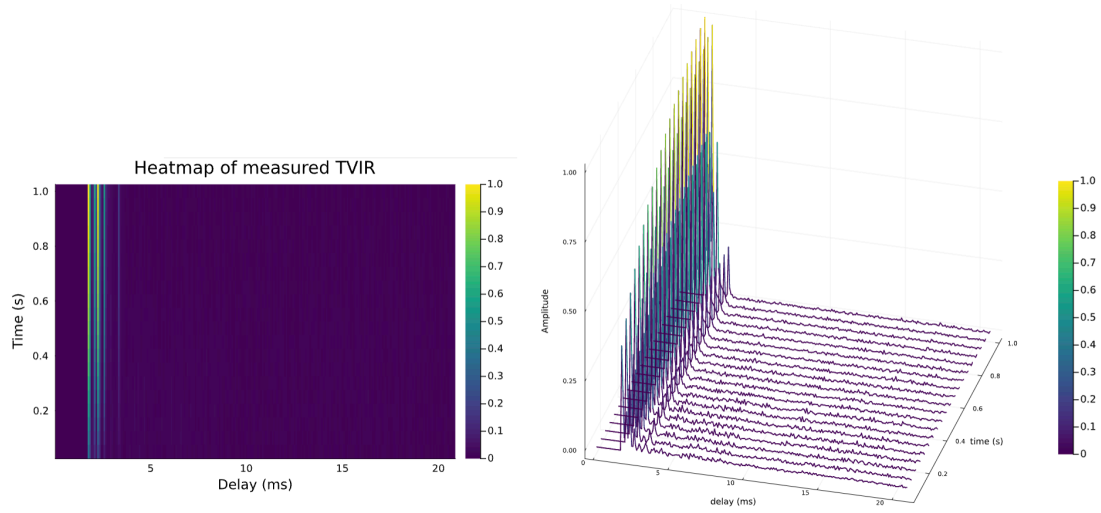


Figure 8.6: TVIR from 1 km virtual communication channel.

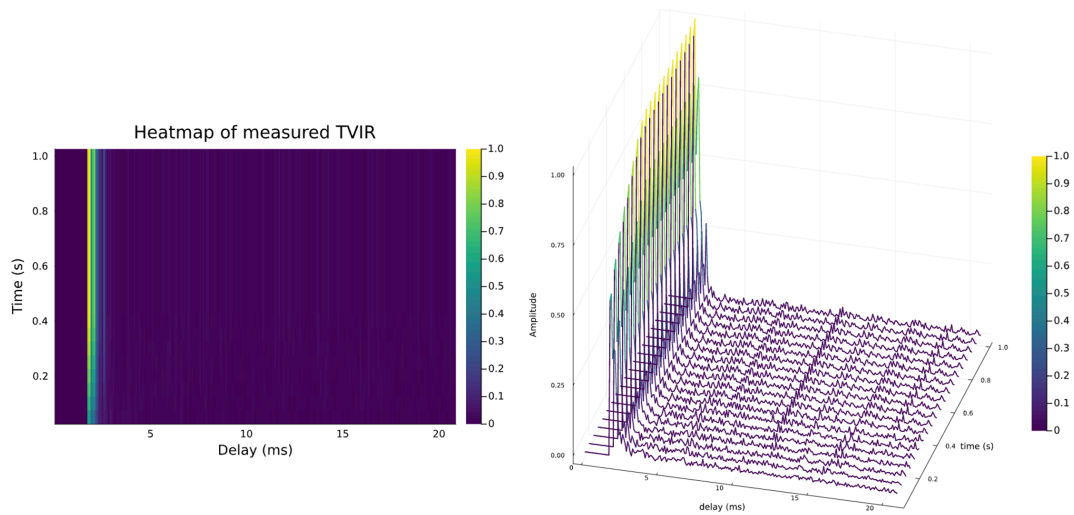


Figure 8.7: TVIR from 2 km virtual communication channel.

Table 8.3: MCS parameters and coded data rate performance through 1 km digital twin.

MCS	n_c	n_p	bw	dc_0	PSK	FEC Rate	Data Rate	BER	Coded Data Rate(bps)
1	4096	160	0.4	136	4	$\frac{1}{2}$	16394	0.0667	8197
2	8192	160	0.4	273	4	$\frac{1}{2}$	15959	0.0655	7979
3	4096	160	0.4	137	4	$\frac{1}{3}$	16394	0.1183	5464
4	8192	160	0.4	274	4	$\frac{1}{3}$	15959	0.0714	5139
5	4096	160	0.4	137	2	$\frac{1}{2}$	8391	0.0550	4195
6	8192	160	0.3	-135	2	$\frac{1}{2}$	6414	0.0542	3207
7	4096	160	0.3	-67	2	$\frac{1}{3}$	6298	0.0772	2099
8	256	160	0.4	8	2	$\frac{1}{6}$	5176	0.1793	862
9	8192	160	0.3	-136	4	NA	12829	0.2481	NA
10	8192	160	0.3	-135	4	NA	12829	0.5079	NA

Table 8.4: MCS parameters and coded data rate performance through 2 km digital twin.

MCS	n_c	n_p	bw	dc_0	PSK	FEC Rate	Data Rate	BER	Coded Data Rate(bps)
1	2048	101	0.5	7	4	$\frac{1}{2}$	20100	0.0615	10050
2	4096	101	0.5	3	4	$\frac{1}{2}$	20079	0.0649	10039
3	4096	101	0.4	3	4	$\frac{1}{2}$	16744	0.0414	8372
4	4096	101	0.4	4	4	$\frac{1}{2}$	16744	0.0585	8372
5	2048	101	0.4	8	4	$\frac{1}{2}$	16557	0.0361	8278
6	2048	101	0.5	8	4	$\frac{1}{3}$	20100	0.0723	6700
7	4096	101	0.5	4	4	$\frac{1}{3}$	20079	0.0769	6693
8	4096	101	0.5	4	4	$\frac{1}{3}$	20079	0.0731	6693
9	2048	101	0.4	9	4	$\frac{1}{3}$	16557	0.1148	5519
10	2048	101	0.5	7	2	$\frac{1}{2}$	10441	0.0642	5220

Table 8.5: MCS parameters and coded data rate performance through 1 km channel.

MCS	n_c	n_p	bw	dc_0	PSK	BER	FEC Rate	Coded Data Rate (bps)
1	4096	160	0.4	136	4	0.0427	$\frac{1}{2}$	8391
2	8192	160	0.4	273	2	0.0209	$\frac{2}{3}$	5704
3	4096	160	0.4	137	4	0.0214	$\frac{2}{3}$	5590
4	8192	160	0.3	-135	2	0.0264	$\frac{2}{3}$	4272
5	4096	160	0.3	-67	2	0.0189	$\frac{2}{3}$	4195

for 1 km simulated channel and a coded data rate of 8.8 kbps by setting $n_c = 8192$, $n_p = 30$, $bw = 0.4$, $dc_0 = 341$, $psk = 4$ and FEC rate as LDPC- $\frac{1}{2}$ for a 2 km simulated channel. In contrast, our algorithm achieves a coded data rate exceeding 8 kbps and 10 kbps in the same simulated channels, demonstrating

Table 8.6: MCS parameters and coded data rate performance through 2 km channel.

MCS	n_c	n_p	bw	dc_0	PSK	BER	FEC Rate	Coded Data Rate (bps)
1	4096	101	0.5	3	4	0.0365	$\frac{1}{2}$	10700
2	4096	101	0.5	4	4	0.0384	$\frac{1}{2}$	10700
3	2048	101	0.5	7	4	0.0380	$\frac{1}{2}$	10441
4	2048	101	0.5	8	4	0.0378	$\frac{1}{2}$	10441
5	4096	101	0.4	4	4	0.0229	$\frac{2}{3}$	10441

a significant improvement in data rate. Through repeated evaluations, we have verified that our algorithm stably achieves a better coded data rate, highlighting its robustness and adaptability.

8.5 Experiment

To validate the effectiveness of our algorithm in real-world underwater communication environments, we conducted a series of sea trials in Singapore waters. Key differences between the simulation and real-sea tests include:

- The presence of environmental noise, such as ambient ocean sounds and vessel interference.
- The impact of real-time channel variations, including Doppler shifts and multipath effects.
- The influence of hardware imperfections, such as power amplifier non-linearity and receiver sensitivity.

The testing procedure follows the framework illustrated in Figure 8.5 and channels we aim to do the link tuning are detailed in Table 8.7. Figure 8.8, Figure 8.9, and Figure 8.10 show the TVIR measurements from these 3 different channels.

Table 8.7: Real sea test setup.

Test Number	Distance (m)	Boat Movement	Condition
1	1100	TX moving, RX anchored	Long-distance communication
2	550	TX moving, RX anchored	Mid-range distance
3	320	Both boats drifting	Dynamic channel due to drift

In Test 1, the observed multipath effect is weaker compared to the shorter-range channels. This is because, in Tests 2 and 3, one modem was consistently positioned near an island with a rock wall, resulting in stronger reflections. Additionally, the background noise in real-sea conditions is significantly higher than in our virtual environments. Meanwhile, the channel in Test 1 exhibits a larger delay spread due to the increased propagation distance. The MCS candidates selected through digital twin validation for each range are listed in Table 8.8, Table 8.9, and Table 8.10. The selected MCS parameters and the final coded data rates for the three test scenarios are summarized in Table 8.11. Despite these challenges, our algorithm demonstrated high reliability and efficiency, achieving competitive coded data rates, thereby confirming the practical feasibility of our method.

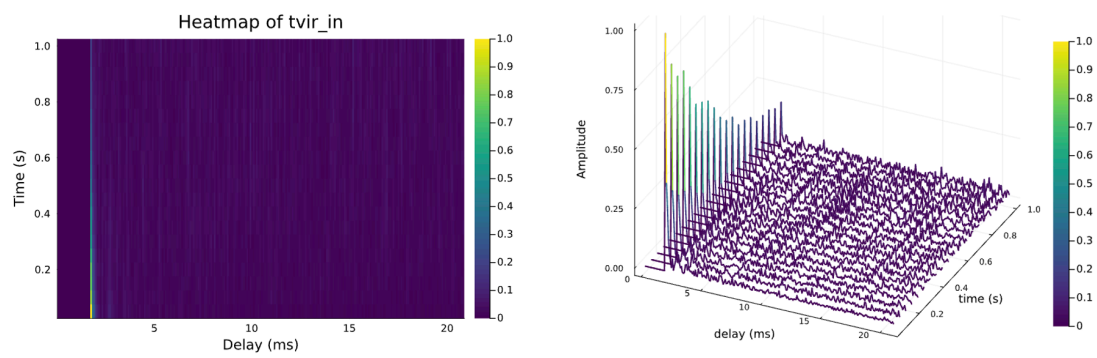


Figure 8.8: Measured TVIR for test 1 channel.

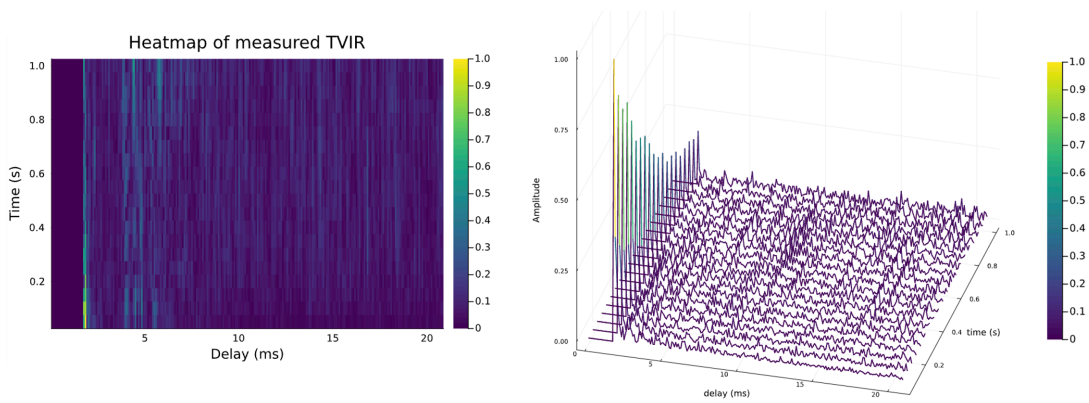


Figure 8.9: Measured TVIR for test 2 channel.

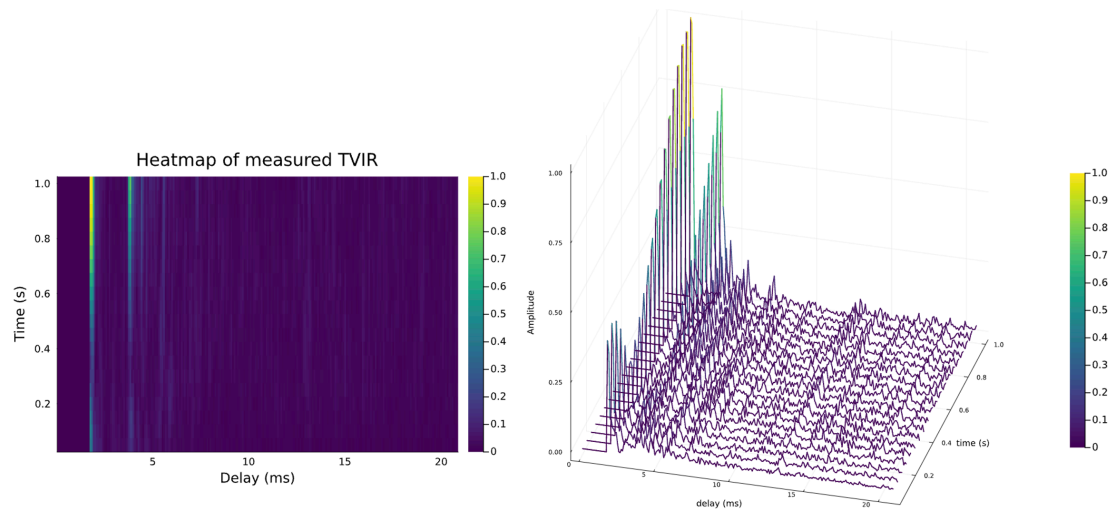


Figure 8.10: Measured TVIR for test 3 channel.

Table 8.8: MCS parameters and coded data rate performance of the channel from test 1 through digital twin.

MCS	n_c	n_p	bw	dc_0	PSK	FEC Rate	Data Rate	BER	Coded Data Rate (bps)
1	8192	1	0.4	273	4	$\frac{1}{4}$	16787	0.1428	4197
2	8192	1	0.4	274	4	$\frac{1}{4}$	16787	0.1470	4197
3	8192	1	0.3	-136	4	$\frac{1}{4}$	13298	0.1389	3325
4	8192	1	0.3	-135	4	$\frac{1}{4}$	13298	0.1367	3325
5	4096	1	0.3	-67	4	$\frac{1}{4}$	13287	0.1499	3322
6	8192	1	0.4	273	2	$\frac{1}{3}$	8869	0.0883	2956
7	8192	1	0.4	274	2	$\frac{1}{3}$	8869	0.0841	2956
8	4096	1	0.4	136	2	$\frac{1}{3}$	8851	0.0844	2950
9	4096	1	0.4	137	2	$\frac{1}{3}$	8851	0.0916	2950
10	4096	1	0.4	136	4	$\frac{1}{6}$	17374	0.1667	2896

8.6 Discussion

With 3 different link tuning approaches, we demonstrated the ability to tune link parameters at sea to achieve the following data rates:

- Direct replay: 2.7 km, 5.6 kbps.
- Diffusion replay: 2.8 km, 3.8 kbps.
- Physics-informed AMC: 1.1 km, 3.3 kbps.
- Physics-informed AMC: 550 m, 3.3 kbps.

Table 8.9: MCS parameters and coded data rate performance of the channel from test 2 through digital twin.

MCS	n_c	n_p	bw	dc_0	PSK	FEC Rate	Data Rate	BER	Coded Data Rate
1	8192	1	0.3	1100	2	$\frac{1}{2}$	6649	0.0587	3325
2	8192	1	0.3	1100	4	$\frac{1}{4}$	13298	0.1236	3325
3	4096	1	0.3	550	2	$\frac{1}{2}$	6644	0.0643	3322
4	8192	1	0.4	1638	2	$\frac{1}{3}$	8869	0.1065	2956
5	4096	1	0.4	755	4	$\frac{1}{6}$	17375	0.1746	2896
6	8192	1	0.4	1638	4	$\frac{1}{6}$	16787	0.1690	2798
7	8192	1	0.4	1510	4	$\frac{1}{6}$	16787	0.1584	2798
8	4096	1	0.4	819	4	NA	17375	0.3084	NA
9	8192	1	0.3	1228	4	NA	13298	0.5028	NA
10	4096	1	0.3	614	4	NA	13288	0.4974	NA

Table 8.10: MCS parameters and coded data rate performance of the channel from test 3 through digital twin.

MCS	n_c	n_p	bw	dc_0	PSK	FEC Rate	Data Rate	BER	Coded Data Rate
1	8192	64	0.4	902	4	$\frac{1}{3}$	16448	0.1166	5483
2	8192	64	0.3	492	4	$\frac{1}{3}$	13108	0.0977	4369
3	8192	64	0.3	484	4	$\frac{1}{3}$	13108	0.0997	4369
4	8192	64	0.4	894	4	$\frac{1}{4}$	16448	0.1278	4112
5	8192	64	0.3	484	2	$\frac{1}{2}$	6554	0.0665	3277
6	4096	64	0.3	246	4	$\frac{1}{4}$	13004	0.1238	3251
7	4096	64	0.3	242	4	$\frac{1}{4}$	13004	0.1276	3251
8	8192	64	0.4	902	2	$\frac{1}{3}$	8742	0.0819	2914
9	8192	64	0.4	894	2	$\frac{1}{3}$	8742	0.0931	2914
10	4096	64	0.4	451	2	$\frac{1}{3}$	8662	0.0982	2888

Table 8.11: Optimal MCS parameters and coded data rate performance for transmission through channel in different tests.

Test	n_c	n_p	bw	dc_0	psk	Uncoded Data Rate (bps)	BER Rate	FEC Rate	Coded Data Rate (bps)
Test 1	8192	1	0.3	-136	4	13298	0.140	$\frac{1}{4}$	3325
Test 2	4096	1	0.3	550	2	6644	0.064	$\frac{1}{2}$	3322
Test 3	4096	64	0.3	246	4	13005	0.0889	$\frac{1}{3}$	4331

- Physics-informed AMC: 320 m, 4.3 kbps.

The tests were conducted in a dynamic environment, and so the data rates achieved cannot be directly compared. However, it is clear from the results that link tuning using a digital twins and other machine learning techniques can help achieve data rates of several kbps in challenging sea conditions consistently.

The digital twin based methods are straightforward to implement in Subnero modems, but are too slow for real-time use today. Future work needs to focus on improving the real-time performance of these methods, and reducing the search space. The physics-informed AMC provides a way forward in reducing the search space. While the physics-informed AMC has shown promise, it has several limitations:

- **Training data limitations:** The AE does not see all kinds of TVIRs during training, possibly leading to limited generalization capability. This might make it difficult for the model to extract robust latent representations that capture the full diversity of underwater channel conditions. As a result, the MCS selector struggles to learn a reliable mapping from TVIR to MCS, requiring more diverse training data to improve its decision-making accuracy. Now our model can handle channels which has obvious arrivals at the first half of the TVIR.
- **Marginal-based modeling limitations:** The current approach models individual marginals between the TVIR latent and each MCS parameter separately. However, this might cause some interactions between different MCS parameters to be lost, potentially leading to suboptimal MCS selection.

Some of these limitations could be overcome by exploring new ideas in the future:

- **MCS selector model improvement:** Our current MCS selector can use the channel physics in a more efficient way to optimize the design of the model structure and lower the load to train such a model. In the future, we can explore structured priors or hybrid models that incorporate both learned representations and known physical properties of the channel.
- **Searching strategy improvement:** We currently sample each MCS parameter from the predicted distribution and determine the order of parameter adjustments based on empirical knowledge. Moving forward, we may optimize the search strategy by directly providing a series of MCS configurations instead of relying on sequential adjustments, which could lead to a more efficient selection process.
- **Improved representation learning for TVIR:** We can investigate whether the current TVIR latent sufficiently preserves CIR characteristics and employ diffusion methods to obtain a statistical TVIR representation. This approach may enhance robustness by incorporating a more expressive and generalizable latent space, allowing the model to better adapt to different underwater environments.

CHAPTER 9

SUMMARY & RECOMMENDATIONS

9.1 Research contributions & findings

The MUSIQUE project made significant theoretical and practical contributions to our understanding and capability in the field of underwater acoustic communication. The project spanned a broad range of topics from physical-layer signal processing to differentiable programming to machine learning to generative AI. Some of the key contributions and findings are summarized below.

9.1.1 Experimentation, data collection and validation of communication techniques

1. A joint US-Singapore experiment conducted in Singapore in July 2022 yielded robust coherent and incoherent communication at ranges up to 2 km. Another joint US-Singapore experiment conducted in Panama City in November 2023 provided more data to show robust and good communication performance at ranges up to 5 km.
2. A novel windowed OFDM (W-OFDM) modulation scheme was developed and tested in various experiments in the MF and VLF bands. While W-OFDM provides a lower data rate than OFDM, it is more resilient to time-variability in the channel and is a good candidate for use in challenging sea conditions. W-OFDM has been implemented and is now available on Subnero modems.
3. We developed spatial diversity combining for Subnero modems and showed very significant performance gains from it during several experiments. This is implemented and is now available on Subnero multi-channel modems.

4. We showed that link adaptation is key to high data rate communication. For example, in the Panama city experiment, we showed that a 12 kbps link could be established if we carefully chose OFDM parameters and a suitable LDPC code for the link. A simple link adaptation agent is now implemented and available on Subnero modems.

9.1.2 Enhanced performance in challenging environments using ML-based techniques

5. To aid link adaptation without the need to transmit excessively, we developed two techniques for building a digital twin of the ocean environment. One technique is based on direct replay, where a channel probe is used to measure the time-varying channel impulse response (TVIR), and subsequently the TVIR is used to optimize communication scheme performance. The other technique is based on generative AI in the form of stable diffusion models that are conditioned on the measured TVIR. This provides a richer digital twin that is able to capture the variability in the underwater channel well. The digital twin was utilized at sea for link adaptation successfully, achieving data rates of 5.6 kbps at 2.7 km in Singapore waters!
6. We developed a physics-informed machine learning model to establish the feasibility of link adaptation techniques that combine the digital twin with a compact representation of the channel in the form of a latent representation extracted using an LSTM autoencoder. The model was demonstrated at sea for link adaptation in challenging sea states.
7. We developed a novel differentiable error correction technique (we call it DFEC) that allows us to combine error correction with channel estimation. Through experimental testing, we demonstrated that the technique significantly outperforms commonly used benchmark techniques such as decision feedback equalization (DFE) combined with state of art error correction codes. While the technique is still too slow for real-time implementation on modems, we expect that its performance can be improved, and that it will become a core building block of future communication systems.

9.1.3 Enhanced capacity for multiuser communication networks

8. MIMO is a technique that could theoretically be used to increase data rates by using multiple transmitters. Practical use of MIMO for underwater communication is, however, challenging due to strict requirements of synchronization, Doppler compensation, etc. We demonstrated that MIMO can indeed be used in the MF band to increase data rate by 50% with 3 transmitters under favorable channel conditions.
9. We developed beamforming based blind source separation (BSS) techniques that may allow us to listen to transmissions from multiple modems simultaneously, reducing the load on a medium access controller (MAC) to avoid collisions. For FH-BFSK frames, we were able to receive from two modems simultaneously with very little frame loss. For OFDM frames, our BSS technique was able to improve the number of successful frames by almost $3\times$, although that still leaves us with plenty of room for improvement.

The MUSIQUE project has many experiments in different frequency bands, locations, ranges, and using different techniques. While the detailed results from the experiments are included in the deliverables of the project, we highlight some of the key results in Table 9.1. The results show data rates up to 12 kbps at 2 km and 1.8 kbps at 5 km with spatial diversity combining in US waters, 7.8 kbps at 700 m with spatial diversity combining in Singapore waters, 7.5 kbps at 1.5 km with MIMO OFDM in Singapore waters, and 5.6 kbps at 2.7 km with link tuning in Singapore waters. Contrast these against the previous benchmark of 1 kbps at 2 km in Singapore waters. Data rates vary significantly with channel conditions, and so the

Table 9.1: Highlights of selected experimental results during the MUSIQUE project.

Date	Location	Frequency	Range	Technique	Data rate
Jul 2022	Selat Pauh	25–33 kHz	700 m	OFDM	3.9 kbps
			1 km	OFDM	2.2 kbps
			2 km	W-OFDM	600 bps
Jul 2022	Selat Pauh	25–33 kHz	700 m	SIMO OFDM	7.8 kbps
			1 km	SIMO OFDM	4.6 kbps
			2 km	SIMO W-OFDM	1.8 kbps
Nov 2023	Panama City	25–33 kHz	500 m	OFDM	6.7 kbps
			1 km	OFDM	6.6 kbps
			2 km	OFDM	7.8 kbps
			5 km	W-OFDM	1.4 kbps
Nov 2023	Panama City	25–33 kHz	500 m	SIMO OFDM	9.5 kbps
			1 km	SIMO OFDM	9.2 kbps
			2 km	SIMO OFDM	11.6 kbps
			5 km	SIMO OFDM	1.8 kbps
Feb 2025	Kusu	20–30 kHz	750 m	OFDM	4.7 kbps
			1.5 km	OFDM	5 kbps
Feb 2025	Kusu	20–30 kHz	750 m	SIMO OFDM	4.7 kbps
			1.5 km	SIMO OFDM	5.2 kbps
Feb 2025	Kusu	20–30 kHz	750 m	MIMO OFDM	7.1 kbps
			1.5 km	MIMO OFDM	7.5 kbps
Feb 2025	Kusu	4.5–6.5 kHz	2.5 km	OFDM	556 bps
			2.5 km	W-OFDM	333 bps
Feb 2025	Kusu	20–30 kHz	2.7 km	Replay AMC	5.6 kbps
			2.8 km	Diffusion AMC	3.8 kbps
			1.1 km	Physics AMC	3.3 kbps
			550 m	Physics AMC	3.3 kbps

data rates in different conditions may differ. Yet, the results obtained show the potential for the techniques developed in MUSIQUE, and we are very excited about them!

9.2 Recommendations & future directions

While the results in the last section are exciting, we must remember that many of these results involve careful choice of MCS, and in some cases need multiple hydrophones or transducers. What we eventually desire is a communication system that simply works! The intelligence needed to tune the MCS should not have to reside with the operator, but should be baked into the modem. We should be able to confidently deploy a modem and have it provide the best performance possible for a given environment, within the

constraints of the physics of that environment. The following recommendations are directed towards getting closer to this grand goal:

1. It has been shown that link adaptation is key to good communication performance in challenging environments. The AMC algorithms developed in MUSIQUE lay the foundation for more work on this topic. We need to refine the algorithms for speed and accuracy, integrate more physics guidance in them, and train them with more data to ensure that they generalize to unseen environments.
2. We should use communication signals to extract as much physics information as possible opportunistically. This information should be tracked, so that it is available for link adaptation algorithms to use, without the need to actively transmit to measure the acoustic channel.
3. Link adaptation has to be a continuous process that monitors each transmission and adapts link parameters dynamically. The current algorithms perform adaptation once, and then use the adapted parameters for data transfer. Protocols need to be developed to enable data transfer to happen in parallel with adaptation, with minimal overhead for adaptation.
4. The work on generative AI as a digital twin is key to rapid link adaptation. Rather than transmit signals in the real world, the digital twin allows adaptation by performance measurement in the virtual world. However, this process is slow and presents a key bottleneck to adaptation. We need faster algorithms for generating TVIRs, and we need them to be better trained to generalize to new environments. Currently the autoencoders and diffusion models we use need to be fine tuned using data from a target environment. We need ways to reduce the need for fine tuning, potentially by training the models on much larger datasets than we have today.
5. In order to incorporate physics guidance into link adaptation, we need better data driven acoustic models that are constrained by physics. Such models will allow a few channel measurements to provide us sufficient information about the environment to extrapolate and allow us to predict performance as our vehicles and modems move to previously unexplored places in the environment.
6. While the data rates demonstrated using differentiable error correction are modest, the benefits over separately equalized and decoded systems were clearly demonstrated in this project. This opens the doors to integration of channel estimation, equalization, demodulation and error correction into a single algorithm. Traditionally, these algorithms have been treated separately to manage complexity. But the separation is sub-optimal and combining them together will allow for more performant physical layer algorithms than we have today. We have demonstrated the potential with DFEC BPSK in this project, but that just scratches the surface of what may be possible. For example, DFEC may also be combined with equalized OFDM to reduce or remove the need for cyclic prefix and hence improve data rates. A deeper exploration into how communication systems can be reformulated in light of differentiable programming is required to fully harness the power of this new technology.
7. While MIMO OFDM has been demonstrated to be feasible in MUSIQUE, this was achieved using customized modems with multiple analog frontends for transmission. Should an operational MIMO system be desired, engineering effort needs to be put in into developing a simple-to-use MIMO modem, and some parts of UnetStack need to be redesigned to handle multichannel transmission and MIMO processing at reception.
8. The BSS algorithms explored so far have a few shortcomings. If two nodes transmit from similar directions, the algorithms cannot separate the signals. The algorithms also destroy the orthogonality of OFDM signals, making high speed DATA collisions unrecoverable. By incorporating some information about the signal class into the algorithms, it will be possible to improve the performance of BSS. When we foresee the need for underwater acoustic networks with multiple nodes, we should invest

in progressing from BSS to using some knowledge of communication signals for source separation and multiple access.

REFERENCES

- [1] Baosheng Li, Jie Huang, Shengli Zhou, Keenan Ball, Milica Stojanovic, Lee Freitag, and Peter Willett. Mimo-ofdm for high-rate underwater acoustic communications. *IEEE journal of oceanic engineering*, 34(4):634–644, 2009.
- [2] Tadashi Ebihara, Geert Leus, and Hanako Ogasawara. Underwater acoustic communication using doppler-resilient orthogonal signal division multiplexing in a harbor environment. *Physical Communication*, 27:24–35, 2018.
- [3] Tadashi Ebihara, Hanako Ogasawara, and Geert Leus. Underwater acoustic communication using multiple-input–multiple-output doppler-resilient orthogonal signal division multiplexing. *IEEE Journal of Oceanic Engineering*, 45(4):1594–1610, 2019.
- [4] Roald Otnes, Paul A van Walree, and Trond Jensenrud. Validation of replay-based underwater acoustic communication channel simulation. *IEEE Journal of Oceanic Engineering*, 38(4):689–700, 2013.
- [5] Francois-Xavier Socheleau, Christophe Laot, and Jean-Michel Passerieux. Stochastic replay of non-wssus underwater acoustic communication channels recorded at sea. *IEEE Transactions on Signal Processing*, 59(10):4838–4849, 2011.
- [6] Yong Yu, Xiaosheng Si, Changhua Hu, and Jianxun Zhang. A review of recurrent neural networks: Lstm cells and network architectures. *Neural computation*, 31(7):1235–1270, 2019.
- [7] PyTorch. Available at: <https://github.com/org-ar1/UnderwaterAcoustics.jl>, Accessed: 2025-02-03, Accessed: 2025-02-03.
- [8] ChiYan Lee, Hideyuki Hasegawa, and Shangce Gao. Complex-valued neural networks: A comprehensive survey. *IEEE/CAA Journal of Automatica Sinica*, 9(8):1406–1426, 2022.
- [9] UnderwaterAcoustics.jl. Available at: https://org-ar1.github.io/UnderwaterAcoustics.jl/v0.1/pm_basic.html#Propagation-modeling-toolkit, Accessed: 2025-02-03.
- [10] Parastoo Qarabaqi and Milica Stojanovic. Acoustic channel simulator. *Northeastern University, Boston, MA*, 2115, 2013.

- [11] AdaptiveEstimators.jl. Available at: <https://github.com/org-arl/AdaptiveEstimators.jl>, Accessed: 2025-02-03.
- [12] M. Stojanovic. Underwater acoustic communications: Design considerations on the physical layer. In *2008 Fifth Annual Conference on Wireless on Demand Network Systems and Services*, pages 1–10, 2008.
- [13] Bo Xu, Xiaoyu Wang, Yu Guo, Jiao Zhang, and Asghar Abbas Razzaqi. A novel adaptive filter for cooperative localization under time-varying delay and non-gaussian noise. *IEEE Transactions on Instrumentation and Measurement*, 70:1–15, 2021.
- [14] A. Radosevic, R. Ahmed, T. M. Duman, J. G. Proakis, and M. Stojanovic. Adaptive OFDM modulation for underwater acoustic communications: Design considerations and experimental results. *IEEE Journal of Oceanic Engineering*, 39(2):357–370, 2014.
- [15] Pengfei Xia, Shengli Zhou, and G.B. Giannakis. Adaptive mimo-ofdm based on partial channel state information. *IEEE Transactions on Signal Processing*, 52(1):202–213, 2004.
- [16] D.J. Love and R.W. Heath. Limited feedback power loading for ofdm. In *IEEE MILCOM 2004. Military Communications Conference, 2004.*, volume 1, pages 71–77 Vol. 1, 2004.
- [17] L. Huang, Q. Zhang, L. Zhang, J. Shi, and L. Zhangb. Efficiency enhancement for underwater adaptive modulation and coding systems: via sparse principal component analysis. *IEEE Communications Letters*, pages 1–1, 2020.
- [18] Wu Shuangshuang, Prasad Anjani, and Mandar Chitre. Adaptive modulation and feedback strategy for an underwater acoustic link. In *2022 Sixth Underwater Communications and Networking Conference (UComms)*, pages 1–5, 2022.
- [19] Mandar Chitre. *Underwater Acoustic Communications in Warm Shallow Water Channels*. PhD thesis, National University of Singapore, Singapore, 2006. See p. 97 for details.
- [20] UnetStack3: the underwater networks project. <https://unetstack.net/>. Accessed: 2022.